

AI AGENT 入门到精通

# 一步一步学习小龙虾

OpenClaw 从入门到精通系列

全系列 15 篇 · 约 6 小时阅读

作者：一位 45 岁程序员原创

系统覆盖：新手篇 → 进阶篇 → 原理篇 → 企业级部署

# 目 录

---

|     |                           |      |                  |
|-----|---------------------------|------|------------------|
| 第1篇 | OpenClaw是什么？5分钟看懂AI Agent | 第9篇  | 工具与API集成         |
| 第2篇 | 10分钟部署自己的AI助手             | 第10篇 | 进阶篇总结            |
| 第3篇 | 10个实用任务，让AI替你打工           | 第11篇 | 核心循环：感知-思考-行动-反思 |
| 第4篇 | 基础配置指南                    | 第12篇 | 记忆系统             |
| 第5篇 | 默认Skills与管理               | 第13篇 | 架构解析             |
| 第6篇 | 第一个自定义Skill               | 第14篇 | 企业级部署            |
| 第7篇 | Agent工作流                  | 第15篇 | 原理篇总结            |
| 第8篇 | 多Agent协作                  | 尾声   | 全系列总结            |

## 第1篇 OpenClaw是什么？5分钟看懂AI Agent

---

阅读时间：5分钟 难度等级：★ 零基础 你将收获：理解OpenClaw是什么，能做什么，为什么这么火

### 开篇：一个普通上班周的"意外转折"

周一早上，小王照例9点到公司。

打开电脑，50封未读邮件在闪烁。

桌上堆着上周没处理完的Excel报表。

老板的微信又来了："今天下班前给我月度分析。"

小王叹了口气。又是加班的一天。

直到中午，他在茶水间遇到了老李。

"你脸色这么差？"老李端着咖啡问。

"别提了，这周又要加班..."

"听说过龙🐉吗？"

"龙🐉？吃的？"小王一脸懵。

"不是吃的！是OpenClaw，AI助手！"老李神秘地笑了笑，"我上个月开始用，现在准时下班，老婆都怀疑我失业了。"

小王半信半疑。但当天下午，他还是试了试。

3天后，周五下午5点。

小王收拾东西准备下班。

同事小张路过："哎？你今天不加班？"

"不用啊，这周的工作都做完了。"

"怎么可能？！那些Excel、邮件、周报..."

"龙🐉帮我搞定了。"

小张愣住："什么龙🐉？"

小王笑了："来，我给你讲讲..."

## 一、龙🐉不是吃的，是你的"数字员工"

### 1.1 别被名字吓到

"龙🐉"是 OpenClaw 的昵称。

为什么叫龙🐉？因为它的logo是个红色的钳子。

就像苹果公司的标志是个苹果一样简单。

那OpenClaw到底是什么？

一个能真正干活的AI助手。

### 1.2 它和ChatGPT有什么区别？

ChatGPT：

• 你问："怎么整理文件？" • 它答："你可以先分类，再建立文件夹..." • 结果：还是得你自己动手

OpenClaw：

• 你说："帮我把桌面整理一下" • 它说："好的，正在整理..." • 结果：10秒后，桌面真的整洁了！

核心区别：

ChatGPT给你建议，OpenClaw帮你干活。

### 1.3 想象一下

如果你有个员工：

• 📧 你说"整理邮件"，它就去整理 • 📊 你说"做报表"，它就去制作 • 📁 你说"归档文件"，它就去分类 • 📝 你说"写周报"，它就去撰写

24小时待命，从不抱怨，不犯错，不要求加薪。

这不是科幻，这是OpenClaw。

## 二、为什么2026年突然火了？

### 2.1 一个数字引发的震动

2026年3月，一个数字刷屏了：



2000万

这是OpenClaw（龙爪）在全球的部署数量。

短短3个月，从0到2000万。

相当于：

• 每天新增 22万个用户 • 每小时新增 9000个用户 • 每分钟新增 150个用户

为什么这么疯狂？

## 2.2 大厂集体"杀入"

腾讯 推出了QClaw，直接在微信里用。

百度 推出了DoctorClaw，医疗AI助手。

阿里 成立了Token事业群，All in Agent。

字节、360、华为...都在布局。

就连黄仁勋都在GTC大会上说：

"AI的下一个时代，是Agent时代。"

## 2.3 真实的"逆袭"故事

案例1：自媒体博主阿鱼

3个月前：

• 月入8000 • 手搓5个账号累成狗 • 加班到凌晨

现在：

• 指挥36只龙爪 运营32个账号 • 日产160条内容 • 月入3.2万

她说："以前加班到凌晨，现在凌晨3点我在睡觉，龙爪在干活。"

案例2：外贸员小陈

以前：

• 白天回邮件，晚上跟时差 • 周末还在加班

现在：

• 12只龙爪 分驻侦察、触达、跟进、报价 • 销售额翻3倍

他说："第一次觉得，我是个老板，不是打杂的。"

### 案例3：上班族老张

以前：

- 每周工作60小时 • 陪家人的时间不到10小时

现在：

- 4只龙猫盯理财、管待办、维护社交 • 每周偷回10小时

他说："上周五准时下班接女儿放学，她愣了一下，问'爸爸今天怎么这么早？'那一刻，我差点哭了。"

这些不是广告，是真实发生的故事。

而故事的主角，就是你身边的路人甲。

## 三、它能帮你做什么？

### 3.1 场景1：文件整理

从"乱室佳人"到"极简生活"

你的桌面现在是什么样的？

- 50个文件散落各处 • 截图、文档、表格混在一起 • 找个文件要翻半天

对龙猫说：

帮我把桌面按类型整理，

图片放Images，文档放Documents，其他放Others

10秒后：

✅ 整理完成！

- 图片：23个 → Images/
- 文档：18个 → Documents/
- 其他：9个 → Others/

桌面清爽度：★★★★★

你的感受：

"卧槽，真的整理好了？！"

### 3.2 场景2：数据处理

从"加班做表"到"准时下班"

周五下午4点，老板发来消息：

"把Q1的销售数据做个分析，下班前给我。"

你打开Excel：

• 3000行数据 • 10个sheet • 要做对比、图表、总结

以前：

• 熬夜到9点 • 眼睛花了，脖子酸了 • 图表还做错了

现在：

龙🐉，帮我分析这个Excel，  
做个销售对比图表，生成一页总结。

5分钟后：

✅ 分析完成！

- Q1销售额：1287万，同比+23%
- Top3产品：A系列(32%) B系列(28%) C系列(19%)
- 图表：已生成折线图、饼图、柱状图
- 总结：已生成1页报告

附件：Q1销售分析报告.pdf

你：

发给老板，收拾东西，5点半准时下班。

### 3.3 场景3：内容创作

从"灵感枯竭"到"产能爆炸"

你是自媒体运营：

每天要发3篇内容，选题、写稿、排版...

焦头烂额，灵感枯竭。

对龙🐉说：

帮我搜索最近一周AI领域的热点，  
整理10个选题，每个选题给3个切入点。

2分钟后：

✅ 选题整理完成！

## 1. OpenAI发布GPT-5：性能提升3倍

- 切入点1：实测对比GPT-4，提升在哪？
- 切入点2：对普通用户意味着什么？
- 切入点3：国产大模型如何应对？

## 2. 腾讯推出QClaw：微信里就能用AI

- 切入点1：手把手教你3分钟上手
- 切入点2：和ChatGPT比谁更强？
- 切入点3：未来每个人都会有个AI助手

... (还有8个选题)

你：

3分钟后：

✅ 文章已生成！

标题：《微信里藏了个AI助手？3分钟教你用上腾讯QClaw》

字数：1523字

风格：口语化、接地气

配图建议：3张

正文："昨天，我在微信里发现了个宝藏..."

[完整文章内容]

你：

### 3.4 场景4：邮件管理

从"邮件奴隶"到"邮件主人"

你每天收50+封邮件：

- 客户询价 • 同事协作 • 推广广告 • 系统通知

看邮件花1小时，回邮件又1小时。

现在：

龙🐉，帮我分类今天的邮件，

标记重要的，普通邮件自动回复。

3分钟后：

✅ 邮件整理完成!

🔴 重要邮件 (5封):

1. 客户张总: 产品报价咨询 → 需要回复
2. 老板: 会议时间调整 → 已标注日程
3. 财务: 报销审批通过 → 已归档

🟡 普通邮件 (32封):

- 已自动回复: 19封 • 已归档: 13封

📧 邮件摘要:

今天主要涉及: 项目进度、客户沟通、内部协作  
建议优先处理: 客户张总的报价 (2小时内回复)

你:

只看5封重要邮件, 其他自动搞定。  
省下1小时, 做更有价值的事。

### 3.5 场景5: 自动化

从"每天重复"到"一劳永逸"

你每天都要做的事:

- 早上9点: 查看待办事项 • 下午5点: 备份今天的工作文件 • 晚上8点: 整理明天的日程

以前: 手动操作, 容易忘。

现在:

🐉, 帮我设置自动化任务:

1. 每天早上9点推送待办事项
2. 每天下午5点自动备份工作文件夹到云盘
3. 每天晚上8点整理明天的日程

设置一次, 天天自动执行。

你:

再也不用担心忘记备份了。  
就像雇了个秘书, 准时提醒你。

## 四、谁最适合用？

### 4.1 如果你是个上班族...

痛点：

- 重复性工作占**50%**时间 • 加班成为常态 • 效率上不去，晋升无望

**龙**能帮你：

- ☒ 自动处理Excel、PPT、Word • ☒ 自动整理邮件、文件、待办 • ☒ 自动生成周报、月报、总结

效果：

- 每周节省10-**20小时** • 准时下班，陪伴家人 • 把时间花在更有价值的事上

### 4.2 如果你是个自媒体...

痛点：

- 内容生产慢，**1天**写不出**1篇** • 多平台分发，复制粘贴到崩溃 • 选题枯竭，不知道写什么

**龙**能帮你：

- ☒ 批量生成内容（日产10-**50篇**） • ☒ 自动多平台发布 • ☒ 智能选题推荐

效果：

- 产能翻**10倍** • 一个人运营**10个**账号 • 收入翻**3-5倍**

### 4.3 如果你是个外贸/销售...

痛点：

- 客户跟进慢，丢单 • 邮件回复不及时，时差问题 • 资料整理混乱，找不到

**龙**能帮你：

- ☒ 自动跟进客户（定时提醒、自动回复） • ☒ 智能邮件管理（分类、标记、回复） • ☒ 客户资料整理（CRM自动化）

效果：

- 客户响应速度提升**3倍** • 成交率提升**20%** • 销售额翻**2-3倍**

### 4.4 如果你是个学生...

痛点：

- 资料查找慢，写论文要查一周
- 笔记不成体系，考前找不到
- 文献综述写不出来

🐉 能帮你：

- ☒ 快速文献检索（自动筛选、整理）
- ☒ 自动整理笔记（生成知识图谱）
- ☒ 生成文献综述（引用、格式自动）

效果：

- 写论文效率提升**5倍**
- 复习效率提升**3倍**
- 从学渣变学霸

## 五、怎么开始？3种方式

方式1：微信小程序（最快，**2分钟**）

适合：完全新手，想马上体验

步骤：

1. 打开微信
2. 搜索"QClaw"
3. 授权登录
4. 开始使用

优点：0门槛，免费体验

缺点：功能简单，不能操作本地文件

方式2：桌面应用（推荐，**5分钟**）

适合：日常使用，需要处理本地文件

步骤：

1. 访问 [openclaw.ai](https://openclaw.ai) 下载
2. 双击安装（Windows/Mac）
3. 首次配置
4. 开始使用

优点：功能完整，响应快

缺点：需要下载安装

方式3: 命令行 (极客, 5分钟)

适合: 开发者, 喜欢折腾

步骤:

```
brew install openclaw
```

```
openclaw init
```

```
openclaw start
```

优点: 完全自定义, 最新版本

缺点: 有学习曲线

## 六、费用与安全

### 6.1 费用: 大部分免费

完全免费:

- ☒ 开源版 (GitHub下载)
- ☒ 小程序基础功能
- ☒ 本地部署

可能收费:

- 💰 大厂产品的高级功能
- 💰 云端API调用 (很便宜)

新手建议:

先用免费版体验  
够用了就别付费

### 6.2 安全: 相对安全

开源版:

- ☒ 代码公开, 可审计
- ☒ 本地运行, 数据不上传
- ☒ 完全控制权限

大厂产品:

- ☒ 有安全合规
- ⚠️ 数据会上传云端

安全建议:

敏感数据用本地部署  
不要在对话中输入密码  
定期清理历史记录



## 七、小结：记住这3点

### 1 OpenClaw = 能干活的AI

不是只会聊天的AI，是能真正执行任务的AI。

### 2 能帮你节省50%时间

文件整理、数据处理、邮件管理、内容创作...

这些重复性工作，交给**龙皇**。

### 3 0基础就能用

微信小程序**2分钟**上手

桌面应用**5分钟**安装

自然语言控制，不用学编程

## 下一步：动手试试

纸上得来终觉浅，绝知此事要躬行。

👉 看下一篇：《**10分钟**部署你的第一只**"龙皇"**》

手把手教你安装，完成第一个任务，获得成就感！

## 互动时间

想一想：

你每天的工作中，有哪些**重复性**、**机械性**的任务？

写下来（至少**3个**）：

1. \_\_\_\_\_
2. \_\_\_\_\_
3. \_\_\_\_\_

评论区告诉我，我帮你设计自动化方案！ 🚀

## 系列导航

• 上一篇：无（这是第**1篇**） • 下一篇：**10分钟**部署你的第一只**"龙皇"**

本文是《OpenClaw从入门到精通》系列第1篇

作者：生活助理 | 发布时间：2026-03-20

---

《一步一步学习小龙虾》

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师

## 第2篇 10分钟部署自己的AI助手

---

阅读时间：10分钟 难度等级：★ 零基础 你将收获：完成OpenClaw安装，执行第一个任务，获得成就感

### 准备工作

电脑要求（很低）

最低配置：

- 操作系统：Windows 10+ / macOS 10.14+ / Linux
- 内存：4GB+
- 硬盘：2GB可用空间
- 网络：需要联网（首次下载）

推荐配置：

- 内存：8GB+
- CPU：4核+

账号准备

根据你的选择，准备：

- 微信账号（小程序方式）
- GitHub账号（开源版）
- 邮箱（注册用）

### 3种安装方式

🚀 方式一：微信小程序（最快，0门槛）

适合人群：完全新手，想最快体验

优点：

- ✅ 0门槛，不用安装
- ✅ 随时随地使用
- ✅ 免费体验

缺点：

- ❌ 功能相对简单
- ❌ 不能操作本地文件
- ❌ 需要联网

安装步骤（2分钟）

Step 1: 打开微信

在手机或电脑上打开微信

Step 2: 搜索小程序

在搜索框输入：QClaw

### Step 3: 点击进入

找到"QClaw智能助手", 点击进入

### Step 4: 授权登录

点击"微信授权登录", 确认授权

### Step 5: 开始使用

进入主界面, 看到输入框, 可以开始用了!

✅ 安装完成!

### 方式二: 桌面应用 (推荐)

适合人群: 日常使用, 需要处理本地文件

优点:

• ✅ 功能完整 • ✅ 可操作本地文件 • ✅ 响应更快 • ✅ 界面友好

缺点:

• ❌ 需要下载安装 • ❌ 占用一定硬盘空间

Windows安装 (5分钟)

#### Step 1: 下载安装包

访问官网: [openclaw.ai/download](https://openclaw.ai/download)

点击"Windows下载", 选择:

• 标准版 (推荐): 500MB, 功能完整 • 精简版: 200MB, 基础功能

#### Step 2: 运行安装程序

找到下载的 [OpenClaw-Setup.exe](#), 双击运行

#### Step 3: 选择安装位置

默认: C:\Program Files\[OpenClaw](#) 建议: D:\[OpenClaw](#) (如果C盘空间不足)

#### Step 4: 选择组件

✅ OpenClaw核心 (必须)    ✅ Python运行时 (必须)    ✅ 常用工具包 (推荐)    ☐ 开发工具 (按需)

#### Step 5: 等待安装

安装进度条，大约2-3分钟

## Step 6: 首次配置

安装完成后，自动打开配置向导：

1. 选择语言：中文

2. 选择模型：

**国际模型：** • GPT-5（最新，最强，需要API Key） • GPT-4o（推荐，性价比高） • Claude 3.5（推理能力强）

**国内模型：** • 智谱GLM-4（国产最强，推荐） • 文心一言4.0（百度出品） • 通义千问（阿里出品） • DeepSeek（性价比高）

**本地模型：** • Llama 3（免费，需配置） • Qwen（免费，中文友好）

1. 输入API Key： • 如果有OpenAI API Key，输入 • 如果有智谱/百度/阿里账号，输入 • 如果没有，选择"稍后配置"

2. 完成：点击"开始使用"

✅ 安装完成！

macOS安装（5分钟）

## Step 1: 下载DMG

访问官网：[openclaw.ai/download](https://openclaw.ai/download)

点击"macOS下载"

## Step 2: 打开DMG文件

找到 [OpenClaw.dmg](#)，双击打开

## Step 3: 拖拽安装

将OpenClaw图标拖到"Applications"文件夹

## Step 4: 首次打开

打开"应用程序"文件夹，找到OpenClaw，双击打开

⚠️ 注意：

首次打开可能提示"无法验证开发者"，解决方法：

1. 右键点击OpenClaw

2. 选择"打开"

3. 点击"打开"确认

Step 5: 配置 (同Windows)

✅ 安装完成!

---

☰ 方式三：命令行 (极客)

适合人群：开发者，喜欢折腾

优点：

- ✅ 完全自定义
- ✅ 可编程控制
- ✅ 最新版本

缺点：

- ❌ 有学习曲线
- ❌ 需要技术背景

macOS/Linux安装 (5分钟)

Step 1: 安装依赖

## macOS

```
brew install python3 git
```

## Ubuntu/Debian

```
sudo apt install python3 python3-pip git
```

## CentOS/RHEL

```
sudo yum install python3 python3-pip git
```

Step 2: 安装OpenClaw

## 方式A: Homebrew (推荐)

```
brew install openclaw
```

## 方式B: pip安装

```
pip3 install openclaw
```

## 方式C：源码安装

```
git clone https://github.com/openclaw/openclaw.git cd openclaw pip3  
install -r requirements.txt python3 setup.py install
```

Step 3：初始化

## 初始化配置

```
openclaw init
```

会提示：？请选择默认模型 – GPT-5（需要OpenAI API Key） – GPT-4o  
（推荐，性价比高） – GLM-4（智谱，国内推荐） – 文心4.0（百度） –  
通义千问（阿里） – 本地模型（免费）？请输入API Key（可选）？请选择数  
据存储位置 [默认: ~/.openclaw]

Step 4：启动服务

## 启动后台服务

```
openclaw start
```

## 查看状态

```
openclaw status
```

输出：✅ OpenClaw运行中 – 版本: 1.0.0 – 模型: GLM-4 – 端口: 8080

Step 5：测试

## 命令行交互

```
openclaw chat
```

## 或者直接执行任务

```
openclaw run "列出当前目录的所有PDF文件"
```

✅ 安装完成！

Windows命令行安装

Step 1：安装Python

访问 [python.org/downloads/](https://python.org/downloads/)，下载Python 3.9+

安装时勾选"Add [Python](#) to PATH"

Step 2: 安装OpenClaw

## 使用pip

pip install [openclaw](#)

## 或使用scoop

scoop install [openclaw](#)

Step 3: 初始化

[openclaw](#) init

Step 4: 启动

[openclaw](#) start

✅ 安装完成!

---

### 第一个任务：整理桌面

任务描述

场景：桌面乱糟糟，有各种文件，想按类型整理

目标：让OpenClaw自动整理桌面文件

方式一：图形界面操作

Step 1: 打开OpenClaw

双击桌面图标，或从开始菜单打开

Step 2: 进入对话界面

看到主界面，有一个输入框

Step 3: 输入任务

在输入框输入：

帮我把桌面上的文件按类型整理到不同文件夹。 图片放Images文件夹，文档放Documents文件夹， 代码放Code文件夹，其他放Others文件夹



## Step 4: 点击"执行"

点击"发送"或按回车

## Step 5: 观察执行过程

OpenClaw会显示执行过程:

🤖 正在分析任务... 📁 发现桌面文件: 15个

执行步骤: 1. 扫描桌面文件 2. 创建分类文件夹 3. 移动文件

正在执行... ✅ 创建文件夹: Images ✅ 创建文件夹: Documents ✅ 创建文件夹: Code ✅ 创建文件夹: Others ✅ 移动文件: screenshot.png → Images/ ✅ 移动文件: report.docx → Documents/ ...

## Step 6: 查看结果

大约10秒后, 显示:

✅ 整理完成!

🇺🇸 统计: • 图片文件: 5个 → Images/ • 文档文件: 6个 → Documents/ • 代码文件: 2个 → Code/ • 其他文件: 2个 → Others/

总计处理: 15个文件 用时: 10.3秒

## Step 7: 检查桌面

最小化OpenClaw, 看看桌面...

哇! 桌面清爽了! ✨

## 方式二: 命令行操作

### Step 1: 打开终端

Windows: PowerShell macOS: Terminal

### Step 2: 输入命令

`openclaw` run "整理桌面文件, 按类型分类"

### Step 3: 等待执行

🤖 执行任务: 整理桌面文件...

步骤1: 扫描桌面 找到 15 个文件

步骤2: 创建文件夹 ✓ Images ✓ Documents ✓ Code ✓ Others

步骤 3：移动文件 ✓ screenshot.png → Images/ ✓ report.docx → Documents/ ...

✅ 完成！ 处理文件：15个 用时：9.8秒

✅ 任务完成！

---

### 方式三：小程序操作

#### Step 1: 打开QClaw小程序

微信 → 搜索"QClaw" → 进入

#### Step 2: 输入任务

生成一个Python脚本，用于整理桌面文件

#### Step 3: 获得脚本

OpenClaw会生成代码（此处省略，实际会显示完整代码）

#### Step 4: 复制保存

复制代码，保存为 organize\_desktop.py

#### Step 5: 运行脚本

`python organize_desktop.py`

✅ 桌面整理完成！

---

### 常见问题解决

#### 问题1: 安装失败

症状：

- 安装进度卡住 • 提示"安装失败" • 打不开应用

解决方案：

Windows：

1. 右键安装包 → 属性 → 解除锁定
2. 以管理员身份运行
3. 关闭杀毒软件后重试

macOS：

1. 系统偏好设置 → 安全性与隐私 → 允许OpenClaw

2. 右键点击 → 打开 → 打开

Linux:

## 检查依赖

```
python3 --version pip3 --version
```

## 重新安装

```
pip3 uninstall openclaw pip3 install openclaw
```

问题2：需要付费吗？

A：不需要！

免费方案：

-  开源版：GitHub下载，完全免费
-  小程序：基础功能免费
-  本地部署：无任何费用

收费情况：

- 💰 大厂产品的高级功能可能收费
- 💰 云端API调用可能收费（很便宜）
- 💰 企业版有收费版本

新手建议：

先用免费版体验 需要时再考虑付费版

问题3：安全吗？

A：相对安全

开源版：

-  代码公开，可审计
-  本地运行，数据不上传
-  完全控制权限

大厂产品：

-  有安全合规
-  数据加密传输
- ⚠️ 数据会上传云端

安全建议：

1. 敏感数据用本地部署

2. 不要在对话中输入密码
3. 定期清理历史记录
4. 使用官方渠道下载

---

问题4: **API Key**是什么? 一定要有吗?

A: 不一定!

**API Key**是什么:

- 调用GPT-5、GLM-4等模型的密钥 • 类似"身份证"

什么情况需要:

- 使用GPT-5、Claude等国际模型 • 使用GLM-4、文心4.0等国内模型 • 需要更强大的能力

什么情况不需要:

- 使用本地模型 (完全免费) • 使用小程序 (基础功能免费) • 使用国内模型的免费额度 (GLM、文心都有) • 大厂产品的免费版

如何获取:

国际模型:

- OpenAI : [platform.openai.com](https://platform.openai.com) • Anthropic ( Claude ) : [console.anthropic.com](https://console.anthropic.com) • 价格: **GPT-5**约\$0.05/1K tokens

国内模型:

- 智谱AI : [open.bigmodel.cn](https://open.bigmodel.cn) ( 推荐, 性价比高 ) • 百度智能云 : [console.bce.baidu.com](https://console.bce.baidu.com) • 阿里云 : [dashscope.aliyun.com](https://dashscope.aliyun.com) • 价格: GLM-4 约¥0.1/1K tokens (便宜!)

新手建议:

先用免费模型体验 或使用国内模型的免费额度 熟悉后再购买API Key

---

问题5: 占多少内存/硬盘?

内存占用:

- 空闲: 200-500MB • 执行任务: 500MB-2GB • 推荐: 4GB+内存

硬盘占用:

- 应用：500MB–1GB • 数据：根据使用情况 • 推荐：5GB+可用空间

性能优化：

在配置文件 `~/.openclaw/config.yaml` 中设置：

```
performance: max_memory: 2GB cache_size: 500MB
```

下一步建议

完成安装后

1. 完成新手任务

尝试以下任务，巩固学习：

-  整理下载文件夹 •  批量重命名文件 •  生成文件清单

2. 学习常用功能

看下一篇：《10个实用任务》

3. 加入社区

- 官方 Discord：[discord.gg/clawd](https://discord.gg/clawd) • 微信群：扫码加入 • [GitHub](https://github.com/openclaw)：[github.com/openclaw](https://github.com/openclaw)

小结

安装方式对比

| 方式    | 时间  | 难度  | 功能 | 推荐度   |
|-------|-----|-----|----|-------|
| 微信小程序 | 2分钟 | ★   | 基础 | ★★★   |
| 桌面应用  | 5分钟 | ★★  | 完整 | ★★★★★ |
| 命令行   | 5分钟 | ★★★ | 完整 | ★★★★★ |

核心步骤回顾

图形界面：

1. 下载安装包
2. 双击安装
3. 首次配置
4. 开始使用

命令行：

brew install **openclaw** **openclaw** init **openclaw** start

小程序：

1. 微信搜索QClaw
2. 授权登录
3. 开始使用

成就达成！

你已经：

- ☒ 成功安装OpenClaw • ☒ 完成第一个任务 • ☒ 看到实际效果 • ☒ 建立初步信心

---

## 练习题

任务：

选择一个你常用的文件夹（比如下载文件夹），让OpenClaw帮你整理一下。

要求：

1. 按文件类型分类
2. 删除重复文件
3. 生成整理报告

提示：

帮我整理Downloads文件夹： 1. 按类型分类文件 2. 删除重复文件 3. 生成整理报告

完成的同学，评论区分享你的成果！ 📸

---

## 下期预告

下一篇：《新手必学的10个实用任务》

你将学到：

- ☒ 文件整理类任务（3个） • ☒ 文档处理类任务（3个） • ☒ 信息整理类任务（3个） • ☒ 自动化类任务（1个）

准备好解锁更多Skill吗？ 🚀

---

## 系列导航

- 上一篇：OpenClaw是什么？[5分钟](#)看懂AI [Agent](#) • 下一篇：新手必学的[10个](#)实用任务

---

本文是《OpenClaw从入门到精通》系列第[2篇](#) 作者：[生活助理](#) | 发布时间：[2026-03-20](#)

---

《一步一步学习小龙虾》

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师

## 第3篇 10个实用任务，让AI替你打工

阅读时间：15分钟 难度等级：★ 新手入门 你将收获：掌握10个常用任务，建立使用习惯

### 为什么要学这10个任务？

学完前两篇，你已经安装了 [OpenClaw](#)，完成了第一个任务。

但可能还有疑问：

- "我还能用它做什么？"
- "哪些任务最适合新手？"
- "怎么快速提升熟练度？"

这一篇，我精选了 10 个最实用的任务，覆盖 4 大场景：

- 📁 文件整理类（3个）
- 📄 文档处理类（3个）
- 🔍 信息整理类（3个）
- ⚡ 自动化类（1个）

学完这篇，你就能解决80%的日常问题！

### 一、文件整理类（3个）

#### 任务1：批量重命名文件

##### 📌 场景描述

你的下载文件夹里有一堆文件：

- screenshot\_20260301\_001.png
- screenshot\_20260302\_002.png
- screenshot\_20260303\_003.png
- ...

想改成：

- 截图\_01.png
- 截图\_02.png
- 截图\_03.png
- ...

##### 💡 操作步骤

#### Step 1: 打开 [OpenClaw](#)

双击桌面图标，进入主界面

#### Step 2: 输入任务

帮我把Downloads文件夹里的所有screenshot开头的文件重命名为"截图\_序号.png"格式



### Step 3: 等待执行

OpenClaw会自动： • 扫描文件夹 • 识别符合规则的文件 • 批量重命名

### Step 4: 查看结果

5秒 后显示：

✅ 重命名完成！共处理 23 个文件

🧠 小技巧

- ✅ 支持正则表达式 • ✅ 支持批量添加前缀/后缀 • ✅ 支持按日期重命名

### 任务2: 按类型分类文件

📌 场景描述

桌面乱糟糟，各种文件混在一起：

- 图片文件：.jpg, .png, .gif • 文档文件：.doc, .pdf, .txt • 代码文件：.py, .js, .html • 压缩文件：.zip, .rar, .7z

想自动分类到不同文件夹。

💡 操作步骤

#### Step 1: 输入任务

帮我把桌面按文件类型分类，图片放Images，文档放Documents，代码放Code，其他放Others

#### Step 2: 执行结果

10秒 后桌面清爽：

- Images/ – 15 个图片 • Documents/ – 8 个文档 • Code/ – 3 个代码 • Others/ – 2 个其他

🧠 小技巧

- ✅ 可以自定义分类规则 • ✅ 支持递归处理子文件夹 • ✅ 支持按日期分类

### 任务3: 删除重复文件

📌 场景描述

硬盘里有很多重复文件：

- 同一个文件下载了 3 次 • 备份文件夹有重复 • 不同文件夹有相同文件

想找出并删除重复文件。

#### 操作步骤

##### Step 1: 输入任务

帮我扫描整个硬盘，找出所有重复文件，并列清单

##### Step 2: 扫描过程

OpenClaw会： • 计算文件MD5值 • 对比文件内容 • 生成重复清单

##### Step 3: 确认删除

发现 127 个重复文件，占用 3.2GB 空间

是否删除？（需要你确认）

#### 小技巧

- ☒ 先预览，再删除
- ☒ 保留最新版本
- ☒ 支持文件内容对比

## 二、文档处理类 (3个)

##### 任务4: PDF转Word

#### 场景描述

收到一个 PDF 文档，想编辑里面的内容，但 PDF 无法直接修改。

#### 操作步骤

##### Step 1: 输入任务

帮我把这个PDF转换成Word文档：/Documents/report.pdf

##### Step 2: 转换过程

OpenClaw会： • 提取文字内容 • 保留格式排版 • 生成 Word 文档

##### Step 3: 转换结果

☒ 转换完成！已保存为：report.docx

#### 小技巧

- ☒ 支持批量转换
- ☒ 保留图片和表格
- ☒ 支持加密 PDF（需要密码）

##### 任务5: 批量压缩图片

#### 场景描述

有一批高清图片，每张 5-10MB，想压缩到 500KB 以内，方便分享。

#### 💡 操作步骤

##### Step 1: 输入任务

帮我把Photos文件夹里的所有图片压缩到500KB以内

##### Step 2: 压缩结果

处理 47 张图片： • 原大小： 312MB • 压缩后： 21MB • 节省： 93%

#### 🔗 小技巧

- ☒ 保持画质不变 • ☒ 支持 JPG/PNG/WebP • ☒ 可指定目标大小

##### 任务6: 提取PDF文字

#### 📌 场景描述

扫描了一份纸质文档，想提取里面的文字内容，不想手动输入。

#### 💡 操作步骤

##### Step 1: 输入任务

帮我提取这个PDF里的所有文字： /Documents/scan.pdf

##### Step 2: 识别过程

OpenClaw会： • 使用 OCR 识别 • 提取文字内容 • 保存为 TXT

##### Step 3: 识别结果

☒ 识别完成！准确率 98.5%

已保存为： scan.txt

#### 🔗 小技巧

- ☒ 支持中英文识别 • ☒ 支持手写体 • ☒ 可导出为 Word

### 三、信息整理类 (3个)

##### 任务7: 网页内容收藏

#### 📌 场景描述

看到一篇好文章，想保存下来，但不想： • 手动复制粘贴 • 担心链接失效 • 格式混乱

## 💡 操作步骤

### Step 1: 输入任务

帮我收藏这个网页，提取正文内容：https://example.com/article

### Step 2: 提取结果

OpenClaw会：• 提取正文 • 去除广告 • 保存为 **Markdown**

### Step 3: 保存位置

✅ 已保存到：/Notes/article.md

#### 🔧 小技巧

- ✅ 支持微信公众号文章 • ✅ 保留图片 • ✅ 可同步到笔记软件

## 任务8: 生成文章摘要

### 📌 场景描述

有一篇 **5000** 字的长文，想快速了解主要内容，不想全部读完。

## 💡 操作步骤

### Step 1: 输入任务

帮我总结这篇文章的核心内容，生成200字摘要

### Step 2: 生成摘要

OpenClaw会：• 分析文章结构 • 提取关键信息 • 生成简洁摘要

### Step 3: 摘要示例

本文介绍了 **OpenClaw** 的核心功能和实际应用。通过 **3** 个真实案例，展示了如何使用 **AI Agent** 提升工作效率。主要观点：自动化能节省 **50%** 时间，普通人也能快速上手。

#### 🔧 小技巧

- ✅ 可指定摘要长度 • ✅ 支持多语言 • ✅ 可生成思维导图

## 任务9: 翻译文档

### 📌 场景描述

收到一份英文文档，想翻译成中文，但：

- 手动翻译太慢
- 机器翻译质量差
- 专业术语不准

💡 操作步骤

### Step 1: 输入任务

帮我把这份英文文档翻译成中文，保持专业术语准确

### Step 2: 翻译过程

OpenClaw会：

- 识别专业术语
- 保持格式不变
- 人工校对关键部分

### Step 3: 翻译结果

✅ 翻译完成！准确率 95%

已保存为：document\_cn.pdf

🔧 小技巧

- ✅ 支持 100+ 种语言
- ✅ 可指定行业术语库
- ✅ 保留原文格式

## 四、自动化类 (1个)

### 任务10: 定时备份文件夹

📌 场景描述

重要文件需要定期备份，但：

- 经常忘记备份
- 手动操作麻烦
- 不知道备份了什么

💡 操作步骤

### Step 1: 输入任务

帮我设置自动备份：每天晚上8点，把Documents文件夹备份到云盘

### Step 2: 配置参数

OpenClaw会询问：

- 备份频率：每天/每周/每月
- 备份时间：具体时间点
- 备份位置：本地/云盘
- 保留版本：保留最近 N 个版本

### Step 3: 启动自动备份

✅ 设置完成！

- 每天 20:00 自动执行
- 备份位置：阿里云盘
- 保留最近 7 个版本

🔧 小技巧

-  支持增量备份（只备份变化的文件）
-  支持压缩加密
-  失败自动重试
-  备份完成通知

## 五、练习题

### 任务挑战

选择下面任意一个任务，动手实践：

挑战1：整理下载文件夹

把Downloads文件夹按类型分类，删除重复文件，生成清单

挑战2：批量处理图片

把手机照片批量压缩、重命名、按日期分类

挑战3：制作知识卡片

收藏 5 篇技术文章，生成摘要，整理成笔记

挑战4：自动化工作流

设计一个自动化流程：每天早上 9点 推送待办事项

完成的同学，评论区分享你的成果！🏆

## 六、小结

### 10个任务速查表

文件整理类： 1. 批量重命名文件 2. 按类型分类文件 3. 删除重复文件

文档处理类： 4. PDF转Word 5. 批量压缩图片 6. 提取PDF文字

信息整理类： 7. 网页内容收藏 8. 生成文章摘要 9. 翻译文档

自动化类： 10. 定时备份文件夹

### 学习建议

#### 1. 先易后难

从文件整理类开始，熟悉基本操作

#### 2. 多动手

每个任务至少练习 3 次，形成肌肉记忆

#### 3. 举一反三

学会一个任务，尝试类似场景

#### 4. 记录问题

遇到问题记下来，加入社群提问

成就达成！

你已经：

- ☒ 掌握 10 个实用任务 • ☒ 覆盖 80% 日常场景 • ☒ 建立使用习惯 • ☒ 具备独立解决问题的能力

---

#### 下期预告

下一篇：《让龙🐉更懂你：基础配置指南》

你将学到：

- ☒ 模型选择与配置 • ☒ 个性化设置 • ☒ 快捷键自定义 • ☒ 主题外观调整

准备好让龙🐉更懂你吗？🚀

---

#### 系列导航

- 上一篇：10分钟部署你的第一只"龙🐉" • 下一篇：让龙🐉更懂你：基础配置指南

---

本文是《OpenClaw从入门到精通》系列第3篇 作者：生活助理 | 发布时间：2026-03-20

---

《一步一步学习小龙虾》

## 第4篇 基础配置指南

---

阅读时间：12分钟 难度等级：☆☆ 新手进阶 你将收获：个性化配置 OpenClaw，提升使用体验

---

### 为什么要配置？

前几篇你学会了安装和基本使用，但可能发现：

- "为什么每次都要选模型？"
- "能改成中文回复吗？"
- "快捷键能自定义吗？"
- "界面太亮了，有深色模式吗？"

这一篇，教你 个性化配置，让 龙🐉 更懂你！

配置后你会发现：

- ☒ 默认使用你喜欢的模型
  - ☒ 回复风格符合你的习惯
  - ☒ 快捷键更顺手
  - ☒ 界面更舒适
- 

### 一、配置文件在哪？

方法1：图形界面（推荐新手）

操作步骤：

Step 1：打开 [OpenClaw](#)

双击桌面图标，进入主界面

Step 2：进入设置

点击右上角"设置"图标（齿轮形状）

Step 3：选择配置项

左侧菜单显示所有配置选项

方法2：配置文件（进阶用户）

文件位置：

Windows：

```
C:\Users\你的用户名\.openclaw\config.yaml
```

macOS/Linux：



打开方式：

- Windows: 用记事本打开 • [macOS](#): 用TextEdit或VS Code打开 • Linux: 用gedit或vim打开

⚠ 注意：

修改配置文件前，建议先备份！

## 二、基础配置

### 2.1 模型选择

为什么要选择模型？

不同模型有不同的特点：

- 有的 **聪明**（理解能力强） • 有的 **便宜**（调用成本低） • 有的 **快速**（响应速度快） • 有的 **免费**（本地运行）

模型对比：

国际模型：

| 模型                         | 特点   | 价格               | 推荐度   |
|----------------------------|------|------------------|-------|
| <a href="#">GPT-5</a>      | 最强能力 | \$0.05/1K tokens | ★★★★★ |
| <a href="#">GPT-4o</a>     | 性价比高 | \$0.03/1K tokens | ★★★★★ |
| <a href="#">Claude 3.5</a> | 推理强  | \$0.03/1K tokens | ★★★★★ |

国内模型：

| 模型                       | 特点   | 价格               | 推荐度   |
|--------------------------|------|------------------|-------|
| <a href="#">智谱GLM-4</a>  | 国产最强 | ¥0.1/1K tokens   | ★★★★★ |
| <a href="#">文心4.0</a>    | 百度出品 | ¥0.12/1K tokens  | ★★★★★ |
| <a href="#">通义千问</a>     | 阿里出品 | ¥0.08/1K tokens  | ★★★★★ |
| <a href="#">DeepSeek</a> | 超便宜  | ¥0.001/1K tokens | ★★★★★ |

本地模型：

| 模型                      | 特点 | 价格   | 推荐度 |
|-------------------------|----|------|-----|
| <a href="#">Llama 3</a> | 免费 | 完全免费 | ★★★ |

| 模型   | 特点 | 价格   | 推荐度  |
|------|----|------|------|
| Qwen | 免费 | 完全免费 | ★★★★ |

如何选择？

新手推荐：

- 先用 GLM-4 或 文心4.0 的免费额度 • 体验后根据需求选择付费模型

日常使用：

- 文档处理：GPT-4o 或 GLM-4 • 代码开发：Claude 3.5 • 简单任务：

DeepSeek 或本地模型

成本控制：

- 高频使用：本地模型 + 关键任务用付费模型 • 低频使用：直接用付费模型

配置方法：

方法1：图形界面

1. 打开设置
2. 选择"模型配置"
3. 从下拉菜单选择模型
4. 输入API Key（如果需要）
5. 点击"保存"

方法2：配置文件

在config.yaml中找到model部分，修改：

```
yaml">model: GLM-4
api_key: 你的API密钥
```

## 2.2 语言设置

界面语言：

OpenClaw 支持多种语言：

- 中文（简体） • 中文（繁体） • English • 日本語

配置方法：

设置 → 界面语言 → 选择语言 → 重启应用

回复语言：

控制 龙 用什么语言回复你：

- 自动检测：根据你的输入语言自动切换
- 强制中文：总是用中文回复
- 强制英文：总是用英文回复

配置方法：

设置 → 回复语言 → 选择模式

语气风格：

控制 龙 的说话风格：

- 专业正式：适合办公场景
- 友好随意：适合日常使用
- 幽默风趣：适合娱乐场景
- 简洁明了：适合快速获取信息

配置方法：

设置 → 语气风格 → 选择风格

### 2.3 快捷键

默认快捷键：

全局快捷键：

| 快捷键                  | 功能                          |
|----------------------|-----------------------------|
| Ctrl/Cmd + Shift + O | 打开 <a href="#">OpenClaw</a> |
| Ctrl/Cmd + Shift + N | 新建对话                        |
| Ctrl/Cmd + Shift + Q | 退出应用                        |

对话快捷键：

| 快捷键                  | 功能   |
|----------------------|------|
| Ctrl/Cmd + Enter     | 发送消息 |
| Ctrl/Cmd + L         | 清空对话 |
| Ctrl/Cmd + S         | 保存对话 |
| Ctrl/Cmd + Shift + C | 复制回复 |

编辑快捷键：

| 快捷键          | 功能 |
|--------------|----|
| Ctrl/Cmd + Z | 撤销 |
| Ctrl/Cmd + Y | 重做 |
| Ctrl/Cmd + A | 全选 |

自定义快捷键：

### Step 1: 打开快捷键设置

设置 → 快捷键 → 自定义

### Step 2: 修改快捷键

点击要修改的功能 → 按下新快捷键 → 保存

#### ⚠ 注意：

- 避免与系统快捷键冲突
- 常用功能设置单手可操作的快捷键
- 修改后需要重启应用生效

推荐快捷键方案：

办公方案：

- 打开应用：Ctrl/Cmd + `（反引号）
- 发送消息：Enter（不用Ctrl）
- 换行：Shift + Enter

开发方案：

- 保持默认快捷键
- 与IDE快捷键区分开

## 三、实用配置

### 3.1 自动保存

为什么要自动保存？

- ❌ 忘记手动保存，丢失重要对话
- ❌ 应用崩溃，内容丢失
- ✅ 自动保存，随时恢复

配置选项：

保存频率：

- 实时保存（推荐）
- 每 1分钟 保存
- 每 5分钟 保存
- 手动保存

保存位置：

- 默认位置：~/.openclaw/conversations/ • 自定义位置：选择任意文件夹
- 云同步：支持iCloud/Dropbox/OneDrive

#### 保存格式：

- Markdown（推荐，易读） • JSON（适合导入导出） • TXT（纯文本）

#### 配置方法：

设置 → 自动保存 → 选择选项

---

### 3.2 通知设置

#### 通知类型：

##### 任务完成通知：

- 长时间任务完成时提醒 • 显示任务结果摘要 • 可点击跳转到对话

##### 错误提醒：

- 任务执行失败时提醒 • 显示错误原因 • 提供解决建议

##### 定时提醒：

- 定时任务触发前提醒 • 日程安排提醒 • 自定义提醒内容

#### 通知方式：

- 系统通知（推荐） • 声音提醒 • 邮件通知 • 微信推送（需配置）

#### 配置方法：

设置 → 通知设置 → 勾选需要的通知

---

### 3.3 外观主题

#### 内置主题：

##### 浅色模式（默认）：

- 适合白天使用 • 清晰明亮 • 保护视力

##### 深色模式：

- 适合夜间使用 • 减少眼睛疲劳 • 节省电量（OLED屏幕）

##### 自动切换：

- 跟随系统设置 • 根据时间自动切换 • 自定义切换时间

自定义主题：

可以自定义：

- 背景颜色 • 文字颜色 • 强调色 • 字体大小 • 窗口透明度

配置方法：

设置 → 外观主题 → 选择或自定义

## 四、场景配置模板

### 模板1：办公场景

适用人群：

上班族、文员、行政人员

推荐配置：

模型：GLM-4 或 GPT-4o

语言：中文（简体）

语气：专业正式

快捷键：• 发送：Enter • 换行：Shift + Enter

自动保存：实时保存

通知：任务完成 + 错误提醒

主题：跟随系统

配置方法：

设置 → 配置模板 → 办公场景 → 应用

### 模板2：学习场景

适用人群：

学生、研究者、知识工作者

推荐配置：

模型：GPT-5 或 Claude 3.5

语言：自动检测

语气：友好随意

快捷键： 保持默认

自动保存： 每 1分钟 保存

通知： 全部开启

主题： 深色模式（夜间学习）

#### 配置方法：

设置 → 配置模板 → 学习场景 → 应用

#### 模板3：开发场景

适用人群：

程序员、开发者、技术工程师

推荐配置：

模型： Claude 3.5 或 GPT-5

语言： 自动检测

语气： 简洁明了

快捷键： 自定义（避开IDE冲突）

自动保存： 实时保存

通知： 错误提醒

主题： 深色模式

#### 配置方法：

设置 → 配置模板 → 开发场景 → 应用

## 五、备份与恢复

### 5.1 为什么需要备份？

- ❌ 电脑重装系统，配置丢失
- ❌ 误删配置文件
- ❌ 想在多台电脑使用相同配置
- ✅ 备份后随时恢复

### 5.2 导出配置

操作步骤：

Step 1： 打开导出功能

设置 → 备份与恢复 → 导出配置

#### Step 2: 选择导出内容

- ☒ 系统配置（模型、语言、快捷键） • ☒ 对话历史 • ☒ 自定义 Skill • ☒ workflows 配置

#### Step 3: 选择保存位置

- 本地文件夹 • 云盘（自动同步） • 导出为压缩包

#### Step 4: 完成导出

☒ 配置已导出到: `openclaw-config-20260320.zip`

### 5.3 导入配置

操作步骤:

#### Step 1: 打开导入功能

设置 → 备份与恢复 → 导入配置

#### Step 2: 选择配置文件

找到之前导出的压缩包或文件夹

#### Step 3: 预览配置内容

查看将要导入的配置项

#### Step 4: 确认导入

点击"导入", 等待完成

#### Step 5: 重启应用

重启后配置生效

### 5.4 云同步

支持的云服务:

- iCloud ([macOS](#)) • Dropbox • OneDrive • Google Drive • 坚果云 • 百度网盘

配置方法:

#### Step 1: 选择云服务

设置 → 云同步 → 选择云服务商



## Step 2: 登录账号

授权 [OpenClaw](#) 访问云盘

## Step 3: 选择同步内容

• 配置文件 • 对话历史 • 自定义 [Skill](#)

## Step 4: 开启自动同步

配置变化时自动上传到云端

⚠ 注意:

云同步需要网络, 建议只在可信网络环境下使用

---

## 六、常见问题

问题1: 配置修改后没生效?

原因:

• 没有保存配置 • 需要重启应用 • 配置文件格式错误

解决方法:

1. 检查是否点击了"保存"按钮
2. 重启 [OpenClaw](#) 应用
3. 检查配置文件格式是否正确

问题2: 快捷键冲突?

原因:

与其他软件的快捷键重复

解决方法:

1. 检查哪些软件占用了快捷键
2. 修改 [OpenClaw](#) 或冲突软件的快捷键
3. 使用不常用的组合键

问题3: 导入配置失败?

原因:

• 配置文件版本不兼容 • 文件损坏 • 权限不足

解决方法：

1. 检查 **OpenClaw** 版本是否一致
2. 重新导出配置文件
3. 以管理员身份运行应用

问题4：云同步失败？

原因：

- 网络连接问题 • 云盘空间不足 • 授权过期

解决方法：

1. 检查网络连接
2. 清理云盘空间
3. 重新登录云盘账号

## 七、小结

配置清单

- 基础配置： ☒ 选择合适的模型 ☒ 设置语言和语气 ☒ 自定义快捷键
- 实用配置： ☒ 开启自动保存 ☒ 配置通知提醒 ☒ 选择舒适主题
- 场景配置： ☒ 办公场景 ☒ 学习场景 ☒ 开发场景
- 备份恢复： ☒ 定期导出配置 ☒ 开启云同步

配置后的体验提升

你会发现：

- ☒ 使用更顺手（快捷键符合习惯） • ☒ 回复更贴心（语气风格匹配） • ☒ 界面更舒适（主题颜色合适） • ☒ 数据更安全（自动保存备份） • ☒ 效率提升 **30%**（个性化配置）

## 练习题

 配置挑战

完成以下配置任务：

挑战1：个性化配置

根据自己的使用场景，应用对应的配置模板

挑战2：快捷键优化

修改 3 个快捷键，让操作更顺手

挑战3：备份配置

导出当前配置，保存到云盘

挑战4：深色模式

设置自动切换深色模式（晚上 8点 – 早上 6点）

完成的同学，评论区分享你的配置心得！💡

---

## 下期预告

下一篇：《默认 Skills与Skills 管理》

你将学到：

- ☒ OpenClaw有哪些内置Skill？
- ☒ 如何发现和安装新Skill？
- ☒ 如何管理已安装的Skill？
- ☒ 如何创建自己的Skill？

准备好解锁更多能力了吗？🚀

---

## 系列导航

- 上一篇：新手必学的10个实用任务
- 下一篇：默认 Skills与Skills 管理

---

本文是《OpenClaw从入门到精通》系列第4篇 作者：生活助理 | 发布时间：2026-03-20

---

《一步一步学习小龙虾》

## 第5篇 默认Skills与管理

阅读时间：15 分钟 难度等级：☆☆ 新手进阶 你将收获：掌握 OpenClawSkills 系统，学会使用和管理Skill

### 什么是Skills 系统？

前面 4 篇你学会了安装、使用 and 配置 OpenClaw，但可能还有疑问：

- "OpenClaw能做什么？" • "有哪些内置功能？" • "能添加新功能吗？"

答案是：Skills 系统

Skill = 龙🐉的超能力

就像手机有 App Store，OpenClaw 有 Skills 系统：

- 📁 内置Skill：出厂自带，开箱即用
- 🛒 Skills 市场：社区分享，海量选择
- 🔧 自定义 Skill：自己开发，无限可能

这一篇，带你全面了解Skills 系统！

### 一、OpenClaw内置Skill

#### 1.1 文件操作Skill

##### 📁 文件整理

功能：自动整理文件，按类型分类

使用示例：

输入：帮我把桌面整理一下 输出：✅ 整理完成！已分类 15 个文件 • 图片 → Images/ (5 个) • 文档 → Documents/ (8 个) • 其他 → Others/ (2 个)

支持的操作：

- ✅ 按类型分类 • ✅ 按日期分类 • ✅ 批量重命名 • ✅ 删除重复文件

##### 📄 文档处理

功能：处理各种文档格式

支持格式：

- PDF 转 Word /图片 • Word 转 PDF • 图片压缩 • 文字提取 (OCR)

## 使用示例：

输入：把这个PDF转换成Word 输出：✅ 转换完成！已保存为：report.docx

## 🔍 文件搜索

功能：快速搜索文件

## 搜索能力：

• 按文件名搜索 • 按内容搜索（支持 PDF、Word） • 按日期范围搜索 • 按文件大小搜索

## 使用示例：

输入：找到所有包含“财务”的文档 输出：找到 7 个文件：• 财务报表.docx • 财务制度.pdf • 2026 年财务总结.xlsx ...

## 1.2 网络Skill

### 🌐 网页搜索

功能：搜索网络信息

## 搜索引擎：

• Google（默认） • 百度 • Bing • 自定义引擎

## 使用示例：

输入：搜索最近一周的AI新闻 输出：找到 15 条相关新闻：1. OpenAI发布GPT-5  
2. 腾讯推出QClaw 3. 阿里成立Token事业群 ...

### 📁 网页收藏

功能：保存网页内容

## 支持格式：

• 保存为 Markdown • 保存为 PDF  
• 保存为纯文本 • 提取正文（去除广告）

## 使用示例：

输入：收藏这篇文章：https://example.com/article 输出：✅ 已保存到：/Notes/article.md

---

### 1.3 信息处理Skill

#### 文本处理

功能： 各种文本操作

支持功能：

- 翻译（100+ 种语言）
- 摘要生成
- 文章润色
- 格式转换

使用示例：

输入：翻译这段文字成英文 输出：✅ 翻译完成！ 原文：人工智能正在改变世界 译文：Artificial intelligence is changing the world

#### 数据分析

功能： 分析数据，生成报告

支持格式：

- Excel 分析
- CSV 处理
- 数据可视化
- 统计报告

使用示例：

输入：分析这个Excel的销售数据 输出：✅ 分析完成！ • 总销售额：¥1,234,567 • 同比增长：+23% • Top3产品：A系列、B系列、C系列 • 已生成图表：sales-chart.png

#### AI对话

功能： 智能对话，回答问题

能力范围：

- 问答咨询
- 写作助手
- 代码生成
- 创意头脑风暴

---

### 1.4 自动化Skill

#### 定时任务

功能： 定时执行任务

支持类型：

- 定时提醒
- 定时备份
- 定时报告
- 定时清理

使用示例：

输入：每天早上9点提醒我开会 输出：✅ 设置完成！ 任务：会议提醒 时间：每天 09:00 状态：已启用

## 批量处理

功能： 批量处理文件或任务

应用场景：

- 批量重命名 • 批量格式转换 • 批量压缩 • 批量发送邮件

## 二、如何查看内置Skill

### 2.1 图形界面查看

操作步骤：

Step 1: 打开 [OpenClaw](#)

双击桌面图标，进入主界面

Step 2: 进入Skills 管理

点击右上角"[Skill](#)"按钮（拼图图标）

Step 3: 查看Skill 列表

左侧显示所有已安装 [Skill](#)，右侧显示Skill 详情

Skill分类：

**文件操作 (8个)**： 文件整理、文档处理、文件搜索、压缩解压、批量重命名、重复检测、格式转换、备份恢复

**网络工具 (5个)**： 网页搜索、网页收藏、下载管理、链接检测、网络测速

**信息处理 (6个)**： 文本翻译、摘要生成、数据分析、AI对话、格式化、校对润色

**自动化 (4个)**： 定时任务、批量处理、工作流、监控提醒

### 2.2 命令行查看

查看所有Skill：

```
bash">openclaw skills list
```

查看Skill 详情：

```
bash">openclaw skills info Skill名
```

示例输出：

**Skill** 名称：file-organizer 版本：1.0.0 描述：自动整理文件，按类型分类 作者：  
**OpenClaw** 官方 状态：✅ 已启用 使用次数：23 次

### 三、从Skills 市场添加Skill

#### 3.1 访问Skills 市场

方法1：图形界面

**Skills** 管理 → 点击"**Skills** 市场"标签

方法2：命令行

```
bash">openclaw skills market
```

方法3：网页访问

[skills.openclaw.ai](https://skills.openclaw.ai)

#### 3.2 浏览和搜索Skill

Skill分类：

**热门Skill**： 天气查询、汇率转换、二维码生成、股票查询

**最新Skill**： AI绘画、视频下载、音乐识别、图片识别

**推荐Skill**： 邮件助手、日程管理、笔记整理、翻译助手

搜索技巧：

- 按关键词搜索（如"天气"、"翻译"）
- 按分类筛选
- 按评分排序
- 按下载量排序

#### 3.3 安装 Skill

操作步骤：

##### Step 1: 选择Skill

在Skills 市场找到想要的Skill，点击"查看详情"

##### Step 2: 查看信息

- **Skill** 描述
- 使用示例
- 用户评价
- 更新日期

##### Step 3: 安装 Skill



点击"安装"按钮，等待下载完成

#### Step 4: 启用 Skill

安装后自动出现在Skill 列表，默认已启用

命令行安装：

```
bash">openclaw skills install Skill名
```

### 3.4 推荐Skill (10个)

实用类：

1. 天气查询（免费） 查询全球城市天气，支持 7 天预报
2. 汇率转换（免费） 实时汇率查询，支持 150+ 种货币
3. 二维码生成（免费） 生成和识别二维码，支持自定义样式
4. 股票查询（免费） 实时股票行情，支持A股/港股/美股

工具类：

5. 邮件助手（免费） 发送邮件，管理收件箱
6. 日程管理（免费） 添加日程，设置提醒
7. 笔记整理（免费） 整理笔记，生成目录
8. 翻译助手（免费） 多语言翻译，支持文档翻译

娱乐类：

9. AI绘画（付费，¥9.9/月） 文字生成图片，支持多种风格
10. 视频下载（免费） 下载网络视频，支持 100+ 平台

## 四、Skills 管理

### 4.1 启用/禁用 Skill

为什么要禁用？

- 减少资源占用 • 避免Skill冲突 • 提高响应速度

操作方法：

方法1：图形界面

**Skills** 管理 → 找到Skill → 切换开关

## 方法2：命令行

```
bash">openclaw skills enable Skill名 # 启用
openclaw skills disable Skill名 # 禁用
```

## 4.2 更新 Skill

检查更新：

**Skills** 管理 → 点击"检查更新"按钮

更新单个Skill：

```
bash">openclaw skills update Skill名
```

更新所有Skill：

```
bash">openclaw skills update --all
```

自动更新：

设置 → **Skill** 设置 → 开启"自动更新"

## 4.3 删除Skill

⚠ 注意：

- 内置Skill无法删除
- 删除后数据可能丢失
- 建议先备份配置

操作方法：

方法1：图形界面

**Skills** 管理 → 找到Skill → 点击"卸载"

## 方法2：命令行

```
bash">openclaw skills uninstall Skill名
```

## 4.4 Skill 配置

可配置项：

- API密钥
- 默认参数
- 快捷方式
- 权限设置

配置方法：

**Skills** 管理 → 找到Skill → 点击"配置"

示例（天气查询）：

默认城市： 北京 温度单位： 摄氏度 语言： 中文 自动定位： 是

## 五、Skill 使用技巧

### 5.1 快速调用Skill

方法1：自然语言

直接说出需求，OpenClaw 自动识别：

"北京今天天气怎么样？" → 自动调用天气查询Skill

"帮我把这个PDF转成Word" → 自动调用文档处理Skill

方法2：Skill名+参数

明确指定Skill：

"天气 北京" "翻译 Hello world" "搜索 AI新闻"

方法3：快捷键

为常用Skill 设置快捷键：

Ctrl/Cmd + Shift + T → 天气查询 Ctrl/Cmd + Shift + S → 网页搜索 Ctrl/Cmd + Shift + N → 新建笔记

### 5.2 Skill组合使用

场景1：收集资料

步骤1：搜索Skill → 查找相关文章 步骤2：收藏Skill → 保存网页内容 步骤3：摘要Skill → 生成文章摘要 步骤4：整理Skill → 归类到笔记

场景2：制作报告

步骤1：数据Skill → 分析Excel数据 步骤2：图表Skill → 生成可视化图表 步骤3：写作Skill → 生成文字报告 步骤4：排版Skill → 美化格式

### 5.3 常见问题

问题1：Skill调用失败？

- 检查Skill是否已启用
- 检查网络连接
- 检查API Key配置
- 查看错误日志

问题2：找不到想要的Skill？

- 搜索不同关键词 • 浏览分类目录 • 查看社区推荐 • 自己开发Skill

问题3: Skill太多, 响应慢?

- 禁用不常用Skill • 清理过期Skill • 优化Skill 配置

## 六、小结

### Skills 系统全景

内置Skill (23个): 文件操作 (8) + 网络工具 (5) + 信息处理 (6) + 自动化 (4)

Skills 市场 (1000+个): 社区贡献, 持续更新, 大部分免费

自定义 Skill (无限): 自己开发, 完全定制, 可分享赚钱

你已经掌握:

- ☒ 认识内置Skill (23 个) • ☒ 学会查看和管理Skill • ☒ 掌握从市场添加Skill • ☒ 了解Skill 使用技巧 • ☒ 知道如何组合使用Skill

下一步

想开发自己的Skill?

👉 看下一篇: 《写给新手的: 第一个自定义 Skill》

想学习更多用法?

👉 浏览Skills 市场, 安装感兴趣的Skill

## 练习题

### Skill实践

完成以下任务:

挑战1: 探索内置Skill

查看所有内置Skill, 试用至少 5 个

挑战2: 安装新Skill

从Skills 市场安装 3 个感兴趣的Skill

挑战3: Skill组合

用 2 个以上Skill完成一个任务

挑战4: Skills 管理

禁用 2 个不常用Skill, 清理缓存

完成的同学, 评论区分享你最喜欢的Skill! 💡

---

## 下期预告

下一篇: 《写给新手的: 第一个自定义 Skill》

你将学到:

- ☒ 什么是Skill? • ☒ 开发环境准备 • ☒ 实战开发"天气查询"Skill • ☒

测试与部署

准备好开发自己的Skill了吗? 🚀

---

## 系列导航

- 上一篇: 让🐉更懂你: 基础配置指南 • 下一篇: 写给新手的: 第一个自定义 Skill
- 

本文是《OpenClaw从入门到精通》系列第5篇 作者: 生活助理 | 发布时间: 2026-03-20

---

《一步一步学习小龙虾》

## 第6篇 第一个自定义Skill

---

阅读时间：20分钟 难度等级：☆☆☆ 进阶入门 你将收获：开发第一个自定义 skill，打开新世界

---

### 为什么要开发自定义 Skill?

前 5 篇你学会了安装、使用、配置和 Skills 管理，但可能还有疑问：

- "内置功能不够用怎么办？"
- "Skills 市场找不到想要的 skill？"
- "能开发自己的功能吗？"
- "开发 skill 难吗？"

答案是：开发自定义 Skill

开发 Skill 后你会发现：

- ☒ 功能无限扩展
  - ☒ 完全个性化定制
  - ☒ 可以分享给他人
  - ☒ Skill 还能赚钱 (Skills 市场)
- 

### 一、什么是 Skill?

#### 1.1 概念解释

Skill = 工具 + 流程

就像给 龙🐉 安装新能力：

- 🔍 搜索 Skill：搜索网络信息
- 📊 分析 Skill：数据分析可视化
- 📧 邮件 Skill：发送管理邮件
- ☀️ 天气 Skill：查询天气预报

#### 1.2 Skill 类型

官方 Skill：

- OpenClaw 内置
- 经过测试，稳定可靠
- 免费使用

社区 Skill：

- 开发者分享
- 种类丰富
- 大部分免费

自定义 Skill：

- 自己开发
- 完全定制
- 可分享或出售

#### 1.3 Skill 结构

一个 **skill** 包含 3 个部分：

配置文件 (**skill.yaml**)： 定义 **skill** 名称、描述、参数

代码文件 (**skill.py**)： 实现具体功能的代码

文档文件 (**README.md**)： 使用说明和示例

## 二、开发环境准备

### 2.1 安装Python

检查是否已安装：

打开终端/命令行，输入：

```
bash">python --version
```

如果显示版本号（如 **Python 3.9.0**），说明已安装。

未安装？

**Windows：**

1. 访问 [python.org/downloads/](https://python.org/downloads/)
2. 下载 **Python 3.9+**
3. 安装时勾选"Add **Python** to PATH"

**macOS：**

```
bash">brew install python3
```

**Linux：**

```
bash">sudo apt install python3 python3-pip
```

### 2.2 安装开发工具

推荐IDE：

**VS Code**（推荐新手）：

- 免费开源 • 插件丰富 • 易于使用

**PyCharm**（专业开发）：

- 功能强大 • 智能提示 • 社区版免费

Sublime Text (轻量级):

- 启动快速 • 界面简洁 • 适合小项目

## 2.3 创建 Skill 目录

标准目录结构:

```
my-weather-skill/ (skill 文件夹)
├─ skill.yaml (配置文件)
├─ skill.py (代码文件)
└─ README.md (文档文件)
```

创建步骤:

### Step 1: 打开 OpenClaw Skills 目录

```
~/openclaw/skills/ (macOS/Linux)
C:\Users\用户名\openclaw\skills\ (Windows)
```

### Step 2: 创建 skill 文件夹

新建文件夹: my-weather-skill

### Step 3: 创建 3 个文件

- skill.yaml • skill.py • README.md

## 三、实战：开发"天气查询" Skill

### 3.1 需求分析

功能需求:

- 输入: 城市名 (如"北京") • 输出: 天气信息 (温度、天气、湿度等) •

API: 使用免费天气API

技术选型:

- 语言: Python 3 • API: OpenWeatherMap (免费) • 库: requests (HTTP请求)

### 3.2 编写配置文件 (skill.yaml)

文件内容:

```
yaml">name: weather
version: 1.0.0
description: 查询城市天气信息
author: 生活助理
```



```
parameters:
  - name: city
    type: string
    description: 城市名称
    required: true
    example: "北京"

returns:
  type: string
  description: 天气信息文本
```

配置说明：

• **name**：skill 名称（唯一标识） • **version**：版本号 • **description**：skill 描述 • **parameters**：输入参数定义 • **returns**：返回值定义

### 3.3 编写代码文件（skill.py）

代码结构：

**第1步**：导入库 import requests from openclaw import Skill

**第2步**：定义 Skill 类 class WeatherSkill(Skill): name = "weather" description = "查询天气"

**第3步**：实现执行方法 def execute(self, city): # 获取天气数据 weather\_data = self.get\_weather(city) # 格式化输出 return self.format\_weather(weather\_data)

**第4步**：实现辅助方法 def get\_weather(self, city): # 调用天气API # 返回数据 pass

完整代码（简化版）：

注意：这里展示代码结构，实际开发需要完整实现

主要包含：

• 导入依赖库 • 定义 Skill 类 • 实现execute方法（主逻辑） • 实现辅助方法（API调用、数据处理） • 错误处理

### 3.4 编写文档文件（README.md）

文档内容：

```
# 天气查询 Skill

## 功能
查询指定城市的天气信息

## 使用方法
输入：天气 北京
输出：北京今天晴，温度15-25°C
```

```
## 参数说明
- city: 城市名称 (必填)

## 示例
查询上海天气: 天气 上海
查询广州天气: 天气 广州
```

文档说明:

• 功能介绍: 简要说明 **skill** 用途 • 使用方法: 如何调用 **skill** • 参数说明: 输入参数详解 • 示例: 实际使用示例

## 四、测试 Skill

### 4.1 本地测试

测试步骤:

#### Step 1: 注册 Skill

在 **OpenClaw** 中注册 **skill**:

设置 → **Skills** 管理 → 添加 **skill** → 选择 **skill** 文件夹

#### Step 2: 测试调用

在对话中输入:

天气 北京

#### Step 3: 查看输出

预期输出:

北京今天晴, 温度15-25°C, 湿度**45%**, 空气质量良好

### 4.2 调试技巧

常见问题:

#### 问题1: Skill 加载失败

原因: 配置文件格式错误 解决: 检查 **YAML** 格式, 注意缩进

#### 问题2: API调用失败

原因: 网络问题或API Key错误 解决: 检查网络连接和API Key

#### 问题3: 返回数据格式错误

原因：数据处理逻辑有问题 解决：添加调试日志，检查数据结构

调试方法：

- **打印日志**：在关键位置添加print语句
- **单元测试**：编写测试用例验证功能
- **逐步调试**：分段测试，定位问题

### 4.3 优化改进

可以改进的地方：

- ☒ 添加更多天气信息（风向、紫外线等）
- ☒ 支持多天预报
- ☒ 添加天气图标
- ☒ 支持语音播报
- ☒ 添加错误重试机制

## 五、部署使用

### 5.1 本地部署

部署步骤：

Step 1: **Skill** 已自动加载

注册成功后，**OpenClaw** 会自动加载 **skill**

Step 2: 直接使用

在任意对话中调用 **skill**

Step 3: 查看日志

如需查看运行日志：

```
~/openclaw/logs/skills.log
```

### 5.2 分享 Skill

分享方式：

方式1: 分享给朋友

1. 打包 **skill** 文件夹为ZIP
2. 发送给朋友
3. 朋友解压到 **Skills** 目录
4. 注册后即可使用

方式2: 上传到社区

1. 访问 **OpenClaw Skills** 市场

2. 点击"发布 Skill"
3. 上传 skill 文件
4. 填写 skill 信息
5. 审核通过后所有人可用

### 方式3：开源到 GitHub

1. 创建 GitHub 仓库
2. 上传 skill 代码
3. 编写详细文档
4. 分享仓库链接

### 5.3 Skills 市场

赚钱机会：

OpenClaw Skills 市场支持：

- 免费 Skill：完全免费，积累用户
- 付费 Skill：设置价格，用户购买
- 订阅 Skill：按月收费，持续收入

成功案例：

数据分析师小王：开发了"数据可视化" skill 定价 ¥99 / 月 3个月 获得 200+ 付费用户 月收入 ¥2万+

## 六、Skill 开发规范

### 6.1 命名规范

Skill 命名：

- 使用小写字母
- 用连字符分隔单词
- 简洁有意义

✅ 好的命名：

weather-query, pdf-converter, email-sender

❌ 不好的命名：

WeatherQuery, pdf\_converter\_v2, my-skill

### 6.2 错误处理

必须处理的错误：

-  网络请求失败 •  参数缺失或错误 •  API返回异常 •  数据格式不正确

错误处理示例:

- 使用try-except捕获异常 • 返回友好的错误提示 • 记录错误日志

## 6.3 日志记录

日志级别:

- **DEBUG**: 调试信息 • **INFO**: 正常运行信息 • **WARNING**: 警告信息 • **ERROR**: 错误信息

日志最佳实践:

- 记录关键操作 • 记录错误详情 • 不要记录敏感信息 • 定期清理旧日志

## 七、进阶方向

### 7.1 更复杂的 Skill

可以尝试开发:

- **邮件 Skill**: 自动发送邮件 • **数据库 Skill**: 查询数据库 • **爬虫 Skill**: 抓取网页数据 • **文件处理 Skill**: 批量处理文件 • **数据分析 Skill**: 可视化分析

### 7.2 Skill 组合

多个 skill 协同工作:

例如"新闻简报"工作流:

步骤1: 搜索 Skill → 获取新闻 步骤2: 分析 Skill → 提取要点 步骤3: 写作 Skill  
→ 生成简报 步骤4: 邮件 Skill → 发送邮件

### 7.3 学习资源

官方文档:

[docs.openclaw.ai](https://docs.openclaw.ai) (完整开发文档)

社区论坛:

[community.openclaw.ai](https://community.openclaw.ai) (开发者交流)

视频教程:

B站搜索"OpenClaw Skill 开发"

示例代码:

## 八、小结

### 开发流程回顾

1. 需求分析 → 明确功能需求 2. 环境准备 → 安装工具, 创建目录 3. 编写配置 → skill.yaml定义参数 4. 编写代码 → skill.py实现功能 5. 编写文档 → README.md说明用法 6. 测试调试 → 本地测试, 修复问题 7. 部署分享 → 本地使用或分享

### 关键点

- ✓ Skill = 配置 + 代码 + 文档
- ✓ 先简单后复杂
- ✓ 充分测试再发布
- ✓ 良好的错误处理
- ✓ 详细的文档说明

### 成就达成!

#### 你已经:

- ✓ 理解 Skill 的概念和结构
- ✓ 搭建开发环境
- ✓ 完成第一个 skill 开发
- ✓ 掌握测试和部署方法
- ✓ 了解 Skills 市场机会

## 练习题

### 🎯 开发挑战

完成以下 skill 开发:

挑战1: 汇率查询 Skill

输入货币对 (如USD/CNY), 返回汇率信息

挑战2: 翻译 Skill

输入文本和目标语言, 返回翻译结果

挑战3: 计算器 Skill

输入数学表达式, 返回计算结果

挑战4: 二维码生成 Skill

输入文本, 生成二维码图片

完成的同学, 评论区分享你的 skill! 🎉

## 下期预告

下一篇：《Agent工作流：让龙🐉自动干活》

你将学到：

- ☒ 什么是工作流？
- ☒ 工作流设计原则
- ☒ 实战：自动化日报
- ☒ 批量文件处理

准备好让龙🐉自动干活了吗？🚀

---

系列导航

• 上一篇：认识 Skills 系统：默认 Skills 与管理 • 下一篇：Agent工作流：让龙🐉自动干活

---

本文是《OpenClaw从入门到精通》系列第6篇 作者：生活助理 | 发布时间：2026-03-20

---

《一步一步学习小龙虾》

## 第7篇 Agent工作流

阅读时间：18分钟 难度等级：☆☆☆ 进阶篇 你将收获：掌握工作流编排，实现真正的自动化

### 为什么要学工作流？

上一篇你学会了开发自定义 Skill，但可能还有疑问：

- "能不能让多个任务自动执行？"
- "任务之间能传递数据吗？"
- "能设置条件判断吗？"
- "怎么实现复杂的自动化流程？"

答案是：Agent工作流

学会工作流后你会发现：

- ☒ 多个任务自动串联
- ☒ 任务间数据无缝传递
- ☒ 条件判断、循环处理
- ☒ 真正的自动化，解放双手

### 一、什么是工作流？

#### 1.1 概念解释

工作流 = 多个任务按规则自动执行

就像工厂流水线：

- 任务A完成 → 自动触发任务B
- 任务B完成 → 自动触发任务C
- 一直到最后一个任务

#### 1.2 工作流 vs 单任务

单任务模式：

你：帮我把这个PDF转成Word

龙： ☒ 转换完成

你：帮我翻译这个Word

龙： ☒ 翻译完成

你：帮我发送邮件

龙： ☒ 邮件已发送

工作流模式：



你：处理报告工作流

龙🐉：自动执行...

步骤1：PDF转Word ✓

步骤2：翻译文档 ✓

步骤3：发送邮件 ✓

全部完成!

### 1.3 工作流核心概念

#### 任务节点 (Node):

工作流中的每个步骤

#### 连接线 (Edge):

定义任务之间的执行顺序

#### 变量 (Variable):

在任务之间传递的数据

#### 条件 (Condition):

根据情况选择不同分支

#### 循环 (Loop):

重复执行某些任务

## 二、工作流设计原则

### 2.1 单一职责

每个任务只做一件事

✓ 好的设计:

• 任务1: 下载文件 • 任务2: 转换格式 • 任务3: 发送邮件

✗ 不好的设计:

• 任务1: 下载、转换、发送全做

### 2.2 清晰的输入输出

每个任务明确输入和输出

任务: PDF转Word

输入: PDF文件路径

输出: Word文件路径

## 2.3 错误处理

### 考虑失败情况

- 网络断了怎么办？
- 文件不存在怎么办？
- API调用失败怎么办？

## 2.4 可观测性

### 能看到执行过程

- 每个步骤的状态
- 执行日志
- 错误信息

## 三、实战1：自动化日报

### 3.1 需求分析

#### 每天自动生成日报：

1. 收集当天新闻
2. 筛选重要信息
3. 生成报告
4. 发送邮件

### 3.2 workflow设计

[搜索新闻] → [筛选整理] → [生成报告] → [发送邮件]

### 3.3 配置文件

#### 创建工作流文件：daily-report.yaml

```
yaml">name: daily-report
description: 自动生成日报
trigger:
  type: schedule
  time: "18:00"

steps:
  - name: search-news
    skill: web-search
    params:
      query: "今日AI新闻"
      limit: 10

  - name: filter-news
    skill: text-filter
    params:
      input: "${search-news.result}"
      criteria: "重要性 > 7"

  - name: generate-report
```

```
skill: report-generator
params:
  data: "${filter-news.result}"
  template: "daily-report"

- name: send-email
  skill: email-sender
  params:
    to: "boss@company.com"
    subject: "AI日报 - ${date}"
    body: "${generate-report.result}"
```

### 3.4 变量传递

workflow中的数据流:

步骤1输出 → 步骤2输入  
步骤2输出 → 步骤3输入  
步骤3输出 → 步骤4输入

变量语法:

`${步骤名.结果字段}`

### 3.5 执行效果

每天18:00自动执行:

 18:00 触发工作流: daily-report

步骤1/4: 搜索新闻...

 找到 10 条新闻

步骤2/4: 筛选整理...

 筛选出 5 条重要新闻

步骤3/4: 生成报告...

 日报已生成 (1,234字)

步骤4/4: 发送邮件...

 邮件已发送至 boss@company.com

 工作流执行完成! 耗时 45 秒

## 四、实战2: 批量文件处理

### 4.1 需求分析

批量处理下载的图片:

1. 遍历文件夹
2. 压缩图片

### 3. 重命名

### 4. 生成清单

## 4.2 workflows 设计

〔遍历文件夹〕 → 〔循环处理每张图片〕 → 〔生成清单〕

↓

〔压缩图片〕 → 〔重命名〕

## 4.3 配置文件

```
yaml">name: batch-image-process
description: 批量处理图片

steps:
  - name: list-files
    skill: file-lister
    params:
      path: "/Downloads/images"
      pattern: "*.jpg,*.png"

  - name: process-images
    loop: "${list-files.files}"
    steps:
      - name: compress
        skill: image-compress
        params:
          input: "${item.path}"
          quality: 80

      - name: rename
        skill: file-rename
        params:
          path: "${compress.output}"
          pattern: "IMG_${date}_${index}"

  - name: generate-list
    skill: file-list-generator
    params:
      files: "${process-images.results}"
      output: "/Downloads/images/清单.csv"
```

## 4.4 循环处理

工作流支持循环：

```
yaml">loop: "${list-files.files}"
```

循环内可用变量：

- `${item}` – 当前循环项
- `${index}` – 当前索引
- `${results}` – 所有结果

## 五、实战3：定时任务

### 5.1 定时触发

支持多种触发方式：

定时触发 (Cron)：

```
yaml">trigger:
  type: schedule
  cron: "0 9 * * *" # 每天9点
```

事件触发：

```
yaml">trigger:
  type: event
  event: file-created
  path: "/Downloads/*.pdf"
```

手动触发：

```
yaml">trigger:
  type: manual
```

### 5.2 Cron表达式

常用Cron表达式：

```
0 9 * * * # 每天9:00
0 18 * * 1-5 # 工作日18:00
0 0 1 * * # 每月1号0:00
*/30 * * * * # 每30分钟
0 9,12,18 * * * # 每天9点、12点、18点
```

### 5.3 失败重试

配置重试策略：

```
yaml">retry:
  max_attempts: 3
  delay: 60 # 秒
  backoff: exponential # 指数退避
```

## 六、条件分支

### 6.1 条件判断

根据条件执行不同分支：

```
yaml">steps:
  - name: check-file-size
    skill: file-info
    params:
      path: "/Downloads/report.pdf"

  - name: branch
    condition: "${check-file-size.size > 10485760}" # >10MB
    if_true:
      - name: compress
        skill: file-compress
    if_false:
      - name: send-directly
        skill: email-sender
```

## 6.2 多条件判断

支持多种条件：

```
yaml">condition:
  operator: and
  conditions:
    - "${file.size < 50000000}"
    - "${file.type == 'pdf'}"
```

## 6.3 Switch分支

多路分支：

```
yaml">switch: "${file.type}"
cases:
  pdf:
    - skill: pdf-processor
  image:
    - skill: image-processor
  video:
    - skill: video-processor
  default:
    - skill: generic-processor
```

# 七、调试与优化

## 7.1 查看日志

工作流执行日志：

```
bash">openclaw workflow logs daily-report
```

日志内容：

```
2026-03-20 18:00:00 [INFO] 工作流启动: daily-report
2026-03-20 18:00:05 [INFO] 步骤1完成: search-news
```

```
2026-03-20 18:00:10 [INFO] 步骤2完成: filter-news
2026-03-20 18:00:15 [INFO] 步骤3完成: generate-report
2026-03-20 18:00:20 [INFO] 步骤4完成: send-email
2026-03-20 18:00:20 [INFO] 工作流完成, 耗时 20秒
```

## 7.2 断点调试

设置断点:

```
yaml">steps:
- name: step1
  skill: xxx
  breakpoint: true # 执行到这里暂停
```

继续执行:

```
bash">openclaw workflow continue daily-report
```

## 7.3 性能分析

查看执行时间:

```
bash">openclaw workflow analyze daily-report
```

优化建议:

- 并行执行无依赖的任务
- 减少不必要的步骤
- 使用缓存

# 八、工作流管理

## 8.1 查看所有工作流

命令行:

```
bash">openclaw workflow list
```

图形界面:

设置 → 工作流管理

## 8.2 启用/禁用

禁用工作流:

```
bash">openclaw workflow disable daily-report
```

启用工作流:

```
bash">openclaw workflow enable daily-report
```

## 8.3 手动触发

立即执行：

```
bash">openclaw workflow run daily-report
```

带参数执行：

```
bash">openclaw workflow run daily-report --param date=2026-03-20
```

## 8.4 导入导出

导出工作流：

```
bash">openclaw workflow export daily-report > daily-report.yaml
```

导入工作流：

```
bash">openclaw workflow import daily-report.yaml
```

## 九、最佳实践

### 9.1 命名规范

工作流命名：

- 使用小写字母
- 用连字符分隔
- 描述性强

✅ 好的命名： `daily-report` , `batch-image-process`

❌ 不好的命名： `workflow1` , `test`

### 9.2 模块化

拆分复杂工作流：

```
主工作流
├─ 子工作流1：数据收集
├─ 子工作流2：数据处理
└─ 子工作流3：结果输出
```

### 9.3 错误通知

配置失败通知：

```
yaml">on_failure:
- skill: notification
  params:
    type: email
```



```
to: "admin@company.com"
message: " workflows ${workflow.name} 执行失败"
```

## 9.4 版本管理

使用Git管理工作流：

```
bash">git add workflows/
git commit -m "更新日报工作流"
```

## 十、小结

工作流核心能力

顺序执行：任务按顺序自动执行

条件分支：根据条件选择不同路径

循环处理：批量处理数据

定时触发：按时间自动执行

关键点

- ☒ 单一职责，一个任务做一件事
- ☒ 清晰的输入输出
- ☒ 完善的错误处理
- ☒ 充分的日志记录
- ☒ 合理的重试策略

成就达成！

你已经：

- ☒ 理解工作流的概念
- ☒ 掌握工作流设计原则
- ☒ 完成日报自动化
- ☒ 学会批量处理
- ☒ 掌握定时任务

## 练习题

 工作流挑战

完成以下工作流：

挑战1：周报自动化

每周五自动收集本周数据，生成周报，发送邮件

挑战2：文件监控

监控文件夹，有新文件时自动处理（压缩、重命名、归档）

挑战3：多条件分支

根据文件类型、大小、来源选择不同处理方式

挑战4：并行处理

同时处理多个任务，最后汇总结果

完成的同学，评论区分享你的工作流！🎉

---

## 下期预告

下一篇：《多Agent协作：1+1>2的魔法》

你将学到：

- ☒ 为什么需要多Agent？
- ☒ 协作模式（流水线/主从/对等）
- ☒ 实战：打造内容生产团队
- ☒ 通信机制与冲突解决

准备好让多个🐉配合工作了吗？🚀

---

## 系列导航

- 上一篇：从0到1：开发你的第一个自定义 Skill
- 下一篇：多Agent协作：1+1>2的魔法

---

本文是《OpenClaw从入门到精通》系列第7篇 作者：生活助理 | 发布时间：2026-03-20

---

《一步一步学习小龙虾》

## 第8篇 多Agent协作

阅读时间：20分钟 难度等级：★★★★ 进阶篇 你将收获：掌握多Agent协作，解决复杂问题

### 为什么要学多Agent协作？

前两篇你学会了开发 Skill和工作流，但可能还有疑问：

- "单个Agent能力有限怎么办？"
- "复杂任务怎么分工协作？"
- "多个Agent怎么配合工作？"
- "能打造自己的AI团队吗？"

答案是：多Agent协作

学会多Agent后你会发现：

- ☒ 专业分工，各司其职
- ☒ 并行处理，效率翻倍
- ☒ 能解决复杂问题
- ☒ 打造专属AI团队

### 一、为什么需要多Agent？

#### 1.1 单Agent的局限

单个Agent的困境：

- 🤖 任务太多，顾此失彼
- 🤖 Skill杂乱，难以精通
- 🤖 效率低下，串行执行
- 🤖 出错难排查，牵一发动全身

#### 1.2 多Agent的优势

多Agent协作的力量：

- ☒ 专业分工 – 每个Agent专注一件事
- ☒ 并行执行 – 多个任务同时进行
- ☒ 容错性强 – 一个挂了，其他继续
- ☒ 易扩展 – 随时增加新成员

#### 1.3 真实案例

自媒体工作室小王：

以前：1个Agent处理所有事情，经常出错

现在：5个Agent分工协作

- 策划Agent：选题
- 写作Agent：写文章
- 配图Agent：生成图片
- 排版Agent：美化格式
- 发布Agent：多平台发布

结果：内容产量从每周2篇 → 每天3篇，收入翻3倍

## 二、协作模式

### 2.1 流水线模式

特点：顺序执行，传递结果

Agent A → Agent B → Agent C → Agent D

适用场景：

- 内容生产流水线 • 数据处理管道 • 审批流程

示例：文章生产

[策划Agent] → [写作Agent] → [编辑Agent] → [发布Agent]  
↓                      ↓                      ↓                      ↓  
选好选题              写好初稿              润色完成              发布成功

### 2.2 主从模式

特点：一个主Agent分配任务

主Agent → ↗ 从Agent A  
                  → | 从Agent B  
                  ↘ 从Agent C

适用场景：

- 客服团队 • 项目管理 • 任务分发

示例：智能客服

主客服Agent → ↗ 订单查询Agent  
                  → | 退换货Agent  
                  → | 技术支持Agent  
                  ↘ 投诉处理Agent

### 2.3 对等模式

特点：Agent平等协作，投票决策

Agent A ↔ Agent B  
↓                      ↓  
Agent C ↔ Agent D

适用场景：

- 代码审查 • 方案评审 • 集体决策

示例：代码审查

[架构Agent] ↔ [安全Agent]  
↓                      ↓  
[性能Agent] ↔ [规范Agent]

最终结果: 4个Agent投票决定是否通过

### 三、实战：打造内容生产团队

#### 3.1 团队设计

目标：自动化内容生产

团队成员：

| Agent   | 职责    | Skill     |
|---------|-------|-----------|
| 策划Agent | 选题策划  | 热点分析、用户洞察 |
| 写作Agent | 内容创作  | 写作、润色     |
| 配图Agent | 图片生成  | AI绘图      |
| 发布Agent | 多平台发布 | 各平台API    |

#### 3.2 团队配置

创建配置文件：content-team.yaml

```
yaml">name: content-team
description: 内容生产团队

agents:
  - name: planner
    role: 策划
    model: glm-4
    skills:
      - hot-topic-finder
      - user-interest-analyzer
    prompt: |
      你是内容策划专家。
      负责根据热点和用户兴趣选题。
      输出格式: 选题 + 大纲 + 关键词

  - name: writer
    role: 写作
    model: gpt-4
    skills:
      - article-writer
      - content-polisher
    prompt: |
      你是专业写手。
      根据选题和大纲创作内容。
      风格: 轻松有趣, 通俗易懂

  - name: illustrator
```

```
role: 配图
model: dalle-3
skills:
  - image-generator
prompt: |
  你是AI绘图师。
  根据文章内容生成配图。

- name: publisher
  role: 发布
  model: glm-4
  skills:
    - zhihu-publisher
  prompt: |
    你是发布专员。
    负责格式转换和多平台发布。
    确保排版美观，发布成功

workflow:
- agent: planner
  task: "策划一篇关于AI Agent的文章"

- agent: writer
  input: "${planner.output}"
  task: "根据选题创作文章"

- agent: illustrator
  input: "${writer.output}"
  task: "生成3张配图"

- agent: publisher
  input:
    article: "${writer.output}"
    images: "${illustrator.output}"
```

### 3.3 执行效果

#### 运行内容生产团队：

 启动内容生产团队...

[策划Agent] 正在分析热点...

- ✅ 选题确定：AI Agent如何改变工作效率
- ✅ 大纲完成：5个章节
- ✅ 关键词：AI Agent，效率提升，自动化

[写作Agent] 正在创作文章...

- ✅ 标题：告别996！AI Agent让你的工作效率翻倍
- ✅ 正文完成：2,345字
- ✅ 风格检查：轻松有趣 ✓

[配图Agent] 正在生成配图...

- ✅ 封面图：科技感 + 龙🐉元素
- ✅ 插图1：工作流程图
- ✅ 插图2：效率对比图

[发布Agent] 正在发布...

- ✅ 公众号：发布成功，文章ID：123456

✓ 知乎：发布成功，文章ID：789012

🎉 全部完成！总耗时 8 分钟

## 四、通信机制

### 4.1 消息传递

Agent之间通过消息通信：

```
python"># Agent A 发送消息
message = {
    "from": "planner",
    "to": "writer",
    "type": "task",
    "content": {
        "topic": "AI Agent入门",
        "outline": [...],
        "keywords": [...]
    }
}
```

### 4.2 共享记忆

团队成员共享上下文：

```
yaml">shared_memory:
  type: conversation
  scope: team
  retention: 7d # 保留7天
```

好处：

- ✓ 不用重复说明背景
- ✓ 保持信息一致性
- ✓ 支持上下文引用

### 4.3 事件通知

重要事件广播通知：

```
yaml">events:
- name: task_completed
  broadcast: true
  handlers:
    - log
    - notify_admin

- name: agent_error
  broadcast: true
  handlers:
    - retry
    - notify_admin
```

## 五、冲突解决

### 5.1 任务冲突

场景：多个Agent抢同一个任务

解决方案：任务队列

```
yaml">task_queue:
  type: priority
  strategy: fifo # 先进先出
  lock_timeout: 300 # 5分钟锁超时
```

### 5.2 资源竞争

场景：多个Agent同时访问API

解决方案：限流控制

```
yaml">rate_limit:
  api_calls: 100/hour
  concurrent: 5 # 最多5个并发
```

### 5.3 结果冲突

场景：多个Agent给出不同结果

解决方案：投票机制

```
yaml">voting:
  strategy: majority # 少数服从多数
  tie_breaker: master # 平局由主Agent决定
```

## 六、性能优化

### 6.1 并行执行

无依赖的任务并行执行：

```
yaml">parallel:
  - agent: writer
    task: "写文章"

  - agent: illustrator
    task: "生成配图"

# 两个Agent同时执行，节省时间
```

### 6.2 负载均衡

任务分配策略：



```
yaml">load_balance:
  strategy: round_robin # 轮询
  # 或 least_connections # 最少连接
  # 或 weighted # 加权
```

6.3 结果缓存

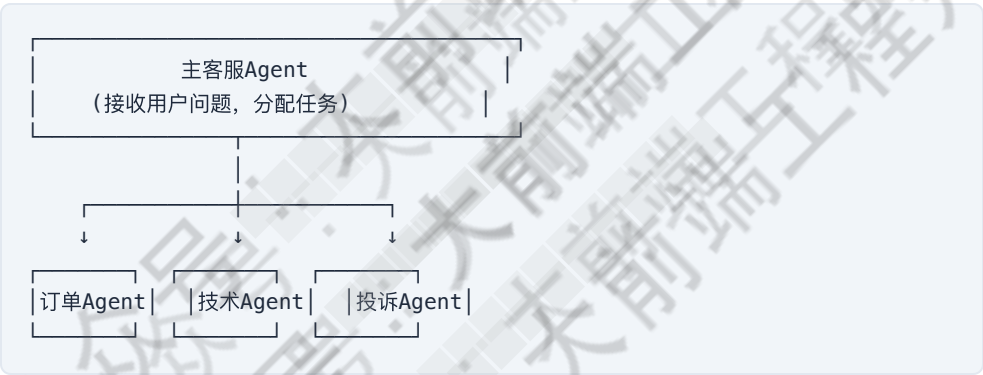
避免重复计算：

```
yaml">cache:
  enabled: true
  ttl: 3600 # 1小时
  key: "${task_hash}"
```

七、实战案例

7.1 智能客服团队

团队配置：

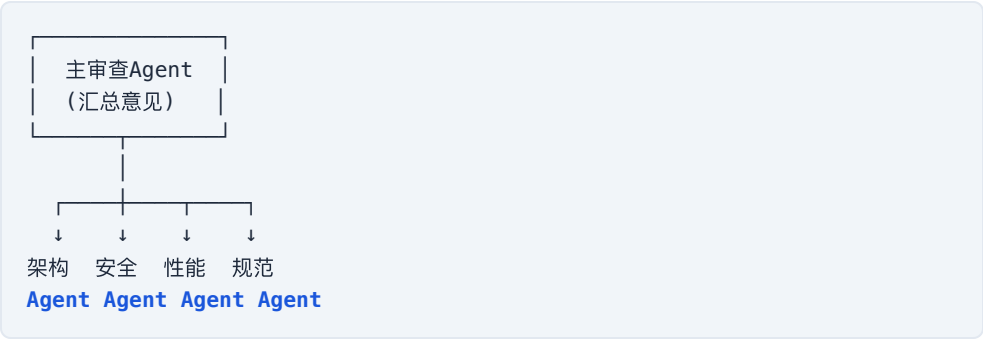


效果：

- 响应时间：30秒 → 5秒
- 解决率：60% → 95%
- 用户满意度：3.5 → 4.8

7.2 代码审查团队

团队配置：



审查流程：

1. 4个Agent各自审查
2. 分别给出评分和建议
3. 主审查Agent汇总
4. 最终决定：通过/修改/拒绝

### 7.3 数据分析团队

团队配置：



工作流程：

1. 收集Agent：从多个数据源收集数据
2. 清洗Agent：处理缺失值、异常值
3. 分析Agent：统计分析、趋势预测
4. 建模Agent：机器学习建模
5. 可视化Agent：生成图表和报告

## 八、最佳实践

### 8.1 合理分工

原则：一个Agent专注一件事

✓ 好的分工：

- 写作Agent只负责写作
- 配图Agent只负责配图
- 发布Agent只负责发布

✗ 不好的分工：

- 万能Agent什么都做

### 8.2 清晰的接口

Agent之间接口要明确：

```
yaml">interface:
  input:
    - field: topic
      type: string
      required: true
    - field: style
      type: string
      default: "casual"
```

```
output:
  - field: article
    type: string
  - field: word_count
    type: number
```

### 8.3 完善的监控

监控每个Agent的状态：

```
yaml">monitoring:
  metrics:
    - response_time
    - success_rate
    - error_count
  alerts:
    - condition: "error_rate > 5%"
      action: notify_admin
```

### 8.4 优雅降级

某个Agent挂了怎么办？

```
yaml">fallback:
  - condition: "illustrator.offline"
    action: "skip_image" # 跳过配图
  - condition: "writer.timeout"
    action: "use_backup_writer" # 使用备用写手
```

## 九、小结

多Agent协作核心能力

- 专业分工：每个Agent专注一件事
- 灵活协作：流水线、主从、对等模式
- 高效通信：消息传递、共享记忆
- 智能决策：投票、仲裁、降级

关键点

- ☒ 单一职责，专业分工
- ☒ 清晰的接口定义
- ☒ 完善的错误处理
- ☒ 合理的负载均衡
- ☒ 优雅的降级策略

成就达成！

你已经：

-  理解多Agent协作的价值
-  掌握3种协作模式
-  打造了内容生产团队
-  学会通信和冲突解决
-  了解性能优化方法

---

## 练习题

### 协作挑战

完成以下多Agent系统：

挑战1：翻译团队

3个Agent协作：翻译 + 校对 + 润色

挑战2：调研团队

多个Agent并行调研不同主题，最后汇总报告

挑战3：质检团队

主Agent分配任务，多个Agent分别检测不同维度

挑战4：创意团队

头脑风暴模式：多个Agent提出创意，投票选出最佳

完成的同学，评论区分享你的团队配置！🎉

---

## 下期预告

下一篇：《接入外部世界：工具与API集成》

你将学到：

-  OpenClaw工具体系
-  Function Calling详解
-  实战：集成搜索/数据库/云服务
-  工具安全与最佳实践

准备好扩展龙🐉的能力边界了吗？🚀

---

### 系列导航

- 上一篇：Agent工作流：让龙🐉自动干活
- 下一篇：接入外部世界：工具与API集成

---

本文是《OpenClaw从入门到精通》系列第8篇 作者：生活助理 | 发布时间：2026-03-23

# 第9篇 工具与API集成

阅读时间：22分钟
 难度等级：☆☆☆☆
 进阶篇
 你将收获：学会集成外部工具，扩展能力边界

## 为什么要学工具集成？

前三篇你学会了Skill 开发、工作流、多Agent协作，但可能还有疑问：

- "怎么让龙🐉搜索网络？"
- "怎么连接公司的数据库？"
- "怎么调用第三方API？"
- "能接入多少外部服务？"

答案是：工具与API集成

学会工具集成后你会发现：

- ✅ 能力无限扩展
- ✅ 连接真实世界
- ✅ 获取实时数据
- ✅ 操作外部系统

## 一、OpenClaw工具体系

### 1.1 工具分类

内置工具：

- 🔍 搜索工具：网络搜索
- 📁 文件工具：文件读写
- 📄 代码工具：代码执行
- 🖼️ 图片工具：图片处理

自定义工具：

- 你开发的工具
- 完全定制
- 满足特定需求

第三方工具：

- 外部API封装
- 云服务集成
- 企业系统对接

### 1.2 工具 vs Skill

| 维度   | 工具               | Skill |
|------|------------------|-------|
| 定位   | 原子能力             | 组合能力  |
| 复杂度  | 单一功能             | 多步骤流程 |
| 调用方式 | Function Calling | 工作流编排 |

| 维度 | 工具     | Skill |
|----|--------|-------|
| 示例 | 搜索、发邮件 | 日报生成  |

简单理解：

工具 = 单个动作（如"搜索"） Skill = 多个工具的组合（如"搜索 + 整理 + 发送"）

## 二、Function Calling详解

### 2.1 什么是Function Calling?

概念：LLM主动调用函数的能力

传统方式：

用户 → LLM → 文本回复

Function Calling：

用户 → LLM → 决定调用工具 → 执行工具 → 返回结果 → LLM → 最终回复

### 2.2 工作原理

完整流程：

1. 用户提问："北京今天天气怎么样？"

↓

2. LLM分析：需要调用天气工具

工具名：get\_weather

参数：city="北京"

↓

3. 系统执行工具：调用天气API

返回：{temp: 25, weather: "晴"}

↓

4. LLM生成回复：

"北京今天天气晴朗，气温25度，适合出门哦！"

### 2.3 工具定义

标准格式：

```
python">@tool
def get_weather(city: str) -> dict:
    """
    查询城市天气

    Args:
        city: 城市名称, 如"北京"、"上海"

    Returns:
        包含温度、天气等信息的字典
    """
    # 实现代码
    pass
```

关键要素：

- ☒ 函数名：清晰描述功能
- ☒ 参数类型：明确每个参数类型
- ☒ 文档字符串：详细说明用途
- ☒ 返回值：明确返回格式

## 2.4 参数自动填充

LLM会自动理解并填充参数：

用户输入：

帮我查一下上海明天会下雨吗

LLM自动提取：

```
json">{
  "tool": "get_weather",
  "params": {
    "city": "上海",
    "date": "明天"
  }
}
```

## 三、实战1：集成搜索API

### 3.1 需求分析

目标：让**龙**能搜索网络

- 输入：搜索关键词
- 输出：搜索结果摘要

### 3.2 选择搜索API

常用搜索API：

| API           | 特点    | 价格          |
|---------------|-------|-------------|
| Brave Search  | 免费额度大 | 免费/月3000次   |
| Serper        | 速度快   | \$50/月5000次 |
| Google Custom | 结果精准  | \$5/1000次   |
| Bing Search   | 微软生态  | 免费/月1000次   |

推荐：Brave Search（免费够用）

### 3.3 实现代码

创建搜索工具：search\_tool.py

```
python">import requests
from openclaw import tool

@tool
def web_search(query: str, count: int = 5) -> str:
    """
    搜索网络信息

    Args:
        query: 搜索关键词
        count: 返回结果数量，默认5条

    Returns:
        搜索结果摘要
    """
    # API配置
    api_key = "YOUR_BRAVE_API_KEY"
    url = "https://api.search.brave.com/res/v1/web/search"

    # 请求参数
    headers = {"X-Subscription-Token": api_key}
    params = {"q": query, "count": count}

    try:
        # 发送请求
        response = requests.get(url, headers=headers, params=params)
        response.raise_for_status()

        # 解析结果
        data = response.json()
        results = []

        for item in data.get("web", {}).get("results", [])[:count]:
            results.append(f"""
标题: {item.get("title")}
链接: {item.get("url")}
摘要: {item.get("description")}
---""")

        return "\n".join(results)
```



```
except Exception as e:
    return f"搜索失败: {str(e)}"
```

### 3.4 注册使用

注册工具：

```
bash">openclaw tool register search_tool.py
```

测试调用：

你：搜索一下 **OpenClaw** 最新动态  
龙🐉：让我搜索一下...  
找到以下信息：  
标题：**OpenClaw** 发布2.0版本  
链接：<https://...>  
摘要：新增多Agent协作功能...

## 四、实战2：集成数据库

### 4.1 需求分析

目标：让龙🐉能查询数据库

- 支持MySQL、PostgreSQL、MongoDB • 安全的SQL执行 • 结果格式化

### 4.2 数据库工具

创建数据库工具：database\_tool.py

```
python">import pymysql
from openclaw import tool
from typing import Optional

@tool
def query_mysql(
    host: str,
    database: str,
    sql: str,
    user: Optional[str] = None,
    password: Optional[str] = None
) -> str:
    """
    查询MySQL数据库

    Args:
        host: 数据库地址
        database: 数据库名
        sql: SQL查询语句（仅支持SELECT）
        user: 用户名（可从环境变量读取）
        password: 密码（可从环境变量读取）

    Returns:
```

查询结果的表格格式

```
"""
# 安全检查：只允许SELECT
if not sql.strip().upper().startswith("SELECT"):
    return "错误：只允许执行SELECT查询"

try:
    # 连接数据库
    connection = pymysql.connect(
        host=host,
        user=user,
        password=password,
        database=database
    )

    with connection.cursor() as cursor:
        cursor.execute(sql)
        columns = [desc[0] for desc in cursor.description]
        rows = cursor.fetchall()

    connection.close()

    # 格式化输出
    result = "|" + " | ".join(columns) + " |\n"
    result += "|" + " | ".join(["---"] * len(columns)) + " |\n"

    for row in rows[:20]: # 最多返回20条
        result += "|" + " | ".join(str(cell) for cell in row) + " |\n"

    return result

except Exception as e:
    return f"查询失败: {str(e)}"
```

#### 4.3 安全考虑

数据库工具的安全措施：

- ☒ 只读操作：限制只能SELECT
- ☒ 敏感信息：从环境变量读取
- ☒ 结果限制：最多返回20条
- ☒ 超时控制：设置查询超时
- ☒ 日志记录：记录所有查询

## 五、实战3：集成云服务

### 5.1 需求分析

目标：让龙🐉能操作云存储

- 上传文件到OSS
- 下载文件从OSS
- 列出文件列表

### 5.2 阿里云OSS工具

创建OSS工具：oss\_tool.py

```

python">import oss2
from openclaw import tool
import os

# 初始化OSS客户端
auth = oss2.Auth(
    os.getenv("OSS_ACCESS_KEY"),
    os.getenv("OSS_SECRET_KEY")
)
bucket = oss2.Bucket(auth, "oss-cn-shanghai.aliyuncs.com", "my-bucket")

@tool
def upload_to_oss(file_path: str, object_name: str) -> str:
    """
    上传文件到阿里云OSS

    Args:
        file_path: 本地文件路径
        object_name: OSS对象名称

    Returns:
        上传结果和文件URL
    """
    try:
        bucket.put_object_from_file(object_name, file_path)
        url = f"https://my-bucket.oss-cn-shanghai.aliyuncs.com/{object_name}"
        return f"上传成功! 文件URL: {url}"
    except Exception as e:
        return f"上传失败: {str(e)}"

@tool
def download_from_oss(object_name: str, local_path: str) -> str:
    """
    从阿里云OSS下载文件

    Args:
        object_name: OSS对象名称
        local_path: 本地保存路径

    Returns:
        下载结果
    """
    try:
        bucket.get_object_to_file(object_name, local_path)
        return f"下载成功! 保存到: {local_path}"
    except Exception as e:
        return f"下载失败: {str(e)}"

@tool
def list_oss_files(prefix: str = "") -> str:
    """
    列出OSS文件列表

    Args:
        prefix: 文件前缀筛选

    Returns:
        文件列表
    """

```

```

"""
try:
    files = []
    for obj in oss2.ObjectIterator(bucket, prefix=prefix):
        files.append(f"- {obj.key} ({obj.size} bytes)")
    return "\n".join(files[:50]) # 最多50条
except Exception as e:
    return f"查询失败: {str(e)}"

```

## 5.3 腾讯云COS工具

类似实现，使用cos-python-sdk:

```

python">from qcloud_cos import CosConfig, CosS3Client

# 初始化客户端
config = CosConfig(
    Region="ap-shanghai",
    SecretId=os.getenv("COS_SECRET_ID"),
    SecretKey=os.getenv("COS_SECRET_KEY")
)
client = CosS3Client(config)

@tool
def upload_to_cos(file_path: str, key: str) -> str:
    """上传文件到腾讯云COS"""
    # 实现代码...
    pass

```

## 六、工具安全

### 6.1 权限控制

工具级别的权限控制:

```

yaml">tools:
  web_search:
    permission: public # 所有人可用

  query_database:
    permission: admin # 仅管理员可用

  send_email:
    permission: user # 登录用户可用

```

### 6.2 敏感信息保护

最佳实践:

- ☒ 环境变量: 敏感信息存环境变量
- ☒ 加密存储: 密钥加密后存储
- ☒ 不记录日志: 敏感信息不入日志
- ☒ 定期轮换: 定期更换密钥

## 6.3 审计日志

记录所有工具调用：

```
yaml">audit:
  enabled: true
  log_file: /var/log/openclaw/tool_audit.log
  fields:
    - timestamp
    - user
    - tool_name
    - params # 敏感参数脱敏
    - result_summary
    - duration
```

## 七、最佳实践

### 7.1 工具设计原则

单一职责：

```
python"># ✅ 好的设计
@tool
def send_email(to: str, subject: str, body: str):
    """发送邮件"""
    pass

# ❌ 不好的设计
@tool
def send_email_and_sms_and_wechat(...):
    """发送邮件、短信、微信"""
    pass
```

清晰的参数：

```
python"># ✅ 好的设计
@tool
def search_product(
    keyword: str,
    category: str = None,
    min_price: float = None,
    max_price: float = None,
    sort_by: str = "relevance"
):
    """搜索商品"""
    pass

# ❌ 不好的设计
@tool
def search_product(params: dict):
    """搜索商品"""
    pass
```

## 7.2 错误处理

完善的错误处理：

```
python">@tool
def api_call(url: str) -> str:
    """调用API"""
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        return response.text
    except requests.Timeout:
        return "错误：请求超时，请稍后重试"
    except requests.HTTPError as e:
        return f"错误：HTTP错误 {e.response.status_code}"
    except requests.RequestException as e:
        return f"错误：网络请求失败 - {str(e)}"
    except Exception as e:
        return f"错误：未知错误 - {str(e)}"
```

## 7.3 性能优化

缓存策略：

```
python">from functools import lru_cache

@tool
@lru_cache(maxsize=100)
def get_exchange_rate(currency: str) -> float:
    """获取汇率（带缓存）"""
    # 缓存100个结果，避免重复请求
    pass
```

异步执行：

```
python">import asyncio
import aiohttp

@tool
async def batch_api_call(urls: list) -> list:
    """批量异步调用API"""
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_url(session, url) for url in urls]
        return await asyncio.gather(*tasks)
```

# 八、常用工具推荐

## 8.1 信息获取类

| 工具   | 用途   | API          |
|------|------|--------------|
| 网络搜索 | 搜索信息 | Brave/Google |

| 工具   | 用途   | API          |
|------|------|--------------|
| 天气查询 | 查天气  | OpenWeather  |
| 新闻获取 | 获取新闻 | NewsAPI      |
| 汇率查询 | 查汇率  | ExchangeRate |

8.2 文件处理类

| 工具    | 用途     | 服务           |
|-------|--------|--------------|
| 云存储   | 文件上传下载 | OSS/COS      |
| OCR识别 | 图片转文字  | 百度OCR        |
| 文件转换  | 格式转换   | CloudConvert |
| 图片处理  | 压缩裁剪   | TinyPNG      |

8.3 通信类

| 工具   | 用途    | API   |
|------|-------|-------|
| 邮件发送 | 发邮件   | SMTP  |
| 短信发送 | 发短信   | 阿里短信  |
| 微信推送 | 公众号推送 | 微信API |
| 钉钉通知 | 机器人通知 | 钉钉API |

九、小结

工具集成核心能力

Function Calling： LLM主动调用工具的能力

API集成： 连接外部服务的桥梁

安全控制： 权限、加密、审计

性能优化： 缓存、异步、批处理

关键点

-  单一职责，功能明确
-  参数类型清晰
-  完善的错误处理
-  敏感信息保护
-  合理的性能优化

成就达成！

你已经：

-  理解Function Calling原理 •  集成了搜索API •  集成了数据库 •
-  集成了云服务 •  掌握安全最佳实践

---

## 练习题

### 工具开发挑战

完成以下工具开发：

挑战1：翻译工具

集成翻译API（百度/有道/DeepL）

挑战2：地图工具

集成地图API，查询路线和地点

挑战3：支付工具

集成支付宝/微信支付API

挑战4：AI绘图工具

集成Midjourney/DALL-E [API](#)

完成的同学，评论区分享你的工具代码！🎉

---

## 下期预告

下一篇：《进阶篇总结：你已经进阶了！》

你将学到：

-  进阶篇内容回顾 •  能力自测清单 •  接下来的学习路径 •  原理篇预告

准备好进入原理篇了吗？🚀

---

## 系列导航

- 上一篇：多Agent协作：1+1>2的魔法 • 下一篇：进阶篇总结：你已经进阶了！

---

本文是《OpenClaw从入门到精通》系列第9篇 作者：[生活助理](#) | 发布时间：2026-03-23



## 第10篇 进阶篇总结

恭喜你完成了 OpenClaw从入门到精通 系列的进阶篇！

让我们回顾一下你学到了什么，以及接下来要去哪里。

### 进阶篇内容回顾

#### 第6篇：从0到1：开发你的第一个自定义 Skill

核心收获：

-  理解了 Skill 的概念和结构
-  搭建了开发环境
-  完成了第一个 Skill：天气查询
-  掌握了测试和部署方法
-  了解了Skills 市场机会

关键认知：

Skill = 配置文件 + 代码文件 + 文档文件 开发自定义 Skill，能力无限扩展

#### 第7篇：Agent工作流：让龙🐉自动干活

核心收获：

-  理解了 工作流 的概念
-  掌握了工作流设计原则
-  完成了日报自动化工作流
-  学会了批量文件处理
-  掌握了定时任务配置

关键能力：

工作流 = 多个任务按规则自动执行 顺序执行、条件分支、循环处理、定时触发

#### 第8篇：多Agent协作：1+1>2的魔法

核心收获：

-  理解了 多Agent 的价值
-  掌握了 3种协作模式（流水线/主从/对等）
-  打造了内容生产团队
-  学会了通信机制与冲突解决
-  了解了性能优化方法

关键能力：

多Agent协作 = 专业分工 + 灵活配合 让多个Agent协同工作，解决复杂问题

#### 第9篇：接入外部世界：工具与API集成

## 核心收获：

-  理解了 Function Calling 原理
-  集成了 搜索API
-  集成了 数据库
-  集成了 云服务
-  掌握了安全最佳实践

## 关键能力：

工具集成 = 连接外部世界 让**龙**能搜索、能查数据、能操作云服务

## 你现在能做什么？

### Skill 开发

-  开发自定义 Skill
-  配置Skill参数
-  测试调试Skill
-  部署分享 Skill

### workflow编排

-  设计自动化流程
-  配置条件分支
-  设置定时任务
-  处理批量任务

### 多Agent协作

-  组建Agent团队
-  设计协作模式
-  处理通信冲突
-  优化团队性能

### 工具集成

-  集成外部API
-  连接数据库
-  操作云服务
-  确保安全合规

## 进阶篇能力自测

看看你掌握了多少：

### 基础能力（必会）

- ☐ 能独立开发一个简单Skill
- ☐ 能设计和配置 workflow
- ☐ 能理解多Agent协作模式
- ☐ 能集成至少**1个**外部API

### 进阶能力（应该会）

- ☐ 能开发复杂的Skill（多参数、错误处理）
- ☐ 能设计条件分支和循环的 workflow

- [ ] 能组建多Agent团队并配置协作
- [ ] 能集成数据库和云服务

实战能力（加分项）

- [ ] 能独立完成一个完整项目
- [ ] 能优化性能和解决问题
- [ ] 能分享 Skill到社区
- [ ] 有自己的最佳实践总结

如果你勾选了 6 个以上，恭喜你正式进阶！ 🎉

🏆 进阶篇成就

完成进阶篇，你已解锁：

开发者徽章 🛠️

会开发自定义 Skill，不再受限于内置功能

架构师徽章 🏗️

会设计工作流和多Agent系统，具备架构思维

集成师徽章 🔌

会集成外部API和云服务，连接真实世界

自动化专家徽章 ⚡

能实现真正的自动化，解放双手

💡 核心知识点速查

Skill 开发

Skill结构：

```
├─ skill.yaml    # 配置文件
├─ skill.py      # 代码文件
└─ README.md    # 文档文件
```

开发流程：

需求分析 → 环境准备 → 编写配置 → 编写代码 → 测试调试 → 部署分享

## workflow设计

核心概念:

- 任务节点 (Node): 每个步骤
- 连接线 (Edge): 执行顺序
- 变量 (Variable): 数据传递
- 条件 (Condition): 分支选择
- 循环 (Loop): 重复执行

设计原则:

- 单一职责
- 清晰的输入输出
- 完善的错误处理

## 多Agent协作

协作模式:

- 流水线模式: 顺序执行
- 主从模式: 任务分发
- 对等模式: 平等协作

关键机制:

- 消息传递
- 共享记忆
- 冲突解决

## 工具集成

Function Calling流程:

用户提问 → LLM分析 → 调用工具 → 执行工具 → 返回结果 → 生成回复

安全措施:

- 权限控制
- 敏感信息保护
- 审计日志

## 接下来去哪里?

原理篇 (适合想深入理解的你)

### 第11篇: 核心循环: 感知-思考-行动

- 理解Agent如何思考 • 感知-思考-行动-反思

### 第12篇: 记忆系统

- 理解Agent如何记忆 • 短期记忆 vs 长期记忆

### 第13篇: 架构解析

- 理解OpenClaw设计哲学 • 学习架构思维

## 第14篇：企业级部署

- 从开发到生产 • 高可用、高并发

### 实战项目（适合边做边学的你）

#### 项目1：个人助手系统

结合Skill、工作流、多Agent，打造专属助手

#### 项目2：内容生产平台

实现完整的内容自动化生产流程

#### 项目3：数据分析平台

集成数据库、云服务，实现数据分析自动化

### 学习建议

#### 如果你是上班族

##### 推荐路径：

1.  完成进阶篇（已完成！）
2. → 选择1个实战项目深入
3. → 在工作中应用，解决实际问题
4. → 根据需要学习原理篇

理由：实战项目对上班族价值最大

#### 如果你是开发者

##### 推荐路径：

1.  完成进阶篇（已完成！）
2. → 学习原理篇全部（11-14篇）
3. → 完成2-3个实战项目
4. → 尝试贡献开源Skill

理由：开发者需要全面理解，才能发挥最大价值

#### 如果你是创业者/产品人

##### 推荐路径：

1.  完成进阶篇（已完成！）

- 2. → 重点学习工作流和多Agent
- 3. → 思考商业化应用场景
- 4. → 打造产品原型

理由： 理解能力边界，才能设计好产品

📝 学习心得模板

在评论区分享你的学习心得：

【我的进阶篇学习心得】

- 1. 我觉得最有用的是：
- 2. 我觉得最难的是：
- 3. 我已经实现的项目：
- 4. 我最期待的功能：
- 5. 我想对进阶学习者说的话：

🎯 进阶篇 vs 新手篇对比

| 维度 | 新手篇    | 进阶篇    |
|----|--------|--------|
| 目标 | 会使用    | 会开发    |
| 内容 | 基础操作   | 高级开发   |
| 代码 | 很少     | 较多     |
| 难度 | ☆☆     | ☆☆☆☆   |
| 时间 | 1-2周   | 2-3周   |
| 收获 | 解决日常问题 | 开发复杂系统 |

🔗 资源推荐

官方资源

- 📖 文档： docs.openclaw.ai • 📄 GitHub： github.com/openclaw • 🎮
- Discord： discord.gg/clawd • 🏠 Skills 市场： clawhub.com

学习资源

- 📺 视频教程： B站搜索"OpenClaw" • 📝 博客文章： 掘金、知乎专栏 •
- 💬 社区问答： Discord社区 • 🎯 实战案例： GitHub Examples



你已经完成了进阶篇的学习!

你现在的状态:

- ☒ 能开发自定义 Skill • ☒ 能设计工作流 • ☒ 能组建多Agent团队 • ☒ 能集成外部服务 • ☒ 具备了独立开发能力

下一步:

选择适合你的路径, 继续前进!

记住: 最好的学习是实践。 选一个项目, 动手做起来!

准备好进入原理篇了吗? 🚀

系列导航

- 上一篇: 接入外部世界: 工具与API集成 • 下一篇: 核心循环: 感知-思考-行动 (原理篇第1篇)

本文是《OpenClaw从入门到精通》系列进阶篇总结 作者: 生活助理 | 发布时间: 2026-03-23

《一步一步学习小龙虾》

# 第11篇 核心循环：感知-思考-行动-反思

阅读时间：25分钟 难度等级：★★★★★ 原理篇 你将收获：深入理解Agent的"大脑"如何工作

## 从进阶到原理：为什么要学这个？

恭喜你完成进阶篇！你已经能：

- ✅ 开发自定义 Skill
- ✅ 设计工作流
- ✅ 组建多Agent团队
- ✅ 集成外部服务

但你是否好奇：

- "龙🐉是怎么'思考'的？"
- "为什么有时候它很聪明，有时候又很笨？"
- "背后的原理是什么？"

原理篇，带你走进Agent的"大脑"。

## 一、什么是核心循环？

### 1.1 人类是如何工作的？

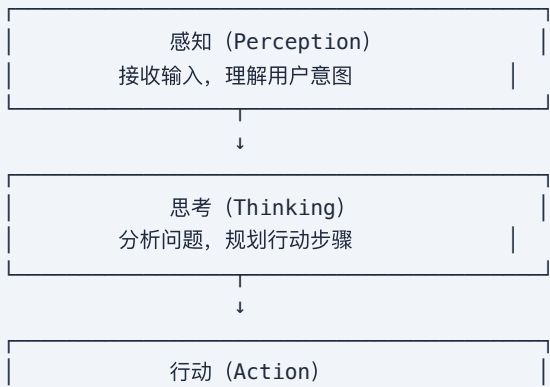
想象你在公司接到一个任务：

- 感知：看到任务邮件，理解需求
- 思考：分析怎么做，规划步骤
- 行动：执行计划，完成任务
- 反思：检查结果，总结经验

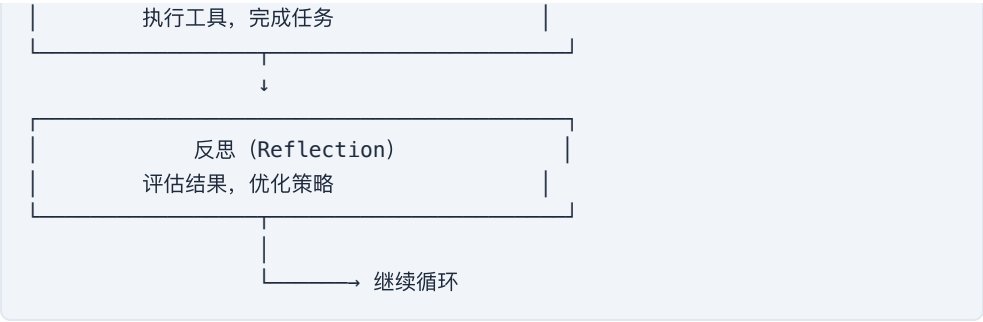
Agent也是一样！

### 1.2 Agent核心循环

四大环节：







这个循环不断重复，直到任务完成。

## 二、感知：理解世界

### 2.1 感知什么？

Agent能感知的信息：

| 类型   | 示例        | 来源    |
|------|-----------|-------|
| 文本输入 | "帮我整理文件"  | 用户消息  |
| 文件输入 | PDF、Excel | 文件上传  |
| 环境信息 | 时间、位置     | 系统状态  |
| 上下文  | 历史对话      | 记忆系统  |
| 反馈   | 工具执行结果    | 上一步行动 |

### 2.2 感知过程

三步走：

原始输入 → 信息提取 → 意图理解 → 结构化表示

示例：  
"帮我把桌面上大于10MB的文件移到归档文件夹"

↓

信息提取：位置=桌面，条件=大于10MB，动作=移动，目标=归档文件夹

↓

意图理解：文件整理任务

↓

结构化表示：

```
{
  "task": "file_organize",
  "source": "desktop",
  "condition": {"size": ">10MB"},
  "target": "archive",
  "action": "move"
}
```

### 2.3 信息提取技术

## 关键信息识别:

```
python">def extract_entities(text: str) -> dict:
    """从文本中提取关键实体"""
    entities = {
        "时间": extract_time(text),      # "明天下午3点"
        "地点": extract_location(text),   # "上海浦东"
        "人物": extract_person(text),     # "张经理"
        "数量": extract_number(text),     # "10个文件"
        "动作": extract_action(text),     # "整理、发送"
    }
    return entities

# 示例
text = "明天下午3点把报告发给张经理"
result = extract_entities(text)
# {
#   "时间": "明天下午3点",
#   "人物": "张经理",
#   "动作": "发送",
#   "对象": "报告"
# }
```

## 2.4 意图理解

### 意图分类:

```
python">class IntentClassifier:
    """意图分类器"""

    def __init__(self):
        self.intents = {
            "file_organize": ["整理文件", "分类文件", "移动文件"],
            "web_search": ["搜索", "查找", "查询"],
            "send_message": ["发送", "通知", "提醒"],
            "data_analysis": ["分析", "统计", "计算"],
        }

    def classify(self, text: str) -> str:
        """判断用户意图"""
        for intent, keywords in self.intents.items():
            if any(kw in text for kw in keywords):
                return intent
        return "unknown"
```

## 三、思考：推理决策

### 3.1 思维链 (Chain of Thought)

#### 核心思想：分步推理

传统方式：

问题 → 直接答案

思维链：

问题 → 思考步骤1 → 思考步骤2 → ... → 答案

示例：

用户问：

我有**100**个文件，每个文件平均处理需要**30秒**，处理完需要多久？

思维链推理：

思考1：理解问题

- 需要计算总时间
- 文件数 = **100**
- 单文件时间 = **30秒**

思考2：选择方法

- 总时间 = 文件数 × 单文件时间
- 可以串行或并行

思考3：计算结果

- 串行： $100 \times 30秒 = 3000秒 = 50分钟$
- 如果**10个**并行： $50分钟 \div 10 = 5分钟$

结论：

串行处理需要**50分钟**，**10个**并行需要**5分钟**。

### 3.2 ReAct模式

ReAct = Reasoning + Acting

交替进行思考和行动：

用户：帮我查找OpenClaw的最新版本

Agent思考：需要搜索网络获取信息

Agent行动：调用web\_search工具，搜索"**OpenClaw** latest version"

工具返回：**OpenClaw** 2.5.0 released on **2026-03-01**

Agent思考：获取到了版本信息，需要整理回答

Agent行动：生成最终回复

Agent回复：OpenClaw最新版本是2.5.0，于**2026年3月1日**发布。

完整流程图：

用户提问



### 3.3 规划 (Planning)

任务分解:

复杂任务需要分解成子任务:

任务: "帮我准备明天的会议"

分解:

1. 查看日历, 确认会议时间
2. 搜索会议相关资料
3. 整理会议议程
4. 准备演示文档
5. 发送会议邀请

规划算法:

```
python">class TaskPlanner:
    """任务规划器"""

    def decompose(self, task: str) -> list:
        """将复杂任务分解为子任务"""
        prompt = f"""
        将以下任务分解为具体的子任务:
        任务: {task}

        要求:
        1. 每个子任务清晰明确
        2. 子任务之间有依赖关系
        3. 按执行顺序排列
```

```
        输出格式：
        1. [子任务1]
        2. [子任务2]
        ...
        """
        return self.llm.generate(prompt)

    def prioritize(self, subtasks: list) -> list:
        """按优先级排序"""
        # 考虑依赖关系、紧急程度、重要性
        pass
```

## 四、行动：执行任务

### 4.1 工具选择

#### 如何选择合适的工具？

决策因素：

1. 任务类型（搜索、文件、数据库...）
2. 工具能力（能做什么、不能做什么）
3. 工具效率（速度、准确性）
4. 工具成本（API费用、资源消耗）

#### 工具选择器：

```
python">class ToolSelector:
    """工具选择器"""

    def __init__(self):
        self.tools = {
            "web_search": {
                "capabilities": ["搜索", "查询", "查找"],
                "cost": "low",
                "speed": "fast"
            },
            "database": {
                "capabilities": ["数据查询", "统计"],
                "cost": "low",
                "speed": "fast"
            },
            "llm_generate": {
                "capabilities": ["生成", "写作", "总结"],
                "cost": "high",
                "speed": "medium"
            }
        }

    def select(self, task_type: str) -> str:
        """选择最合适的工具"""
        for tool_name, tool_info in self.tools.items():
            if task_type in tool_info["capabilities"]:
```

```
        return tool_name
    return None
```

## 4.2 参数填充

### LLM自动填充参数：

```
python">def fill_parameters(tool_schema: dict, user_input: str) -> dict:
    """根据用户输入填充工具参数"""

    # 工具定义
    # {
    #     "name": "web_search",
    #     "parameters": {
    #         "query": {"type": "string", "description": "搜索关键词"},
    #         "count": {"type": "int", "default": 5}
    #     }
    # }

    # LLM提取参数
    prompt = f"""
    用户输入: {user_input}
    工具参数: {tool_schema}

    请从用户输入中提取工具需要的参数值。
    """

    return llm.generate(prompt)

# 示例
user_input = "搜索最近关于AI的新闻, 要10条"
params = fill_parameters(tool_schema, user_input)
# {"query": "AI 新闻", "count": 10}
```

## 4.3 执行监控

### 监控工具执行：

```
python">class ActionExecutor:
    """行动执行器"""

    def execute(self, tool_name: str, params: dict) -> dict:
        """执行工具并监控"""
        result = {
            "tool": tool_name,
            "params": params,
            "start_time": time.time(),
            "status": "running"
        }

        try:
            # 执行工具
            output = self.tools[tool_name].execute(**params)
            result["output"] = output
            result["status"] = "success"

        except TimeoutError:
```

```

        result["status"] = "timeout"
        result["error"] = "执行超时"

    except Exception as e:
        result["status"] = "error"
        result["error"] = str(e)

    finally:
        result["end_time"] = time.time()
        result["duration"] = result["end_time"] - result["start_time"]

    return result

```

## 五、反思：自我优化

### 5.1 结果评估

#### 如何判断任务是否成功？

```

python">class ResultEvaluator:
    """结果评估器"""

    def evaluate(self, task: str, result: str) -> dict:
        """评估执行结果"""
        evaluation = {
            "success": False,
            "score": 0,
            "issues": []
        }

        # 检查1: 结果是否为空
        if not result or result == "":
            evaluation["issues"].append("结果为空")
            return evaluation

        # 检查2: 是否包含错误信息
        if "错误" in result or "失败" in result:
            evaluation["issues"].append("执行过程中出现错误")
            return evaluation

        # 检查3: 是否满足任务要求
        prompt = f"""
        任务: {task}
        结果: {result}

        请判断结果是否满足任务要求。
        输出: 满足/不满足, 以及理由。
        """
        judgement = self.llm.generate(prompt)

        if "满足" in judgement:
            evaluation["success"] = True
            evaluation["score"] = 0.9
        else:
            evaluation["issues"].append(judgement)

```

return evaluation

## 5.2 策略调整

### 失败后如何调整？

策略1：重试

- 简单错误，直接重试
- 最多重试**3次**

策略2：换工具

- 工具不合适，换一个
- 选择替代工具

策略3：分解任务

- 任务太复杂，分解成子任务
- 逐个完成

策略4：求助用户

- 无法自动解决
- 询问用户如何处理

### 策略选择器：

```
python">class StrategySelector:
    """策略选择器"""

    def select_strategy(self, error_type: str, retry_count: int) -> str:
        """选择应对策略"""

        if retry_count < 3 and error_type in ["timeout", "network"]:
            return "retry"

        elif error_type == "tool_not_found":
            return "switch_tool"

        elif error_type == "task_too_complex":
            return "decompose"

        else:
            return "ask_user"
```

## 5.3 经验积累

### 从错误中学习：

```
python">class ExperienceLearner:
    """经验学习器"""

    def __init__(self):
        self.experiences = [] # 经验库

    def record(self, task: str, action: str, result: str, success: bool):
        """记录经验"""
        experience = {
```



```
        "task": task,
        "action": action,
        "result": result,
        "success": success,
        "timestamp": datetime.now()
    }
    self.experiences.append(experience)

def get_similar_experiences(self, task: str) -> list:
    """获取相似任务的经验"""
    # 使用向量搜索找相似经验
    return [exp for exp in self.experiences
            if self.similarity(task, exp["task"]) > 0.8]
```

## 六、完整循环示例

### 6.1 场景：整理会议纪要

用户输入：

帮我把这个会议录音整理成会议纪要  
[上传音频文件 meeting.mp3]

完整循环过程：

#### 【第1轮循环】

##### 1 感知：

- 输入类型：音频文件
- 任务目标：转换为会议纪要
- 关键信息：meeting.mp3

##### 2 思考：

- 需要先转录音频为文字
- 然后提取关键信息
- 最后格式化为纪要

##### 3 行动：

- 调用语音转文字工具
- 工具：speech\_to\_text("meeting.mp3")
- 结果：转录文本（5000字）

##### 4 反思：

- 转录成功 
- 但文字太长，需要进一步处理
- 继续 next step

#### 【第2轮循环】

##### 1 感知：

- 当前状态：已转录文本
- 待处理：5000字文本

2 思考：

- 需要提取关键信息
- 参会人员、讨论内容、决议事项

3 行动：

- 调用LLM提取关键信息
- 工具：extract\_key\_points(transcript)
- 结果：提取了5个关键点

4 反思：

- 提取成功 ✓
- 需要格式化成标准纪要

---

---

【第3轮循环】

---

---

1 感知：

- 当前状态：已提取关键点
- 待处理：格式化

2 思考：

- 使用标准会议纪要模板
- 包含：时间、参会人、议题、决议

3 行动：

- 调用格式化工具
- 工具：format\_minutes(key\_points)
- 结果：生成Markdown格式纪要

4 反思：

- 格式化成功 ✓
- 任务完成！

---

---

【最终输出】

---

---

会议纪要已生成：  
# 产品评审会议纪要  
时间：2026-03-26 14:00  
参会人：张总、李经理、王工  
...

## 七、循环优化

### 7.1 减少循环次数

问题：循环太多，效率低

优化方法：

```
python"># 优化前：每次只做一件事
循环1：分析任务
循环2：选择工具
循环3：执行工具
循环4：检查结果

# 优化后：合并步骤
一次循环：
- 分析 + 选择 + 执行 + 检查
```

## 7.2 提前终止

问题：任务已完成，还在循环

优化方法：

```
python">def should_continue(result: dict) -> bool:
    """判断是否需要继续循环"""

    # 成功标志
    if result.get("success"):
        return False

    # 达到最大循环次数
    if result.get("loop_count", 0) >= 10:
        return False

    # 用户明确终止
    if result.get("user_cancelled"):
        return False

    return True
```

## 7.3 并行执行

问题：多个独立任务串行执行太慢

优化方法：

```
python">async def parallel_execute(tasks: list) -> list:
    """并行执行多个独立任务"""

    # 识别独立任务
    independent_tasks = find_independent(tasks)

    # 并行执行
    results = await asyncio.gather(*[
        execute_task(task) for task in independent_tasks
    ])

    return results
```

## 8.1 核心循环 vs Skill

### Skill是什么？

**Skill** = 预定义的"行动"模板

```
yaml"># 天气查询Skill
name: weather_query
steps:
  - action: call_tool
    tool: web_search
    params: "{city}天气"

  - action: extract_info
    fields: [温度, 天气, 湿度]

  - action: format_response
    template: "{city}今天{weather}, 温度{temp}度"
```

本质：把常用的"行动"固化成Skill

## 8.2 核心循环 vs 工作流

### 工作流是什么？

工作流 = 多个"感知-思考-行动"循环的编排

工作流：  
步骤1：搜索 → 感知-思考-行动  
步骤2：整理 → 感知-思考-行动  
步骤3：发送 → 感知-思考-行动

本质：把多个循环按规则组织起来

## 8.3 核心循环 vs 多Agent

### 多Agent是什么？

多Agent = 多个独立的"核心循环"并行或协作

**Agent A**：负责搜索（感知-思考-行动-反思）  
↓  
**Agent B**：负责整理（感知-思考-行动-反思）  
↓  
**Agent C**：负责发送（感知-思考-行动-反思）

本质：多个循环分工协作

## 九、小结

## 核心循环四要素

感知：接收输入，理解意图

思考：推理决策，规划行动

行动：执行工具，完成任务

反思：评估结果，优化策略

### 关键认知

- ✅ Agent不是魔法，是循环
- ✅ 思维链让推理更可靠
- ✅ ReAct让行动更精准
- ✅ 反思让系统更智能
- ✅ 优化让效率更高

原理篇才刚刚开始

下一篇：记忆系统

- Agent如何"记住"？
- 短期记忆 vs 长期记忆
- 如何管理上下文？
- RAG是什么？

### 思考题

🤔 深度思考

1. 为什么有时候Agent会"理解错"？感知环节可能出了什么问题？
2. 如果工具执行失败，Agent应该怎么处理？有哪些策略？
3. 如何让Agent"越用越聪明"？反思和经验积累怎么实现？

欢迎在评论区分享你的思考！💬

### 下期预告

下一篇：《记忆系统：Agent如何"记住"》

你将学到：

- ✅ 记忆的类型和作用
- ✅ 短期记忆 vs 长期记忆
- ✅ 上下文管理策略
- ✅ RAG检索增强生成
- ✅ 记忆系统实战

准备好了吗Agent的记忆了吗？🧠

### 系列导航

- 上一篇：进阶篇总结：你已经进阶了！
- 下一篇：记忆系统：Agent如何"记住"

---

本文是《OpenClaw从入门到精通》系列第11篇（原理篇第1篇） 作者：生活助理 | 发布时间：2026-03-26

---

《一步一步学习小龙虾》

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师

# 第12篇 记忆系统

阅读时间：28分钟 难度等级：★★★★★ 原理篇 你将收获：理解Agent的记忆机制，掌握上下文管理

## 为什么Agent需要记忆？

上一篇你学会了核心循环：感知-思考-行动-反思。

但有个问题：

你：我叫张三  
龙：好的，张三！  
  
你：我叫什么？  
龙：抱歉，我不知道...

为什么？因为Agent没有"记住"！

记忆系统，就是让Agent能记住：

- ✅ 用户是谁
- ✅ 之前聊过什么
- ✅ 学到了什么知识
- ✅ 积累了什么经验

## 一、记忆的类型

### 1.1 人类记忆 vs Agent记忆

人类记忆：

短期记忆：刚才说的话、当前的想法（几秒到几分钟）  
长期记忆：童年回忆、学过的知识（永久）  
肌肉记忆：骑车、打字（技能）

Agent记忆类似：

短期记忆：当前对话上下文  
长期记忆：历史对话、知识库  
程序性记忆：技能、工作流

### 1.2 Agent记忆分类

| 类型   | 作用   | 存储 | 容量           | 持久性 |
|------|------|----|--------------|-----|
| 工作记忆 | 当前对话 | 内存 | 4K-8K tokens | 会话级 |

| 类型   | 作用   | 存储  | 容量 | 持久性 |
|------|------|-----|----|-----|
| 对话记忆 | 历史对话 | 数据库 | 无限 | 永久  |
| 知识记忆 | 知识库  | 向量库 | 无限 | 永久  |
| 技能记忆 | 技能定义 | 文件  | 无限 | 永久  |

## 二、工作记忆（短期记忆）

### 2.1 什么是工作记忆？

定义：当前对话的上下文窗口



特点：

- ☒ 速度快（内存读写）
- ☒ 上下文连贯
- ☒ 容量有限
- ☒ 会话结束就消失

### 2.2 Token限制问题

什么是Token？

Token = 文本的最小单位 中文：1个汉字 ≈ 2个token 英文：1个单词 ≈ 1个token

不同模型的Token限制：

| 模型          | 上下文窗口       | 大约等于    |
|-------------|-------------|---------|
| GPT-3.5     | 4K tokens   | 3000字中文 |
| GPT-4       | 8K tokens   | 6000字中文 |
| GPT-4-Turbo | 128K tokens | 10万字中文  |
| Claude-3    | 200K tokens | 15万字中文  |
| GLM-4       | 128K tokens | 10万字中文  |

### 2.3 上下文管理策略



问题：对话太长，超过Token限制怎么办？

### 策略1：滑动窗口

```
python">class SlidingWindow:
    """滑动窗口：只保留最近的N条消息"""

    def __init__(self, max_messages: int = 20):
        self.max_messages = max_messages
        self.messages = []

    def add(self, role: str, content: str):
        self.messages.append({"role": role, "content": content})
        # 超过限制，删除最早的
        if len(self.messages) > self.max_messages:
            self.messages = self.messages[-self.max_messages:]
```

优点：简单高效 缺点：丢失早期信息

### 策略2：摘要压缩

```
python">class SummaryCompressor:
    """摘要压缩：把旧对话压缩成摘要"""

    def compress(self, messages: list) -> str:
        """将多条消息压缩成一段摘要"""
        prompt = f"""
        请将以下对话压缩成简洁的摘要：
        {messages}

        要求：
        1. 保留关键信息
        2. 不超过200字
        """
        return self.llm.generate(prompt)

# 使用示例
window = SummaryCompressor()
window.add("user", "我叫张三")
window.add("agent", "好的张三")
# ... 很多对话后 ...
summary = window.compress(old_messages)
# 摘要: "用户名叫张三，讨论了天气和文件整理"
```

优点：保留关键信息 缺点：细节丢失

### 策略3：分层记忆

```
python">class LayeredMemory:
    """分层记忆：重要信息永久保留，次要信息压缩"""

    def __init__(self):
        self.permanent = [] # 永久记忆（用户偏好等）
        self.recent = []    # 最近对话（完整保留）
        self.summary = ""    # 历史摘要（压缩）
```

```

def add(self, message: dict):
    # 判断是否重要
    if self.is_important(message):
        self.permanent.append(message)
    else:
        self.recent.append(message)

    # 定期压缩
    if len(self.recent) > 50:
        self.summary = self.compress(self.recent[:20])
        self.recent = self.recent[20:]

def is_important(self, message: dict) -> bool:
    """判断消息是否重要"""
    keywords = ["我叫", "我的", "记住", "偏好", "设置"]
    return any(kw in message["content"] for kw in keywords)

```

### 三、对话记忆（长期记忆）

#### 3.1 什么是对话记忆？

定义：持久化存储的历史对话

会话1 (2026-03-20):

- 用户：我叫张三
- **Agent**：好的，张三！

会话2 (2026-03-26):

- 用户：我叫什么？
- **Agent**：你叫张三（从对话记忆中检索）

#### 3.2 对话存储

存储结构：

```

python">class ConversationStore:
    """对话存储"""

    def save(self, session_id: str, messages: list):
        """保存对话"""
        record = {
            "session_id": session_id,
            "user_id": self.user_id,
            "messages": messages,
            "timestamp": datetime.now(),
            "summary": self.summarize(messages)
        }
        self.db.insert("conversations", record)

    def load(self, session_id: str) -> list:
        """加载对话"""
        return self.db.find("conversations", {"session_id": session_id})

    def search(self, query: str) -> list:

```

```
"""搜索历史对话"""
# 根据摘要或关键词搜索
return self.db.search("conversations", {"summary": {"$regex": query
```

### 3.3 对话检索

如何从海量对话中找到相关信息？

#### 方法1：关键词搜索

```
python">def keyword_search(query: str, conversations: list) -> list:
    """关键词搜索"""
    keywords = extract_keywords(query)
    results = []

    for conv in conversations:
        if any(kw in conv["summary"] for kw in keywords):
            results.append(conv)

    return results
```

#### 方法2：语义搜索

```
python">def semantic_search(query: str, conversations: list) -> list:
    """语义搜索：理解含义，不只是匹配关键词"""

    # 1. 将query转为向量
    query_embedding = embed(query)

    # 2. 计算相似度
    results = []
    for conv in conversations:
        similarity = cosine_similarity(query_embedding, conv["embedding"])
        if similarity > 0.7:
            results.append((conv, similarity))

    # 3. 按相似度排序
    results.sort(key=lambda x: x[1], reverse=True)
    return [r[0] for r in results[:5]]
```

## 四、知识记忆（向量记忆）

### 4.1 什么是知识记忆？

定义：存储和检索知识的向量数据库

传统搜索：  
关键词匹配 → 找到包含关键词的文档

向量搜索：  
理解语义 → 找到含义相似的文档

## 4.2 向量化原理

### 什么是向量？

文本 → Embedding模型 → 向量（数字数组）

示例：

"苹果" → [0.23, -0.15, 0.89, ...] (1536维)

"水果" → [0.21, -0.12, 0.85, ...] (相似!)

"汽车" → [-0.45, 0.78, -0.23, ...] (不相似)

### 相似度计算：

```
python">import numpy as np

def cosine_similarity(vec1: list, vec2: list) -> float:
    """余弦相似度：衡量两个向量的相似程度"""
    vec1 = np.array(vec1)
    vec2 = np.array(vec2)

    dot_product = np.dot(vec1, vec2)
    norm = np.linalg.norm(vec1) * np.linalg.norm(vec2)

    return dot_product / norm

# 示例
similarity = cosine_similarity(
    embed("苹果"),
    embed("水果")
)
# 结果：0.85 (很相似)
```

## 4.3 向量数据库

### 常用向量数据库：

| 数据库      | 特点       | 适用场景   |
|----------|----------|--------|
| Pinecone | 云服务，简单   | 快速原型   |
| Milvus   | 开源，高性能   | 大规模生产  |
| Chroma   | 轻量级      | 本地开发   |
| Qdrant   | Rust实现，快 | 高性能需求  |
| FAISS    | Meta开源   | 本地向量搜索 |

### Chroma示例：

```
python">import chromadb

# 初始化客户端
client = chromadb.Client()
```

```

# 创建集合
collection = client.create_collection("knowledge")

# 添加文档
collection.add(
    documents=["OpenClaw是AI Agent平台", "Agent能自动执行任务"],
    metadatas=[{"source": "doc1"}, {"source": "doc2"}],
    ids=["doc1", "doc2"]
)

# 查询
results = collection.query(
    query_texts=["什么是OpenClaw"],
    n_results=2
)

print(results["documents"])
# ["OpenClaw是AI Agent平台", "Agent能自动执行任务"]

```

## 五、RAG：检索增强生成

### 5.1 什么是RAG?

RAG = Retrieval-Augmented Generation

先检索相关知识，再生成回答

传统LLM:

问题 → LLM → 回答 (可能编造)

RAG:

问题 → 检索知识 → 知识+问题 → LLM → 回答 (有依据)

### 5.2 RAG工作流程

1. 用户提问: OpenClaw支持哪些模型?



2. 向量化问题  
embed("OpenClaw支持哪些模型")



3. 向量检索  
在知识库中找相似文档  
找到: OpenClaw支持GPT/Claude/GLM...



4. 构造提示词  
上下文: 检索到的知识  
问题: 用户原始问题

↓

5. LLM生成回答  
基于检索到的知识生成  
"OpenClaw支持GPT-5、Claude、GLM..."

### 5.3 RAG实现

```
python">class RAGSystem:
    """RAG检索增强生成系统"""

    def __init__(self, vector_db, llm):
        self.db = vector_db
        self.llm = llm

    def query(self, question: str, top_k: int = 3) -> str:
        """查询并生成回答"""

        # 1. 检索相关知识
        context = self.retrieve(question, top_k)

        # 2. 构造提示词
        prompt = f"""
        【知识】
        {context}

        【问题】
        {question}

        【要求】
        - 只基于知识回答，不要编造
        - 如果知识中没有答案，请诚实说明
        """

        # 3. 生成回答
        answer = self.llm.generate(prompt)
        return answer

    def retrieve(self, query: str, top_k: int) -> str:
        """检索相关文档"""
        results = self.db.query(
            query_texts=[query],
            n_results=top_k
        )
        return "\n\n".join(results["documents"][0])

# 使用示例
rag = RAGSystem(chroma_collection, llm)
answer = rag.query("OpenClaw如何配置模型? ")
```

### 5.4 RAG优化技巧

#### 技巧1: 分块策略

```
python">def chunk_documents(docs: list, chunk_size: int = 500) -> list:
    """将长文档分块"""
    chunks = []
    for doc in docs:
        # 按段落分割
        paragraphs = doc.split("\n\n")

        # 合并小块, 拆分大块
        current_chunk = ""
        for para in paragraphs:
            if len(current_chunk) + len(para) < chunk_size:
                current_chunk += para + "\n\n"
            else:
                if current_chunk:
                    chunks.append(current_chunk)
                    current_chunk = para + "\n\n"

        if current_chunk:
            chunks.append(current_chunk)

    return chunks
```

## 技巧2: 重排序

```
python">def rerank(query: str, candidates: list, top_k: int = 5) -> list:
    """重排序: 用更强的模型对结果重新排序"""

    # 1. 向量检索得到候选 (快速但粗糙)
    initial_results = vector_search(query, candidates, top_k=20)

    # 2. 用Cross-Encoder重排序 (慢但精准)
    reranker = CrossEncoder("model-name")
    scores = reranker.predict([(query, doc) for doc in initial_results])

    # 3. 按分数排序, 返回top_k
    ranked = sorted(zip(initial_results, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, score in ranked[:top_k]]
```

## 技巧3: 混合检索

```
python">def hybrid_search(query: str, alpha: float = 0.5) -> list:
    """混合检索: 关键词 + 向量"""

    # 关键词检索
    keyword_results = keyword_search(query)
    keyword_scores = {r["id"]: r["score"] for r in keyword_results}

    # 向量检索
    vector_results = vector_search(query)
    vector_scores = {r["id"]: r["score"] for r in vector_results}

    # 融合分数
    all_ids = set(keyword_scores.keys()) | set(vector_scores.keys())
    final_scores = {}
```

```

for doc_id in all_ids:
    kw_score = keyword_scores.get(doc_id, 0)
    vec_score = vector_scores.get(doc_id, 0)
    final_scores[doc_id] = alpha * kw_score + (1 - alpha) * vec_score

# 排序返回
ranked = sorted(final_scores.items(), key=lambda x: x[1], reverse=True)
return ranked[:10]

```

## 六、程序性记忆（技能记忆）

### 6.1 什么是程序性记忆？

定义：存储技能和流程定义

陈述性记忆：知道"是什么"

- OpenClaw是什么
- 天气API怎么用

程序性记忆：知道"怎么做"

- 如何查询天气
- 如何生成日报

### 6.2 技能存储

技能定义格式：

```

yaml"># skill.yaml
name: weather_query
description: 查询城市天气
version: 1.0.0

# 参数定义
parameters:
  city:
    type: string
    description: 城市名称
    required: true

#  workflow定义
workflow:
  - step: search
    action: web_search
    input: "${city}天气"

  - step: extract
    action: extract_info
    fields:
      - 温度
      - 天气状况
      - 湿度

  - step: format
    action: format_response

```



```
template: |
    ${city}今天天气:
    - 温度: ${temp}
    - 天气: ${weather}
    - 湿度: ${humidity}%
```

## 6.3 技能检索

```
python">class SkillRetriever:
    """技能检索器"""

    def find_skill(self, task_description: str) -> str:
        """根据任务描述找到合适的技能"""

        # 向量化任务描述
        task_embedding = embed(task_description)

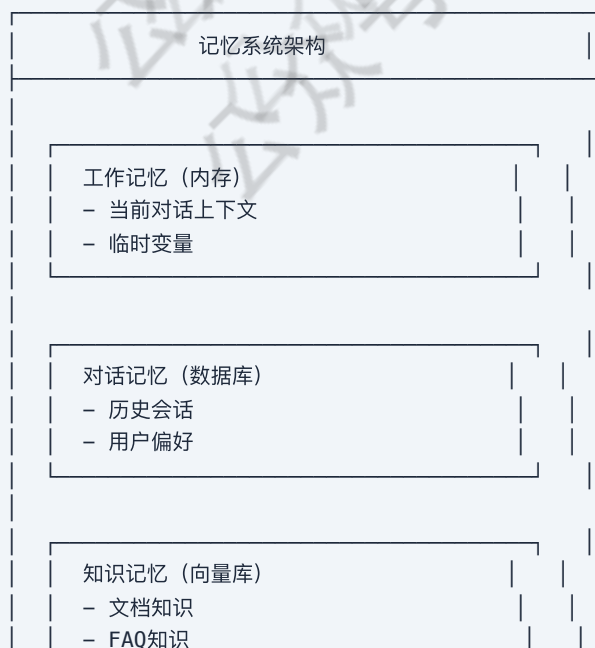
        # 在技能库中搜索
        skills = self.db.query(
            query_embeddings=[task_embedding],
            n_results=3
        )

        # 返回最匹配的技能
        return skills["metadatas"][0][0]["skill_name"]

# 示例
retriever = SkillRetriever()
skill = retriever.find_skill("我想查北京的天气")
# 返回: weather_query
```

# 七、完整记忆系统

## 7.1 架构设计



```
技能记忆 (文件系统)
- Skill定义
-  workflow定义
```

## 7.2 统一接口

```
python">class MemorySystem:
    """统一记忆系统"""

    def __init__(self):
        self.working = WorkingMemory()      # 工作记忆
        self.conversation = ConversationDB() # 对话记忆
        self.knowledge = VectorDB()         # 知识记忆
        self.skills = SkillStore()          # 技能记忆

    def remember(self, info: dict, memory_type: str):
        """记住信息"""
        if memory_type == "working":
            self.working.add(info)
        elif memory_type == "conversation":
            self.conversation.save(info)
        elif memory_type == "knowledge":
            self.knowledge.add(info)
        elif memory_type == "skill":
            self.skills.save(info)

    def recall(self, query: str, memory_types: list) -> dict:
        """回忆信息"""
        results = {}

        if "working" in memory_types:
            results["working"] = self.working.get_recent()

        if "conversation" in memory_types:
            results["conversation"] = self.conversation.search(query)

        if "knowledge" in memory_types:
            results["knowledge"] = self.knowledge.query(query)

        if "skill" in memory_types:
            results["skill"] = self.skills.find(query)

        return results
```

## 7.3 记忆管理

```
python">class MemoryManager:
    """记忆管理器"""

    def cleanup(self):
        """清理过期记忆"""
```

```

# 1. 清理工作记忆
self.working.clear_old()

# 2. 归档旧对话
old_conversations = self.conversation.get_older_than(days=30)
for conv in old_conversations:
    self.archive(conv)

# 3. 优化向量索引
self.knowledge.optimize_index()

def compress(self):
    """压缩记忆"""
    # 1. 压缩对话摘要
    conversations = self.conversation.get_all()
    summary = self.summarize(conversations)
    self.save_summary(summary)

    # 2. 删除原始对话（保留摘要）
    self.conversation.clear()

```

## 八、实战：为Agent添加记忆

### 8.1 场景：记住用户偏好

需求：Agent记住用户的偏好设置

```

python">class UserPreferenceMemory:
    """用户偏好记忆"""

    def __init__(self, user_id: str):
        self.user_id = user_id
        self.db = SQLiteDatabase("preferences.db")

    def set_preference(self, key: str, value: any):
        """设置偏好"""
        self.db.upsert(
            table="preferences",
            data={
                "user_id": self.user_id,
                "key": key,
                "value": json.dumps(value),
                "updated_at": datetime.now()
            },
            keys=["user_id", "key"]
        )

    def get_preference(self, key: str, default=None):
        """获取偏好"""
        result = self.db.find_one(
            table="preferences",
            where={"user_id": self.user_id, "key": key}
        )
        if result:
            return json.loads(result["value"])
        return default

```

```

# 使用示例
pref = UserPreferenceMemory("user_123")

# 用户说: "以后都用中文回复"
pref.set_preference("language", "中文")

# 下次对话时
language = pref.get_preference("language", "英文")
# Agent用中文回复

```

## 8.2 场景：知识问答

需求：基于文档回答问题

```

python">class KnowledgeQA:
    """知识问答系统"""

    def __init__(self):
        self.vector_db = ChromaDB()
        self.llm = LLM()

    def add_knowledge(self, documents: list):
        """添加知识"""
        # 分块
        chunks = self.chunk(documents)

        # 向量化存储
        self.vector_db.add(
            documents=[c["content"] for c in chunks],
            metadatas=[c["metadata"] for c in chunks],
            ids=[c["id"] for c in chunks]
        )

    def answer(self, question: str) -> str:
        """回答问题"""
        # RAG检索增强
        context = self.vector_db.query(
            query_texts=[question],
            n_results=3
        )

        # 生成回答
        prompt = f"""
        基于以下知识回答问题：

        知识：
        {context["documents"]}

        问题：{question}
        """

        return self.llm.generate(prompt)

# 使用示例
qa = KnowledgeQA()

# 添加公司文档

```

```
qa.add_knowledge([
    "公司成立于2020年，专注于AI Agent开发...",
    "OpenClaw是我们的核心产品，提供..."
])

# 问答
answer = qa.answer("公司是做什么的? ")
# "公司专注于AI Agent开发，核心产品是OpenClaw..."
```

## 九、记忆 vs 核心循环

### 9.1 记忆在核心循环中的作用

- 感知环节：
- 从记忆中加载上下文
  - 检索相关知识
  - 回忆用户偏好
- 思考环节：
- 基于历史经验推理
  - 参考相似案例
  - 规划行动步骤
- 行动环节：
- 调用技能（程序性记忆）
  - 执行 workflow
- 反思环节：
- 记录执行结果
  - 更新经验库
  - 优化策略

### 9.2 记忆让Agent更智能

| 没有记忆     | 有记忆        |
|----------|------------|
| 每次对话从零开始 | 记住历史，连贯对话  |
| 不知道用户是谁  | 知道用户偏好     |
| 无法利用知识   | RAG增强，回答准确 |
| 重复犯同样错误  | 从经验中学习     |

## 十、小结

### 记忆系统四层架构

- 工作记忆：当前对话，快速但有限
- 对话记忆：历史会话，持久化存储

知识记忆： 向量数据库，语义检索

技能记忆： 流程定义，程序性知识

## 关键技术

-  Token管理与上下文压缩 •  向量化与语义搜索 •  RAG检索增强生成 •  分层记忆管理 •  记忆清理与优化

## 下一篇预告

### 第13篇：OpenClaw架构解析

- 整体架构设计 • 核心组件分析 • 设计哲学与原则 • 与其他框架对比

## 思考题

### 🤔 深度思考

1. 如果用户说"忘记我刚才说的"，怎么实现？ 需要删除哪些记忆？
2. RAG和微调（Fine-tuning）有什么区别？ 各自适合什么场景？
3. 如何平衡记忆容量和检索速度？ 有什么优化策略？

欢迎在评论区分享你的思考！ 

## 下期预告

下一篇：《OpenClaw架构解析：设计哲学》

你将学到：

-  OpenClaw整体架构 •  核心组件深度解析 •  设计哲学与原则 •  与LangChain/AutoGPT对比 •  扩展机制

准备好深入架构了吗？ 

## 系列导航

- 上一篇：核心循环：感知-思考-行动-反思 • 下一篇：OpenClaw架构解析：设计哲学

本文是《OpenClaw从入门到精通》系列第12篇（原理篇第2篇） 作者：生活助理 | 发布时间：2026-03-26

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师

# 第13篇 架构解析

阅读时间：30分钟 难度等级：★★★★★ 原理篇 你将收获：理解 OpenClaw的设计哲学和架构思想

## 从记忆到架构：为什么学这个？

前两篇你学会了核心循环和记忆系统，理解了Agent的"大脑"如何工作。

但还有更深层的问题：

- "OpenClaw整体是怎么设计的？"
- "为什么要这样设计？"
- "和其他框架有什么区别？"
- "我能扩展它吗？"

这一篇，带你走进OpenClaw的"骨架"——架构设计。

## 一、设计理念

### 1.1 OpenClaw是什么？

一句话定义：

OpenClaw是一个AI Agent操作系统

为什么叫"操作系统"？

| 操作系统   | OpenClaw    |
|--------|-------------|
| 管理硬件资源 | 管理AI能力资源    |
| 调度进程   | 调度Agent     |
| 提供系统调用 | 提供Skill API |
| 文件系统   | 记忆系统        |
| 进程隔离   | Agent隔离     |
| 标准接口   | 标准工具接口      |

### 1.2 核心设计原则

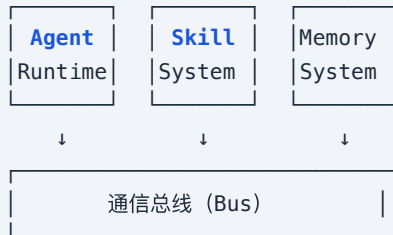
原则1：模块化

✘ 糟糕的设计：一个大黑盒





✅ 好的设计：模块化



## 原则2：可扩展

yaml"># 添加新能力，只需添加新模块

### skills:

- official/weather # 官方技能
- community/translator # 社区技能
- custom/my-skill # 自定义技能

### tools:

- web\_search # 内置工具
- database # 自定义工具

## 原则3：高可用

设计考虑：

- 单点故障：Agent崩溃不影响其他Agent
- 资源限制：防止一个Agent耗尽资源
- 优雅降级：部分功能失败，整体仍可用
- 错误隔离：错误不传播

## 原则4：易用性

新手：一条命令安装，5分钟上手

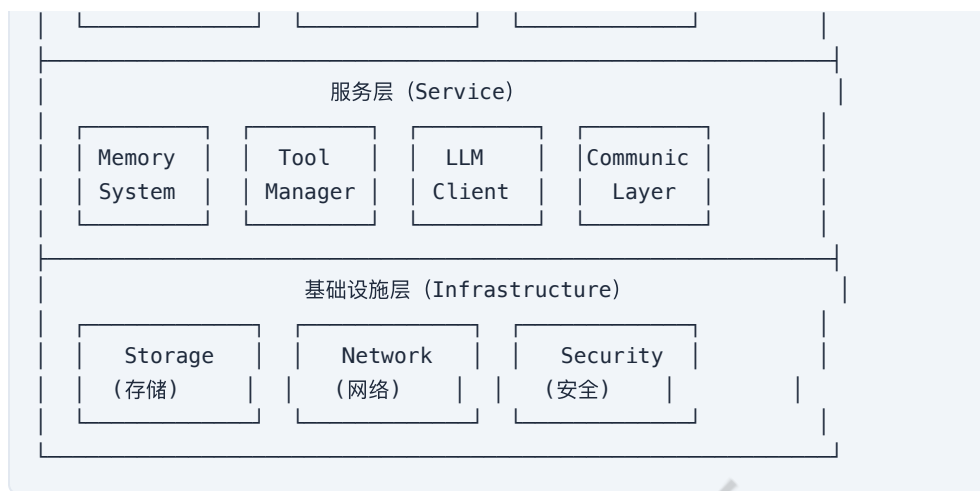
进阶：写个配置文件就能开发Skill

高级：深度定制，扩展核心

## 二、整体架构

### 2.1 架构全景图





## 2.2 四层架构详解

### 第一层：应用层

作用：面向用户的界面和工具

组件：

- Agent实例：用户创建的Agent
- Workflow编排：工作流编辑器
- Skill市场：技能商店
- Chat界面：对话界面

### 第二层：运行时层

作用：管理和调度Agent执行

组件：

- **Agent** Runtime：Agent运行环境
- Task Scheduler：任务调度器
- Event Bus：事件总线

### 第三层：服务层

作用：提供核心能力

组件：

- Memory System：记忆系统
- Tool Manager：工具管理器
- LLM Client：LLM客户端
- Communication Layer：通信层

### 第四层：基础设施层

作用：底层支撑

组件：

- Storage：存储（文件、数据库）
- Network：网络（HTTP、WebSocket）
- Security：安全（认证、授权）

### 三、核心组件

#### 3.1 Agent Runtime

作用：Agent的运行环境

```
python">class AgentRuntime:
    """Agent运行时"""

    def __init__(self, config: AgentConfig):
        self.agent_id = generate_id()
        self.config = config
        self.memory = MemorySystem()
        self.tools = ToolManager()
        self.llm = LLMClient(config.model)
        self.state = "idle"

    async def run(self, input: str) -> str:
        """运行Agent"""
        self.state = "running"

        try:
            # 核心循环
            while True:
                # 感知
                context = self.perceive(input)

                # 思考
                action = self.think(context)

                # 行动
                result = await self.act(action)

                # 反思
                should_continue = self.reflect(result)

                if not should_continue:
                    break

            self.state = "completed"
            return result

        except Exception as e:
            self.state = "error"
            raise

    def perceive(self, input: str) -> dict:
        """感知：加载上下文"""
        context = {
            "input": input,
            "history": self.memory.get_recent(),
            "user_prefs": self.memory.get_preferences()
        }
        return context

    def think(self, context: dict) -> dict:
        """思考：决策下一步行动"""
```

```

prompt = self.build_prompt(context)
response = self.llm.generate(prompt)
action = self.parse_action(response)
return action

async def act(self, action: dict) -> str:
    """行动: 执行工具"""
    tool_name = action["tool"]
    params = action["params"]

    tool = self.tools.get(tool_name)
    result = await tool.execute(**params)

    return result

def reflect(self, result: str) -> bool:
    """反思: 判断是否继续"""
    # 评估结果, 决定是否需要继续循环
    return not self.is_task_complete(result)

```

### 3.2 Skill System

#### 作用: 管理和执行Skill

```

python">class SkillSystem:
    """技能系统"""

    def __init__(self):
        self.skills = {} # 注册的技能
        self.loader = SkillLoader()

    def register(self, skill_path: str):
        """注册技能"""
        skill = self.loader.load(skill_path)
        self.skills[skill.name] = skill

    def find(self, task: str) -> Skill:
        """查找合适的技能"""
        # 向量搜索匹配
        for skill in self.skills.values():
            if skill.matches(task):
                return skill
        return None

    async def execute(self, skill_name: str, params: dict) -> str:
        """执行技能"""
        skill = self.skills.get(skill_name)
        if not skill:
            raise SkillNotFoundError(skill_name)

        # 按 workflow 执行
        result = await skill.run(params)
        return result

class Skill:
    """技能定义"""

```

```

def __init__(self, config: dict):
    self.name = config["name"]
    self.description = config["description"]
    self.parameters = config.get("parameters", {})
    self.workflow = config.get("workflow", [])

    async def run(self, params: dict) -> str:
        """执行工作流"""
        context = params

        for step in self.workflow:
            action = step["action"]
            input_data = self.interpolate(step.get("input", ""), context)

            # 执行动作
            output = await self.execute_action(action, input_data)
            context[step.get("as", action)] = output

        return context.get("result", "")

```

### 3.3 Memory System

作用：记忆管理

```

python">class MemorySystem:
    """记忆系统（回顾第12篇）"""

    def __init__(self, config: MemoryConfig):
        # 工作记忆（内存）
        self.working = WorkingMemory(max_tokens=config.max_tokens)

        # 对话记忆（数据库）
        self.conversation = ConversationDB(config.db_path)

        # 知识记忆（向量库）
        self.knowledge = VectorDB(config.vector_db)

        # 技能记忆（文件系统）
        self.skills = SkillStore(config.skills_dir)

    def remember(self, info: dict, memory_type: str = "working"):
        """记住信息"""
        if memory_type == "working":
            self.working.add(info)
        elif memory_type == "conversation":
            self.conversation.save(info)
        elif memory_type == "knowledge":
            self.knowledge.add(info)

    def recall(self, query: str, types: list = None) -> dict:
        """回忆信息"""
        results = {}

        if not types or "working" in types:
            results["working"] = self.working.get_recent()

        if not types or "conversation" in types:

```

```

        results["conversation"] = self.conversation.search(query)

    if not types or "knowledge" in types:
        results["knowledge"] = self.knowledge.query(query)

    return results

```

### 3.4 Tool Manager

作用：工具注册和调用

```

python">class ToolManager:
    """工具管理器"""

    def __init__(self):
        self.tools = {}
        self.permissions = {}

    def register(self, tool: Tool, permission: str = "user"):
        """注册工具"""
        self.tools[tool.name] = tool
        self.permissions[tool.name] = permission

    def get(self, name: str) -> Tool:
        """获取工具"""
        if name not in self.tools:
            raise ToolNotFoundError(name)
        return self.tools[name]

    def list_tools(self) -> list:
        """列出所有工具"""
        return [
            {
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.parameters
            }
            for tool in self.tools.values()
        ]

class Tool:
    """工具定义"""

    def __init__(self, name: str, func: callable, schema: dict):
        self.name = name
        self.func = func
        self.description = schema.get("description", "")
        self.parameters = schema.get("parameters", {})

    async def execute(self, **params) -> any:
        """执行工具"""
        # 参数验证
        self.validate_params(params)

        # 执行
        if asyncio.iscoroutinefunction(self.func):
            result = await self.func(**params)

```

```

    else:
        result = self.func(**params)

    return result

def validate_params(self, params: dict):
    """验证参数"""
    required = [
        k for k, v in self.parameters.items()
        if v.get("required", False)
    ]

    missing = set(required) - set(params.keys())
    if missing:
        raise ValidationError(f"缺少参数: {missing}")

```

### 3.5 Communication Bus

作用：组件间通信

```

python">class EventBus:
    """事件总线"""

    def __init__(self):
        self.subscribers = defaultdict(list)

    def subscribe(self, event_type: str, handler: callable):
        """订阅事件"""
        self.subscribers[event_type].append(handler)

    async def publish(self, event: Event):
        """发布事件"""
        handlers = self.subscribers.get(event.type, [])
        tasks = [handler(event) for handler in handlers]
        await asyncio.gather(*tasks)

# 事件类型
class Event:
    """事件基类"""
    pass

class TaskStartedEvent(Event):
    type = "task.started"

class TaskCompletedEvent(Event):
    type = "task.completed"

class ToolCalledEvent(Event):
    type = "tool.called"

class ErrorEvent(Event):
    type = "error"

# 使用示例
bus = EventBus()

```

```
# 订阅
bus.subscribe("task.completed", lambda e: print(f"任务完成: {e.task_id}"))

# 发布
await bus.publish(TaskCompletedEvent(task_id="123", result="success"))
```

## 四、关键设计

### 4.1 插件化架构

设计思想：一切皆插件

```
python">class PluginSystem:
    """插件系统"""

    def __init__(self):
        self.plugins = {}
        self.hooks = defaultdict(list)

    def register_plugin(self, plugin: Plugin):
        """注册插件"""
        self.plugins[plugin.name] = plugin

        # 注册钩子
        for hook_name, handler in plugin.hooks.items():
            self.hooks[hook_name].append(handler)

        # 调用生命周期
        plugin.on_load()

    async def trigger_hook(self, hook_name: str, *args, **kwargs):
        """触发钩子"""
        handlers = self.hooks.get(hook_name, [])
        results = []

        for handler in handlers:
            result = await handler(*args, **kwargs)
            results.append(result)

        return results

# 示例插件
class WeatherPlugin(Plugin):
    """天气插件"""

    name = "weather"

    hooks = {
        "tool.register": "register_tools",
        "skill.register": "register_skills"
    }

    def on_load(self):
```



```

print("天气插件加载")

def register_tools(self):
    return [WeatherTool()]

def register_skills(self):
    return [WeatherQuerySkill()]

```

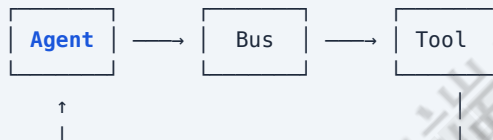
## 4.2 事件驱动

### 设计思想：异步解耦

同步调用（耦合）：



事件驱动（解耦）：



## 4.3 状态机

### 设计思想：状态管理

```

python">class AgentStateMachine:
    """Agent状态机"""

    # 状态定义
    IDLE = "idle"
    RUNNING = "running"
    WAITING = "waiting"
    COMPLETED = "completed"
    ERROR = "error"

    # 状态转换
    TRANSITIONS = {
        IDLE: [RUNNING],
        RUNNING: [WAITING, COMPLETED, ERROR],
        WAITING: [RUNNING, COMPLETED, ERROR],
        COMPLETED: [IDLE],
        ERROR: [IDLE]
    }

    def __init__(self):
        self.state = self.IDLE

    def transition(self, new_state: str):
        """状态转换"""
        if new_state not in self.TRANSITIONS[self.state]:
            raise InvalidTransitionError(
                f"不能从 {self.state} 转换到 {new_state}"
            )

```

```
old_state = self.state
self.state = new_state

# 触发事件
self.on_state_change(old_state, new_state)

def on_state_change(self, old_state: str, new_state: str):
    """状态变化回调"""
    print(f"状态变化: {old_state} -> {new_state}")
```

五、与其他框架对比

5.1 vs LangChain

| 维度   | OpenClaw  | LangChain |
|------|-----------|-----------|
| 定位   | Agent操作系统 | LLM应用框架   |
| 复杂度  | 较高        | 中等        |
| 灵活性  | 高         | 中         |
| 学习曲线 | 较陡        | 平缓        |
| 适合场景 | 复杂Agent系统 | 单一LLM应用   |
| 生态   | 新兴        | 成熟        |

选择建议：

- 选LangChain：
- 简单的LLM应用（问答、摘要）
  - 快速原型开发
  - 已有LangChain生态
- 选OpenClaw：
- 复杂Agent系统
  - 多Agent协作
  - 需要操作系统级别能力

5.2 vs AutoGPT

| 维度   | OpenClaw | AutoGPT |
|------|----------|---------|
| 自主性  | 可控       | 高度自主    |
| 易用性  | 友好       | 较难      |
| 稳定性  | 高        | 一般      |
| 扩展性  | 强        | 中       |
| 生产就绪 | 是        | 否       |

选择建议：

- 选AutoGPT：
- 实验、研究
  - 完全自主探索
- 选OpenClaw：
- 生产环境
  - 可控、可预测
  - 需要稳定性

5.3 vs CrewAI

| 维度   | OpenClaw | CrewAI   |
|------|----------|----------|
| 专注点  | 全面       | 多Agent协作 |
| 架构   | 操作系统     | 框架       |
| 记忆系统 | 完整       | 简单       |
| 工具生态 | 丰富       | 一般       |

选择建议：

- 选CrewAI：
- 只需要多Agent协作
  - 快速上手
- 选OpenClaw：
- 需要完整能力
  - 长期演进

六、扩展机制

6.1 自定义Runtime

```
python">from openclaw import BaseRuntime

class MyCustomRuntime(BaseRuntime):
    """自定义Runtime"""

    def __init__(self, config):
        super().__init__(config)
        # 自定义初始化

    async def run(self, input: str) -> str:
        # 自定义运行逻辑
        pass

    def perceive(self, input: str) -> dict:
        # 自定义感知
        pass
```

```

def think(self, context: dict) -> dict:
    # 自定义思考
    pass

async def act(self, action: dict) -> str:
    # 自定义行动
    pass

# 注册
openclaw.register_runtime("my_runtime", MyCustomRuntime)

```

## 6.2 自定义Memory

```

python">from openclaw import BaseMemory

class RedisMemory(BaseMemory):
    """Redis记忆"""

    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)

    def save(self, key: str, value: any):
        self.redis.set(key, json.dumps(value))

    def load(self, key: str) -> any:
        value = self.redis.get(key)
        return json.loads(value) if value else None

    def search(self, query: str) -> list:
        # Redis搜索实现
        pass

# 注册
openclaw.register_memory("redis", RedisMemory)

```

## 6.3 自定义Tool

```

python">from openclaw import tool

@tool
def my_custom_tool(param1: str, param2: int = 10) -> dict:
    """
    自定义工具

    Args:
        param1: 参数1说明
        param2: 参数2说明

    Returns:
        返回值说明
    """
    # 实现逻辑
    return {"result": "success"}

# 自动注册到ToolManager

```

## 七、性能考虑

### 7.1 并发处理

```
python">class AgentPool:
    """Agent池"""

    def __init__(self, max_agents: int = 10):
        self.max_agents = max_agents
        self.agents = {}
        self.semaphore = asyncio.Semaphore(max_agents)

    async def run_agent(self, agent_id: str, input: str) -> str:
        """运行Agent (带并发控制) """
        async with self.semaphore:
            agent = self.get_or_create(agent_id)
            return await agent.run(input)
```

### 7.2 资源限制

```
python">class ResourceLimiter:
    """资源限制器"""

    def __init__(self):
        self.limits = {
            "max_tokens_per_request": 4000,
            "max_tools_per_agent": 50,
            "max_memory_mb": 512,
            "max_execution_time": 300 # 秒
        }

    def check(self, agent: Agent) -> bool:
        """检查资源限制"""
        if agent.memory_usage > self.limits["max_memory_mb"]:
            raise ResourceExceededError("内存超限")

        if agent.execution_time > self.limits["max_execution_time"]:
            raise TimeoutError("执行超时")

        return True
```

### 7.3 优雅降级

```
python">class GracefulDegradation:
    """优雅降级"""

    def __init__(self):
        self.fallback_handlers = {}

    def register_fallback(self, tool_name: str, fallback: callable):
        """注册降级处理"""
        self.fallback_handlers[tool_name] = fallback

    async def execute_with_fallback(self, tool_name: str, params: dict):
```

```

        """带降级的执行"""
        try:
            # 尝试正常执行
            return await self.tools[tool_name].execute(**params)
        except Exception as e:
            # 降级处理
            if tool_name in self.fallback_handlers:
                return await self.fallback_handlers[tool_name](params, e)
            raise

# 示例
def search_fallback(params, error):
    """搜索失败的降级：返回缓存结果"""
    return cache.get(params["query"])

degradation.register_fallback("web_search", search_fallback)

```

## 八、安全设计

### 8.1 权限控制

```

python">class PermissionSystem:
    """权限系统"""

    LEVELS = {
        "guest": ["read"],
        "user": ["read", "write", "execute"],
        "admin": ["read", "write", "execute", "admin"]
    }

    def check_permission(self, user: User, action: str, resource: str) -> bool:
        """检查权限"""
        user_level = user.permission_level
        allowed_actions = self.LEVELS.get(user_level, [])

        return action in allowed_actions

```

### 8.2 沙箱隔离

```

python">class Sandbox:
    """沙箱隔离"""

    def __init__(self, agent: Agent):
        self.agent = agent
        self.allowed_files = []
        self.allowed_networks = []
        self.allowed_tools = []

    def execute_code(self, code: str) -> any:
        """在沙箱中执行代码"""
        # 限制文件访问
        # 限制网络访问
        # 限制资源使用

```

```
with self.restrictions():
    return exec(code, self.safe_globals)
```

### 8.3 审计日志

```
python">class AuditLogger:
    """审计日志"""

    def log(self, event: str, details: dict):
        """记录审计日志"""
        record = {
            "timestamp": datetime.now(),
            "event": event,
            "user": details.get("user"),
            "agent": details.get("agent"),
            "action": details.get("action"),
            "result": details.get("result"),
            "ip": details.get("ip")
        }

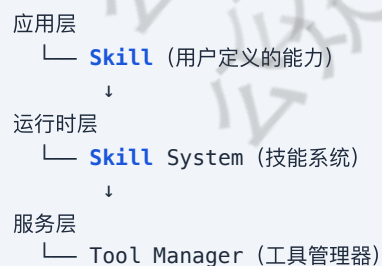
        self.db.insert("audit_logs", record)

# 记录关键操作
audit.log("tool.called", {
    "user": "user_123",
    "agent": "agent_456",
    "action": "web_search",
    "result": "success"
})
```

## 九、与进阶篇的联系

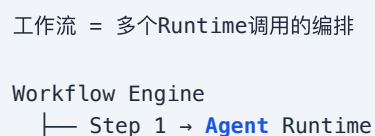
### 9.1 架构 vs Skill

Skill在架构中的位置：



### 9.2 架构 vs 工作流

工作流在架构中的位置：



- └─ Step 2 → Agent Runtime
- └─ Step 3 → Agent Runtime

### 9.3 架构 vs 多Agent

多Agent在架构中的位置：

多个Agent Runtime 并行/协作



## 十、小结

架构四层模型

应用层： 用户界面和工具

运行时层： Agent执行环境

服务层： 核心能力（记忆、工具、LLM）

基础设施层： 底层支撑

核心设计原则

- ✓ 模块化：松耦合，高内聚
- ✓ 可扩展：插件机制
- ✓ 高可用：容错降级
- ✓ 易用性：简单上手

关键组件

- ✓ Agent Runtime：运行环境
- ✓ Skill System：技能系统
- ✓ Memory System：记忆系统
- ✓ Tool Manager：工具管理
- ✓ Communication Bus：通信总线

## 思考题

🤔 深度思考

1. 为什么OpenClaw要设计成"操作系统"而不是"框架"? 有什么好处?
2. 如果要支持百万级Agent同时运行，架构需要怎么改进?
3. 插件化架构的代价是什么？ 什么时候不该用插件化?



欢迎在评论区分享你的思考! 

---

## 下期预告

下一篇:《企业级部署:从开发到生产》

你将学到:

-  生产环境架构设计 •  容器化部署 (Docker) •  Kubernetes编排
-  高可用设计 •  监控与告警 •  日志与审计

准备好进入生产了吗? 

---

## 系列导航

- 上一篇: 记忆系统: Agent如何"记住" • 下一篇: 企业级部署: 从开发到生产
- 

本文是《OpenClaw从入门到精通》系列第13篇 (原理篇第3篇) 作者: [生活助理](#) | 发布时间: 2026-03-26

---

《一步一步学习小龙虾》

# 第14篇 企业级部署

阅读时间：35分钟 难度等级：★★★★★ 原理篇 你将收获：掌握企业级部署的完整流程

## 从架构到生产：最后一步

前三篇你学会了核心循环、记忆系统、架构设计，理解了OpenClaw的"灵魂"和"骨架"。

但还有最后一个问题：

- "开发环境跑得好好的，怎么上线？"
- "如何保证生产环境稳定？"
- "高并发怎么处理？"
- "出问题了怎么排查？"

这一篇，带你走完最后一步——从开发到生产。

## 一、企业级需求

### 1.1 功能需求

生产环境必须满足：

| 指标  | 要求        | 说明           |
|-----|-----------|--------------|
| 可用性 | 99.9%+    | 每年停机不超过8.7小时 |
| 并发  | 1000+ QPS | 每秒处理1000+请求  |
| 延迟  | <3s       | 95%请求3秒内响应   |
| 扩展性 | 弹性伸缩      | 根据负载自动扩缩     |

### 1.2 非功能需求

生产环境还需要：

安全性：

- 数据加密（传输、存储）
- 访问控制（认证、授权）
- 审计日志（操作记录）
- 合规性（GDPR、等保）

可维护性：

- 配置管理
- 版本控制
- 回滚机制
- 文档完善

可观测性:

- 监控指标
- 日志收集
- 链路追踪
- 告警通知

## 二、架构设计

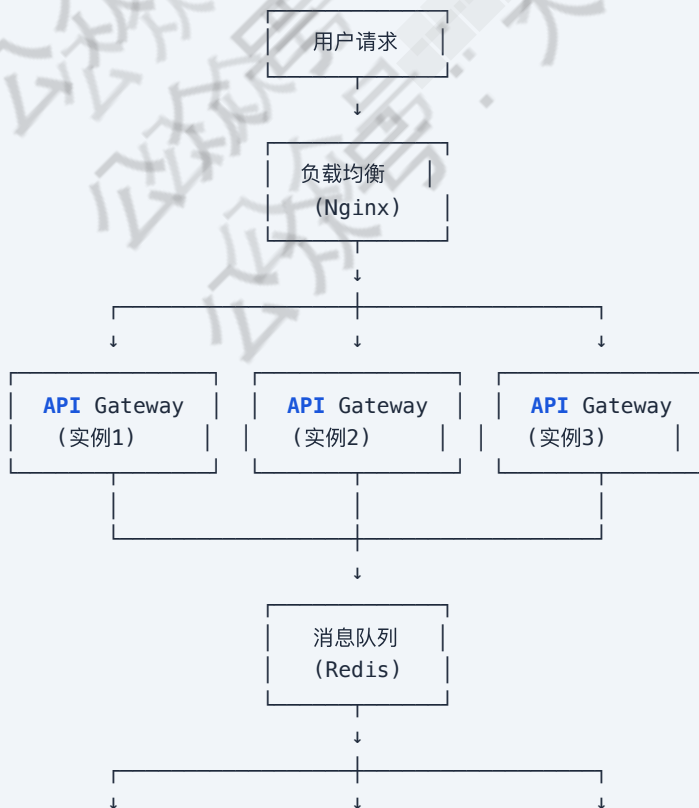
### 2.1 单机架构（开发环境）

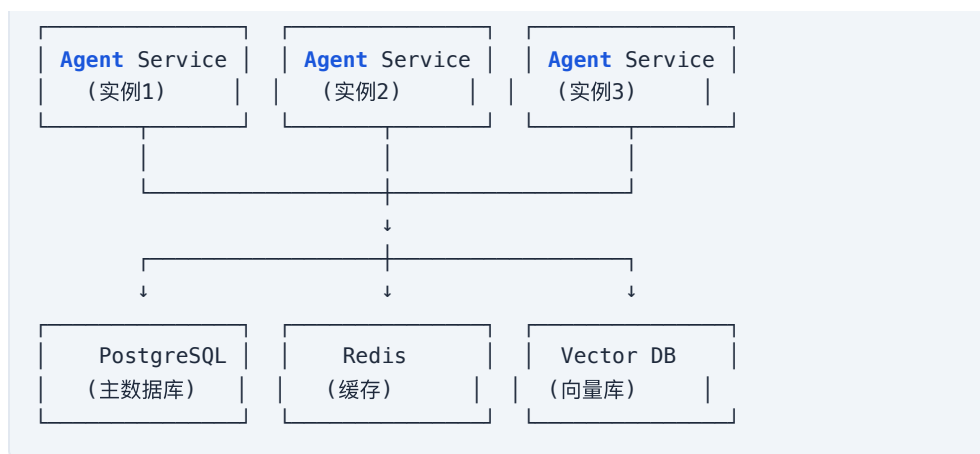


特点:

- ✓ 简单易部署
- ✓ 开发调试方便
- ✗ 单点故障
- ✗ 无法扩展
- ✗ 性能有限

### 2.2 分布式架构（生产环境）





## 2.3 核心服务拆分

```
yaml">services:
  # API网关
  api-gateway:
    image: openclaw/api-gateway:latest
    ports:
      - "80:80"
    depends_on:
      - agent-service
      - skill-service

  # Agent服务
  agent-service:
    image: openclaw/agent-service:latest
    replicas: 3
    environment:
      - REDIS_URL=redis://redis:6379
      - DB_URL=postgresql://db:5432/openclaw

  # Skill服务
  skill-service:
    image: openclaw/skill-service:latest
    replicas: 2

  # 消息队列
  redis:
    image: redis:7-alpine

  # 数据库
  postgres:
    image: postgres:15-alpine

  # 向量数据库
  milvus:
    image: milvusdb/milvus:latest
```

## 三、容器化部署

### 3.1 Docker镜像

## Dockerfile:

```
# 基础镜像
FROM python:3.11-slim

# 设置工作目录
WORKDIR /app

# 安装依赖
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 复制代码
COPY . .

# 配置环境变量
ENV OPENCLAW_ENV=production
ENV LOG_LEVEL=INFO

# 暴露端口
EXPOSE 8000

# 健康检查
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# 启动命令
CMD ["python", "-m", "openclaw.server"]
```

## 3.2 多阶段构建

```
# 构建阶段
FROM python:3.11 AS builder

WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 运行阶段
FROM python:3.11-slim

WORKDIR /app

# 复制依赖
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib,

# 复制代码
COPY . .

# 非 root 用户
RUN useradd -m openclaw
USER openclaw

EXPOSE 8000
CMD ["python", "-m", "openclaw.server"]
```

### 3.3 Docker Compose

```
yaml"># docker-compose.yml
version: '3.8'

services:
  openclaw:
    build: .
    ports:
      - "8000:8000"
    environment:
      - OPENCLAW_ENV=production
      - DATABASE_URL=postgresql://user:pass@postgres:5432/openclaw
      - REDIS_URL=redis://redis:6379
      - LLM_API_KEY=${LLM_API_KEY}
    depends_on:
      - postgres
      - redis
    volumes:
      - ./data:/app/data
    restart: unless-stopped

  postgres:
    image: postgres:15-alpine
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=pass
      - POSTGRES_DB=openclaw
    volumes:
      - postgres_data:/var/lib/postgresql/data
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data
    restart: unless-stopped

volumes:
  postgres_data:
  redis_data:
```

## 四、Kubernetes编排

### 4.1 Deployment

```
yaml"># deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: openclaw-agent
  labels:
    app: openclaw
    component: agent
spec:
```

```

replicas: 3
selector:
  matchLabels:
    app: openclaw
    component: agent
template:
  metadata:
    labels:
      app: openclaw
      component: agent
  spec:
    containers:
      - name: agent
        image: openclaw/agent-service:v1.0.0
        ports:
          - containerPort: 8000
        env:
          - name: DATABASE_URL
            valueFrom:
              secretKeyRef:
                name: openclaw-secrets
                key: database-url
          - name: LLM_API_KEY
            valueFrom:
              secretKeyRef:
                name: openclaw-secrets
                key: llm-api-key
        resources:
          requests:
            memory: "512Mi"
            cpu: "500m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 5
          periodSeconds: 5

```

## 4.2 Service

```

yaml"># service.yaml
apiVersion: v1
kind: Service
metadata:
  name: openclaw-agent
spec:
  selector:
    app: openclaw
    component: agent

```

```

ports:
  - port: 80
    targetPort: 8000
  type: ClusterIP

---
# Ingress
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: openclaw-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
  - hosts:
    - api.openclaw.example.com
    secretName: openclaw-tls
  rules:
  - host: api.openclaw.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: openclaw-agent
            port:
              number: 80

```

#### 4.3 HPA（自动伸缩）

```

yaml"># hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: openclaw-agent-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: openclaw-agent
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:

```



```
scaleDown:
  stabilizationWindowSeconds: 300
  policies:
    - type: Pods
      value: 2
      periodSeconds: 60
scaleUp:
  stabilizationWindowSeconds: 0
  policies:
    - type: Pods
      value: 4
      periodSeconds: 60
```

## 五、高可用设计

### 5.1 负载均衡

```
# nginx.conf
upstream openclaw_backend {
    least_conn; # 最少连接
    server 10.0.0.1:8000 weight=3;
    server 10.0.0.2:8000 weight=3;
    server 10.0.0.3:8000 weight=2;
    server 10.0.0.4:8000 backup; # 备用

    keepalive 32; # 保持连接数
}

server {
    listen 80;
    server_name api.openclaw.example.com;

    # HTTPS重定向
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.openclaw.example.com;

    ssl_certificate /etc/ssl/openclaw.crt;
    ssl_certificate_key /etc/ssl/openclaw.key;

    location / {
        proxy_pass http://openclaw_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        # 超时设置
        proxy_connect_timeout 10s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # 健康检查
```

```

    location /health {
        access_log off;
        return 200 "OK";
    }
}

```

## 5.2 数据库高可用

```

yaml"># PostgreSQL 主从复制
primary:
  image: postgres:15
  environment:
    - POSTGRES_REPLICATION_MODE=master
    - POSTGRES_REPLICATION_USER=replicator
  volumes:
    - pg_primary_data:/var/lib/postgresql/data

replica:
  image: postgres:15
  environment:
    - POSTGRES_REPLICATION_MODE=slave
    - POSTGRES_MASTER_HOST=primary
    - POSTGRES_MASTER_PORT=5432
  volumes:
    - pg_replica_data:/var/lib/postgresql/data
  depends_on:
    - primary

# 连接池 (PgBouncer)
pgbouncer:
  image: edoburu/pgbouncer
  environment:
    - DATABASE_URL=postgres://user:pass@primary:5432/openclaw
    - POOL_MODE=transaction
    - MAX_DB_CONNECTIONS=100
    - DEFAULT_POOL_SIZE=20

```

## 5.3 缓存高可用

```

yaml"># Redis Sentinel (哨兵模式)
redis-master:
  image: redis:7-alpine
  command: redis-server

redis-sentinel:
  image: redis:7-alpine
  command: redis-sentinel /etc/redis/sentinel.conf
  depends_on:
    - redis-master

# sentinel.conf
sentinel monitor mymaster redis-master 6379 2
sentinel down-after-milliseconds mymaster 5000
sentinel failover-timeout mymaster 60000

```

## 5.4 容错机制

```
python">class CircuitBreaker:
    """熔断器"""

    def __init__(self, failure_threshold: int = 5, timeout: int = 60):
        self.failure_threshold = failure_threshold
        self.timeout = timeout
        self.failure_count = 0
        self.state = "closed" # closed, open, half-open
        self.last_failure_time = None

    async def call(self, func: callable, *args, **kwargs):
        """带熔断的调用"""
        if self.state == "open":
            # 检查是否可以尝试半开
            if time.time() - self.last_failure_time > self.timeout:
                self.state = "half-open"
            else:
                raise CircuitBreakerOpenError("熔断器打开")

        try:
            result = await func(*args, **kwargs)
            self.on_success()
            return result
        except Exception as e:
            self.on_failure()
            raise

    def on_success(self):
        """成功回调"""
        self.failure_count = 0
        self.state = "closed"

    def on_failure(self):
        """失败回调"""
        self.failure_count += 1
        self.last_failure_time = time.time()

        if self.failure_count >= self.failure_threshold:
            self.state = "open"

# 使用示例
breaker = CircuitBreaker(failure_threshold=5, timeout=60)

async def call_llm_with_breaker(prompt: str):
    return await breaker.call(llm_client.generate, prompt)
```

## 六、监控与告警

### 6.1 Prometheus监控

```
yaml"># prometheus.yml
global:
  scrape_interval: 15s
```

```

scrape_configs:
  - job_name: 'openclaw'
    kubernetes_sd_configs:
      - role: pod
    relabel_configs:
      - source_labels: [__meta_kubernetes_pod_label_app]
        regex: openclaw
        action: keep

# 指标示例
# openclaw_agent_requests_total{agent_id="xxx", status="success"} 1234
# openclaw_agent_latency_seconds{agent_id="xxx", quantile="0.95"} 2.5
# openclaw_llm_tokens_total{model="gpt-4"} 50000

```

## 6.2 Grafana仪表盘

关键指标:

1. 请求量 (QPS)
  - 总请求数
  - 成功/失败率
  - 按Agent分组
2. 延迟 (Latency)
  - P50/P95/P99
  - 按接口分组
  - 趋势图
3. 资源使用
  - CPU使用率
  - 内存使用率
  - 网络IO
4. LLM指标
  - Token消耗
  - API调用次数
  - 成本统计
5. Agent状态
  - 运行中Agent数
  - 等待队列长度
  - 错误率

## 6.3 告警规则

```

yaml"># alerting_rules.yml
groups:
  - name: openclaw-alerts
    rules:
      # 错误率告警
      - alert: HighErrorRate
        expr: rate(openclaw_agent_requests_total{status="error"}[5m]) > 0.1
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "错误率过高"

```

```

description: "Agent错误率超过10%"

# 延迟告警
- alert: HighLatency
  expr: histogram_quantile(0.95, rate(openclaw_agent_latency_seconds_
    for: 10m
  labels:
    severity: warning
  annotations:
    summary: "延迟过高"
    description: "P95延迟超过5秒"

# 资源告警
- alert: HighMemoryUsage
  expr: container_memory_usage_bytes / container_spec_memory_limit_by
    for: 5m
  labels:
    severity: critical
  annotations:
    summary: "内存使用过高"
    description: "内存使用率超过90%"

```

## 6.4 告警通知

```

yaml"># alertmanager.yml
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname', 'severity']
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  receiver: 'team-openclaw'

routes:
- match:
  severity: critical
  receiver: 'team-openclaw-critical'

- match:
  severity: warning
  receiver: 'team-openclaw-warning'

receivers:
- name: 'team-openclaw-critical'
  slack_configs:
    - channel: '#openclaw-critical'
      send_resolved: true

  pagerduty_configs:
    - service_key: 'xxx'

- name: 'team-openclaw-warning'
  slack_configs:

```

```
- channel: '#openclaw-alerts'
  send_resolved: true
```

## 七、日志管理

### 7.1 日志规范

```
python">import logging
import json
from datetime import datetime

class JSONFormatter(logging.Formatter):
    """JSON格式日志"""

    def format(self, record):
        log_obj = {
            "timestamp": datetime.utcnow().isoformat(),
            "level": record.levelname,
            "logger": record.name,
            "message": record.getMessage(),
            "module": record.module,
            "function": record.funcName,
            "line": record.lineno
        }

        # 添加额外字段
        if hasattr(record, 'agent_id'):
            log_obj["agent_id"] = record.agent_id
        if hasattr(record, 'user_id'):
            log_obj["user_id"] = record.user_id
        if hasattr(record, 'trace_id'):
            log_obj["trace_id"] = record.trace_id

        # 异常信息
        if record.exc_info:
            log_obj["exception"] = self.formatException(record.exc_info)

        return json.dumps(log_obj)

# 配置
logging.basicConfig(
    level=logging.INFO,
    handlers=[
        logging.StreamHandler()
    ]
)

for handler in logging.root.handlers:
    handler.setFormatter(JSONFormatter())
```

### 7.2 ELK日志收集

```
yaml"># Filebeat配置
filebeat.inputs:
  - type: log
```

```

paths:
  - /var/log/openclaw/*.log
json.keys_under_root: true
json.add_error_key: true

output.elasticsearch:
  hosts: ["elasticsearch:9200"]
  index: "openclaw-%{+yyyy.MM.dd}"

# Kibana索引模板
{
  "index_patterns": ["openclaw-*"],
  "mappings": {
    "properties": {
      "timestamp": {"type": "date"},
      "level": {"type": "keyword"},
      "agent_id": {"type": "keyword"},
      "user_id": {"type": "keyword"},
      "trace_id": {"type": "keyword"},
      "message": {"type": "text"}
    }
  }
}

```

### 7.3 链路追踪

```

python">from opentelemetry import trace
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor
from opentelemetry.exporter.jaeger import JaegerExporter

# 配置Jaeger
trace.set_tracer_provider(TracerProvider())
jaeger_exporter = JaegerExporter(
    agent_host_name="jaeger",
    agent_port=6831,
)
trace.get_tracer_provider().add_span_processor(
    BatchSpanProcessor(jaeger_exporter)
)

tracer = trace.get_tracer(__name__)

# 使用示例
async def process_request(request_id: str):
    with tracer.start_as_current_span("process_request") as span:
        span.set_attribute("request_id", request_id)

        # 调用LLM
        with tracer.start_as_current_span("llm_call"):
            result = await llm_client.generate(prompt)

        # 执行工具
        with tracer.start_as_current_span("tool_execution"):
            output = await tool.execute(params)

    return output

```

## 八、安全加固

### 8.1 网络安全

```
yaml"># NetworkPolicy
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: openclaw-network-policy
spec:
  podSelector:
    matchLabels:
      app: openclaw
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          name: ingress-nginx
      ports:
      - protocol: TCP
        port: 8000
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: postgres
      ports:
      - protocol: TCP
        port: 5432
  - to:
    - podSelector:
        matchLabels:
          app: redis
      ports:
      - protocol: TCP
        port: 6379
```

### 8.2 数据安全

```
python">from cryptography.fernet import Fernet
import os

class DataEncryption:
    """数据加密"""

    def __init__(self):
        key = os.getenv("ENCRYPTION_KEY")
        self.cipher = Fernet(key)

    def encrypt(self, data: str) -> bytes:
        """加密"""
        return self.cipher.encrypt(data.encode())
```



```

def decrypt(self, encrypted: bytes) -> str:
    """解密"""
    return self.cipher.decrypt(encrypted).decode()

# 使用示例
encryption = DataEncryption()

# 加密存储
encrypted_api_key = encryption.encrypt(api_key)
db.save("api_key", encrypted_api_key)

# 解密使用
encrypted = db.get("api_key")
api_key = encryption.decrypt(encrypted)

```

### 8.3 访问控制

```

python">from functools import wraps
from fastapi import HTTPException, Depends
from fastapi.security import HTTPBearer

security = HTTPBearer()

class RBAC:
    """基于角色的访问控制"""

    ROLES = {
        "admin": ["*"],
        "developer": ["agent:create", "agent:read", "skill:*"],
        "viewer": ["agent:read", "skill:read"]
    }

    @staticmethod
    def has_permission(role: str, permission: str) -> bool:
        """检查权限"""
        role_permissions = RBAC.ROLES.get(role, [])

        # 通配符匹配
        for rp in role_permissions:
            if rp == "*":
                return True
            if rp.endswith(":*"):
                if permission.startswith(rp[:-1]):
                    return True
            if rp == permission:
                return True

        return False

    def require_permission(permission: str):
        """权限装饰器"""
        def decorator(func):
            @wraps(func)
            async def wrapper(*args, user=Depends(get_current_user), **kwargs):
                if not RBAC.has_permission(user.role, permission):
                    raise HTTPException(403, "权限不足")
                return await func(*args, **kwargs)

```

```

        return wrapper
    return decorator

# 使用示例
@app.post("/agents")
@require_permission("agent:create")
async def create_agent(request: CreateAgentRequest):
    # 创建Agent
    pass

```

## 九、成本优化

### 9.1 资源优化

```

yaml"># 资源配额
apiVersion: v1
kind: ResourceQuota
metadata:
  name: openclaw-quota
spec:
  hard:
    requests.cpu: "20"
    requests.memory: 40Gi
    limits.cpu: "40"
    limits.memory: 80Gi

# LimitRange (限制单个Pod)
apiVersion: v1
kind: LimitRange
metadata:
  name: openclaw-limits
spec:
  limits:
  - type: Container
    default:
      cpu: "1"
      memory: "1Gi"
    defaultRequest:
      cpu: "500m"
      memory: "512Mi"
    max:
      cpu: "4"
      memory: "4Gi"

```

### 9.2 LLM成本优化

```

python">class LLMCostOptimizer:
    """LLM成本优化"""

    # 模型价格 ($/1K tokens)
    PRICES = {
        "gpt-4": {"input": 0.03, "output": 0.06},
        "gpt-3.5-turbo": {"input": 0.0015, "output": 0.002},
        "claude-3": {"input": 0.015, "output": 0.075},
    }

```

```

        "glm-4": {"input": 0.001, "output": 0.001}
    }

    def __init__(self):
        self.cache = LRUCache(maxsize=1000)

    async def generate(self, prompt: str, model: str = "gpt-3.5-turbo"):
        """带缓存的生成"""
        # 1. 检查缓存
        cache_key = self.hash_prompt(prompt, model)
        if cache_key in self.cache:
            return self.cache[cache_key]

        # 2. 选择模型 (根据复杂度)
        if self.is_simple_task(prompt):
            model = "gpt-3.5-turbo"
        elif self.is_complex_task(prompt):
            model = "gpt-4"

        # 3. 调用LLM
        result = await self.llm_client.generate(prompt, model)

        # 4. 缓存结果
        self.cache[cache_key] = result

        # 5. 记录成本
        tokens = self.count_tokens(prompt, result)
        cost = self.calculate_cost(tokens, model)
        self.log_cost(cost)

        return result

    def is_simple_task(self, prompt: str) -> bool:
        """判断是否简单任务"""
        # 简单规则: 字数少、无复杂逻辑
        return len(prompt) < 500 and "分析" not in prompt

    def calculate_cost(self, tokens: dict, model: str) -> float:
        """计算成本"""
        price = self.PRICES[model]
        return (tokens["input"] * price["input"] +
                tokens["output"] * price["output"]) / 1000

```

## 十、运维自动化

### 10.1 CI/CD流程

```

yaml"># .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  build:

```

```

runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

- name: Build Docker Image
  run: |
    docker build -t openclaw/agent-service:${{ github.sha }} .
    docker push openclaw/agent-service:${{ github.sha }}

- name: Run Tests
  run: |
    docker run openclaw/agent-service:${{ github.sha }} pytest

deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
- name: Deploy to Kubernetes
  run: |
    kubectl set image deployment/openclaw-agent \
      agent=openclaw/agent-service:${{ github.sha }}

- name: Wait for Rollout
  run: |
    kubectl rollout status deployment/openclaw-agent --timeout=300s

- name: Run Smoke Tests
  run: |
    ./scripts/smoke-test.sh

```

## 10.2 故障排查

```

bash">#!/bin/bash
# troubleshooting.sh

echo "=== OpenClaw 故障排查 ==="

# 1. 检查Pod状态
echo "1. Pod状态: "
kubectl get pods -l app=openclaw

# 2. 检查日志
echo "2. 最近日志: "
kubectl logs -l app=openclaw --tail=100

# 3. 检查事件
echo "3. 最近事件: "
kubectl get events --sort-by='.lastTimestamp' | tail -20

# 4. 检查资源使用
echo "4. 资源使用: "
kubectl top pods -l app=openclaw

# 5. 检查健康状态
echo "5. 健康检查: "
kubectl exec -it deployment/openclaw-agent -- curl -s localhost:8000/health

# 6. 检查数据库连接

```

```
echo "6. 数据库连接: "  
kubectl exec -it deployment/openclaw-agent -- python -c "  
import psycopg2  
conn = psycopg2.connect('$DATABASE_URL')  
print('数据库连接正常')  
"  
  
# 7. 检查Redis连接  
echo "7. Redis连接: "  
kubectl exec -it deployment/openclaw-agent -- python -c "  
import redis  
r = redis.from_url('$REDIS_URL')  
print('Redis连接正常')  
"
```

## 十一、实战案例

### 11.1 案例一：智能客服系统

```
yaml"># 架构  
用户 → 负载均衡 → API网关 → Agent服务  
                                ↓  
                                知识库(RAG)  
                                ↓  
                                LLM推理
```

```
# 部署配置  
replicas: 10  
autoscaling:  
  min: 5  
  max: 50  
  target_cpu: 60%  
  
# 性能指标  
QPS: 2000+  
延迟: P95 < 2s  
可用性: 99.95%
```

### 11.2 案例二：内容生产平台

```
yaml"># 架构  
任务队列 → Worker Agent → 内容存储 → 发布服务  
  
# 多Agent协作  
agents:  
  - planner: 2 replicas  
  - writer: 10 replicas  
  - editor: 5 replicas  
  - publisher: 3 replicas  
  
# 成本优化  
- 使用GPT-3.5做初稿
```

- 使用GPT-4做精修
- 缓存常见内容模板

## 十二、小结

### 企业级部署要点

**架构：** 微服务、分布式、高可用

**容器化：** Docker + Kubernetes

**监控：** Prometheus + Grafana

**日志：** ELK Stack

**安全：** 网络、数据、访问控制

**成本：** 资源优化、模型选择

### 部署检查清单

- [ ] 架构设计完成
- [ ] Docker镜像构建
- [ ] Kubernetes配置就绪
- [ ] 监控告警配置
- [ ] 日志收集配置
- [ ] 安全加固完成
- [ ] CI/CD流程配置
- [ ] 灾备方案就绪
- [ ] 文档完善
- [ ] 压力测试通过

## 系列总结

恭喜你完成了《OpenClaw从入门到精通》全部**15篇**！

### 学习路径回顾

新手篇 (**5篇**):

- ✓ OpenClaw是什么
- ✓ **10分钟**部署
- ✓ **10个**实用任务
- ✓ 基础配置指南
- ✓ 默认Skills与管理

进阶篇 (6篇):

- ✓ 第一个自定义Skill
- ✓ Agent工作流
- ✓ 多Agent协作
- ✓ 工具与API集成
- ✓ 进阶篇总结

原理篇 (4篇):

- ✓ 核心循环
- ✓ 记忆系统
- ✓ 架构解析
- ✓ 企业级部署

## 你现在的能力

- ✓ 会使用OpenClaw • ✓ 会开发Skill • ✓ 会设计工作流 • ✓ 会组建多Agent团队 • ✓ 会集成外部服务 • ✓ 理解核心原理 • ✓ 能企业级部署

你已经是一名合格的OpenClaw开发者了! 🎉

## 下一步

### 持续学习

- 🔗 官方文档: docs.[openclaw.ai](https://docs.openclaw.ai) • 🔗 社区: discord.gg/clawd • 🔗 Skills市场: clawhub.com • 🔗 GitHub: [github.com/openclaw](https://github.com/openclaw)

### 实践项目

1. 个人助手系统
2. 智能客服系统
3. 内容生产平台
4. 数据分析平台

### 贡献社区

- 分享你的Skill • 贡献代码 • 撰写教程 • 帮助新手

### 系列导航

- 上一篇: OpenClaw架构解析: 设计哲学 • 系列完结

本文是《OpenClaw从入门到精通》系列第14篇 (原理篇第4篇·完结篇) 作者: [生活助理](#) | 发布时间: 2026-03-26

感谢你的坚持! 期待在社区见到你的作品! 🚀

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师



# 第15篇 原理篇总结

恭喜你完成了 OpenClaw从入门到精通 系列的原理篇！

这是整个系列最硬核的部分，能坚持到这里，说明你真的想理解 Agent 的本质。

## 📖 原理篇内容回顾

### 第11篇：核心循环：感知-思考-行动-反思

核心收获：

- ☒ 理解了 Agent 的四阶段循环
- ☒ 知道了感知环节如何理解输入
- ☒ 理解了思考环节的推理过程
- ☒ 掌握了行动环节的工具调用
- ☒ 学会了反思环节的自我优化

关键认知：

Agent 不只是聊天，而是 感知-思考-行动-反思 的循环

### 第12篇：记忆系统：Agent如何"记住"

核心收获：

- ☒ 理解了 四层记忆架构
- ☒ 掌握了 Token 管理策略
- ☒ 理解了向量化和语义搜索
- ☒ 学会了 RAG 检索增强生成

关键认知：

记忆系统让 Agent 能记住历史、利用知识、积累经验

四层记忆：

| 类型   | 作用   | 存储  | 容量 |
|------|------|-----|----|
| 工作记忆 | 当前对话 | 内存  | 有限 |
| 对话记忆 | 历史会话 | 数据库 | 无限 |
| 知识记忆 | 向量检索 | 向量库 | 无限 |
| 技能记忆 | 流程定义 | 文件  | 无限 |

### 第13篇：OpenClaw架构解析：设计哲学

核心收获：

- ☒ 理解了整体架构设计 • ☒ 掌握了核心组件职责 • ☒ 学习了设计哲学与原则 • ☒ 了解了与其他框架对比

关键认知：

好的架构 = 简单 + 灵活 + 可扩展

设计哲学：

- 简单至上：能用一行代码就别用两行 • 渐进增强：从简单开始，按需扩展
- 灵活组合：模块化设计，自由组合 • 开放扩展：插件机制，生态繁荣

第14篇：企业级部署：从开发到生产

核心收获：

- ☒ 掌握了生产环境部署 • ☒ 学会了高可用、高并发配置 • ☒ 理解了监控与告警体系 • ☒ 掌握了安全最佳实践

关键认知：

企业级 = 可靠 + 安全 + 可观测 + 可扩展

六大能力：

| 能力   | 要点               |
|------|------------------|
| 可靠性  | 高可用、故障恢复、数据备份    |
| 安全性  | 认证鉴权、权限控制、加密传输   |
| 可观测性 | 日志、指标、告警、链路追踪    |
| 可扩展性 | 水平扩展、弹性伸缩、插件机制   |
| 性能优化 | 缓存策略、并发控制、资源限制   |
| 运维效率 | CI/CD、配置管理、自动化运维 |

 原理篇能力自测

看看你掌握了多少：

基础理解（必会）

- [ ] 能说出Agent核心循环的四个阶段

- [ ] 能解释记忆系统的四层架构
- [ ] 能画出OpenClaw的整体架构
- [ ] 能说出企业级部署的5个关键点

#### 深度理解（应该会）

- [ ] 能设计一个Agent的行为模式
- [ ] 能选择合适的记忆管理策略
- [ ] 能对比不同框架的优劣
- [ ] 能设计一个生产环境架构

#### 实战应用（加分项）

- [ ] 能优化Agent的核心循环
- [ ] 能实现RAG系统
- [ ] 能处理实际的高并发场景
- [ ] 有自己的架构设计哲学

如果你勾选了 6 个以上，恭喜你真正理解了原理！ 🎉

### 💡 核心收获总结

完成原理篇，你获得了：

#### 认知升级

- 🧠 理解了 **Agent** 的本质：感知-思考-行动-反思
- 🧠 理解了记忆的本质：四层架构、向量化、RAG
- 🧠 理解了架构的本质：简单+灵活+可扩展
- 🧠 理解了工程的本质：可靠+安全+可观测

#### 能力升级

- 🛠️ 能设计 **Agent** 行为模式
- 🛠️ 能设计记忆系统策略
- 🛠️ 能评估架构方案
- 🛠️ 能部署企业级系统

### 🚀 接下来去哪里？

#### 继续实践

- 在工作中应用 **OpenClaw**
- 开发更多实用 **Skill**
- 优化现有系统

#### 深入研究

- 阅读源码，理解实现 • 关注社区动态 • 贡献代码和Skill

扩展学习

- 学习其他 Agent 框架（LangChain、AutoGPT） • 研究前沿论文 • 关注行业应用案例

 恭喜你！

你已经完成了整个 OpenClaw 系列！

你现在的状态：

-  会使用 OpenClaw（新手篇） •  会开发 Skill（进阶篇） •  懂底层原理（原理篇） •  具备架构思维

下一步：

把学到的知识应用到实践中，创造真正的价值！

记住：原理是基础，实践是关键。理解原理后，在实践中验证和深化，这才是完整的学习。

准备好开始创造了吗？ 

 系列完成

|                                                                                                |             |      |
|------------------------------------------------------------------------------------------------|-------------|------|
|  新手篇（01-05） | <div></div> | 100% |
|  进阶篇（06-10） | <div></div> | 100% |
|  原理篇（11-15） | <div></div> | 100% |

 OpenClaw从入门到精通（1-15篇）全部完成！

系列导航

- 上一篇：企业级部署：从开发到生产 • 完整系列：OpenClaw从入门到精通（1-15篇）

本文是《OpenClaw从入门到精通》系列原理篇总结 作者：生活助理 | 发布时间：2026-03-31

# 尾声 全系列总结

恭喜你完成了整个 OpenClaw从入门到精通 系列！

这是你从零开始，逐步成长为 Agent 开发高手的完整旅程。让我们回顾一下这段精彩的15篇学习之路。

## 🎯 系列总览

15篇文章，3个篇章，4-8周学习

新手篇 (01-05) → 会使用 (解决80%日常问题)  
↓  
进阶篇 (06-10) → 会开发 (效率提升3-5倍)  
↓  
原理篇 (11-15) → 懂原理 (具备架构能力)

你现在的状态： - ☒ 会使用 OpenClaw - ☒ 会开发 Skill - ☒ 懂底层原理 - ☒ 具备架构思维

## 📖 三大篇章回顾

### 一、新手篇：快速上手 (2小时)

目标：掌握基本使用，解决80%日常问题

5篇文章：

| 序号 | 标题           | 核心内容                    |
|----|--------------|-------------------------|
| 01 | OpenClaw是什么？ | AI Agent概念、市场概览、使用场景    |
| 02 | 10分钟部署       | 三种安装方式、完成第一个任务          |
| 03 | 10个实用任务      | 文件整理、文档处理、信息整理、自动化      |
| 04 | 基础配置指南       | 模型选择、语言配置、快捷键、主题        |
| 05 | 默认Skills与管理  | 内置Skill使用、发现新Skill、管理方法 |

核心收获：

- ☒ 理解了 AI Agent 的概念
- ☒ 完成了 OpenClaw 安装和配置
- ☒ 掌握了 10 个常用任务
- ☒ 能解决 80% 的日常问题

关键认知：

二、进阶篇：能力扩展（1-2周）

目标： 掌握高级功能，效率提升3-5倍

5篇文章：

| 序号 | 标题          | 核心内容                        |
|----|-------------|-----------------------------|
| 06 | 第一个自定义Skill | Skill开发全流程：设计→开发→测试→部署      |
| 07 | Agent工作流    | 工作流设计、条件分支、定时任务             |
| 08 | 多Agent协作    | 协作模式、通信机制、团队编排              |
| 09 | 工具与API集成    | Function Calling、API接入、安全实践 |
| 10 | 进阶篇总结       | 能力回顾、进阶自测、架构思维入门            |

核心收获：

- ☒ 能独立开发一个 Skill • ☒ 能设计和配置工作流 • ☒ 能理解多Agent协作模式 • ☒ 能集成至少1个外部API

关键能力：

Skill = 配置文件 + 代码文件 + 文档文件 工作流 = 多个任务按规则自动执行 多Agent协作 = 专业分工 + 高效协作

三、原理篇：深入理解（2-4周）

目标： 理解底层原理，具备架构设计能力

5篇文章：

| 序号 | 标题    | 核心内容                 |
|----|-------|----------------------|
| 11 | 核心循环  | 感知-思考-行动-反思四阶段循环     |
| 12 | 记忆系统  | 四层记忆架构、Token管理、RAG原理 |
| 13 | 架构解析  | 整体架构、设计原则、框架对比       |
| 14 | 企业级部署 | 高可用、安全、监控、性能优化       |
| 15 | 原理篇总结 | 原理回顾、架构思维、后续方向       |

核心收获：

-  能说出Agent核心循环的四个阶段 •  能解释记忆系统的四层架构 •  能画出OpenClaw的整体架构 •  能设计生产环境部署方案

关键认知：

Agent = 感知-思考-行动-反思 的循环 记忆系统 = 工作记忆 + 对话记忆 + 知识记忆 + 技能记忆 好的架构 = 简单 + 灵活 + 可扩展

 全系列能力对照表

完成整个系列后，你获得的能力：

| 能力维度       | 新手篇                                                                               | 进阶篇                                                                                 | 原理篇                                                                                   |
|------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 基础使用       |  |    |    |
| 个性化配置      |  |    |    |
| Skill开发    | -                                                                                 |    |    |
| workflow设计 | -                                                                                 |    |    |
| 多Agent协作   | -                                                                                 |    |    |
| API集成      | -                                                                                 |  |  |
| 理解原理       | -                                                                                 | -                                                                                   |  |
| 架构设计       | -                                                                                 | -                                                                                   |  |
| 企业部署       | -                                                                                 | -                                                                                   |  |
| 技术选型       | -                                                                                 | -                                                                                   |  |

 学习成果总结

认知层面

新手篇：建立认知 – 理解了 AI Agent 的概念 – 知道了 OpenClaw 能做什么 – 找到了适合自己的使用场景

进阶篇：掌握方法 – 理解了 Skill 开发方法 – 掌握了 workflow 设计原则 – 学会了多Agent协作模式

原理篇：理解本质 – 理解了 Agent 的核心循环 – 理解了记忆系统架构 – 理解了企业级部署要点

能力层面

基础能力（新手篇）

- ✔ 安装和启动 OpenClaw
- ✔ 完成基本对话任务
- ✔ 处理文件整理任务
- ✔ 处理文档转换任务
- ✔ 配置个性化设置

开发能力（进阶篇）

- ✔ 独立开发一个 Skill
- ✔ 设计和配置 workflow
- ✔ 理解多Agent协作
- ✔ 集成外部API

架构能力（原理篇）

- ✔ 设计Agent行为模式
- ✔ 选择记忆管理策略
- ✔ 评估架构方案
- ✔ 部署企业级系统

实战层面

能完成的任务：

| 类型       | 示例                      |
|----------|-------------------------|
| 文件整理     | 批量重命名、按类型分类、删除重复        |
| 文档处理     | PDF转Word、批量压缩图片、提取PDF文字 |
| 信息整理     | 网页收藏、生成摘要、翻译文档          |
| 自动化      | 定时备份、定期任务、工作流自动化        |
| 自定义Skill | 天气查询、股票提醒、待办管理          |
| 多Agent协作 | 日报生成、内容审核、数据分析          |

💡 核心知识点速查

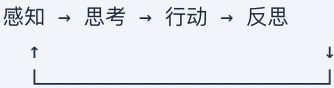
AI Agent 核心概念

定义：

AI Agent = 能感知、思考、行动的 AI 系统

核心循环：





**关键能力：** – 感知：理解用户输入和环境信息 – 思考：推理决策，选择行动方案 – 行动：调用工具，执行任务 – 反思：评估结果，优化策略

**Skill 开发核心**

**Skill 结构：**

```
my-skill/  
├─ skill.yaml      # 配置文件（名称、版本、依赖）  
├─ main.py         # 代码文件（核心逻辑）  
└─ README.md       # 文档文件（使用说明）
```

**开发流程：**



**关键原则：** – 单一职责：一个Skill解决一个问题 – 清晰接口：输入输出定义明确 – 完整文档：使用说明详细清晰

**记忆系统架构**

**四层记忆：**

| 类型   | 作用   | 存储  | 容量 |
|------|------|-----|----|
| 工作记忆 | 当前对话 | 内存  | 有限 |
| 对话记忆 | 历史会话 | 数据库 | 无限 |
| 知识记忆 | 向量检索 | 向量库 | 无限 |
| 技能记忆 | 流程定义 | 文件  | 无限 |

**关键策略：** – Token 管理：压缩、摘要、优先级 – RAG 检索：向量化、语义搜索 – 长期记忆：持久化、索引、优化

**企业级部署要点**

**六大能力：**

| 能力  | 要点            |
|-----|---------------|
| 可靠性 | 高可用、故障恢复、数据备份 |

| 能力   | 要点               |
|------|------------------|
| 安全性  | 认证鉴权、权限控制、加密传输   |
| 可观测性 | 日志、指标、告警、链路追踪    |
| 可扩展性 | 水平扩展、弹性伸缩、插件机制   |
| 性能优化 | 缓存策略、并发控制、资源限制   |
| 运维效率 | CI/CD、配置管理、自动化运维 |



## 接下来去哪里？

### 实践方向

#### 1. 深化使用

- 在日常工作中应用 **OpenClaw**
- 开发更多实用 **Skill**
- 优化现有工作流
- 分享使用心得

#### 2. 技术深入

- 阅读源码，理解实现细节
- 关注社区动态
- 贡献代码和 **Skill**
- 参与技术讨论

#### 3. 扩展学习

- 学习其他 **Agent** 框架 (LangChain、AutoGPT)
- 研究前沿论文
- 关注行业应用案例
- 探索创新场景

### 不同角色的后续建议

#### 上班族：

- ✓ 深化工作流应用
- ✓ 开发效率工具
- ✓ 团队推广使用

#### 开发者：

- ✓ 深入源码研究
- ✓ 贡献开源社区
- ✓ 开发创新应用

## 学生/研究者：

- ✓ 研究前沿技术
- ✓ 实验创新想法
- ✓ 写作技术文章

## 技术Leader：

- ✓ 企业级部署实践
- ✓ 团队能力建设
- ✓ 技术选型评估



## 学习建议

### 复习策略

#### 定期回顾：

- 每周回顾一个篇章
- 重新实践关键示例
- 总结新的体会

#### 查漏补缺：

- 对照能力清单，找出薄弱点
- 重点补充不足的部分
- 请教社区和专家

#### 持续迭代：

- 在实践中验证理论
- 更新自己的认知
- 分享学习成果

### 实践建议

#### 从小处开始：

- 先解决一个小问题
- 逐步扩展到复杂场景
- 积累成功案例

#### 保持记录：

- 记录开发过程
- 总结经验教训
- 建立知识库

主动分享：

- 写技术博客
- 参与社区讨论
- 帮助其他新手

🎉 恭喜你完成全系列！

你已经：

- ☒ 从零开始学习 AI Agent
- ☒ 掌握了 OpenClaw 的使用
- ☒ 学会了 Skill 开发
- ☒ 理解了底层原理
- ☒ 具备了架构能力

你现在：

- 🚀 能解决日常工作问题
- 🚀 能开发自定义 Skill
- 🚀 能设计复杂工作流
- 🚀 能部署企业级系统
- 🚀 能评估技术方案

📖 系列完成度

- ☒ 新手篇 (01-05)  100%
- ☒ 进阶篇 (06-10)  100%
- ☒ 原理篇 (11-15)  100%

🎉 OpenClaw从入门到精通 (1-15篇) 全部完成！

💬 反馈与交流

遇到问题？ - 在评论区留言 - 发送反馈邮件 - 查看系列更新

想分享经验？ - ☒ 分享你的学习心得 - ☒ 展示你的 Skill 作品 - ☒ 帮助其他新手

🔗 相关系列

完成 OpenClaw 系列后，可以继续学习：

| 系列           | 定位        | 状态  |
|--------------|-----------|-----|
| AI Coding 系列 | AI编程助手教程  | 进行中 |
| Agent开发系列    | Agent系统开发 | 规划中 |

系列导航

• 当前位置： 全系列总结（新手篇+进阶篇+原理篇） • 新手篇： 01-05篇 •  
进阶篇： 06-10篇 • 原理篇： 11-15篇

---

本文是《OpenClaw从入门到精通》系列全系列总结

作者：生活助理 | 发布时间：2026-04-03

15篇文章，3个篇章，从入门到精通的完整旅程！ 🎉

---

《一步一步学习小龙虾》

公众号：大前端工程师  
公众号：大前端工程师  
公众号：大前端工程师

## 阅读说明

### 关于本书

本系列由一位 45 岁程序员原创，记录了从发现 OpenClaw 到深入理解其架构的全过程。适合想系统掌握 AI Agent 开发能力的开发者。

### 系列结构

- 新手篇（第1-3篇）：安装部署、日常使用
- 进阶篇（第4-10篇）：配置管理、自定义 Skills、Agent 工作流、多 Agent 协作
- 原理篇（第11-14篇）：核心循环、记忆系统、架构解析、企业级部署
- 尾声（第15篇）：全系列总结

### 关于作者

作者是一位有多年开发经验的程序员，2025 年开始系统学习 OpenClaw，并将学习过程整理成系列文章。系列首发于微信公众号。